```
!pip install albumentations --quiet

import tensorflow as tf
from tensorflow.keras import layers, models, Model, callbacks
from tensorflow.keras.layers import (
    Input, Conv2D, BatchNormalization, Activation, Add,
    MaxPooling2D, UpSampling2D, Concatenate, Multiply, Dropout
)
import os
import numpy as np
import matplotlib.pyplot as plt
import cv2
import albumentations as A
from datetime import datetime
from sklearn.metrics import confusion_matrix
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# GPU check
print(f"TensorFlow Version: {tf.__version__}")
print(f"GPU Available: {tf.config.list_physical_devices('GPU')}")
```

```
2026-01-31 18:18:18.746692: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to
register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are
written to STDERR
E0000 00:00:1769883498.998617      55 cuda_dnn.cc:8579] Unable to
register cuDNN factory: Attempting to register factory for plugin
cuDNN when one has already been registered
E0000 00:00:1769883499.072592      55 cuda_blas.cc:1407] Unable to
register cuBLAS factory: Attempting to register factory for plugin
cuBLAS when one has already been registered
W0000 00:00:1769883499.681865      55 computation_placer.cc:177]
computation placer already registered. Please check linkage and avoid
linking the same target more than once.
W0000 00:00:1769883499.681910      55 computation_placer.cc:177]
computation placer already registered. Please check linkage and avoid
linking the same target more than once.
W0000 00:00:1769883499.681912      55 computation_placer.cc:177]
computation placer already registered. Please check linkage and avoid
linking the same target more than once.
W0000 00:00:1769883499.681915      55 computation_placer.cc:177]
computation placer already registered. Please check linkage and avoid
linking the same target more than once.

TensorFlow Version: 2.19.0
GPU Available: [PhysicalDevice(name='/physical_device:GPU:0',
```

```
device_type='GPU'), PhysicalDevice(name='/physical_device:GPU:1',
device_type='GPU')]

def clean_mask_from_path(mask_path, threshold=15, min_area=10):
    # Reads the mask image and cleans noise (pixels that appear black
but are not strictly 0)
    # Read the file in grayscale mode
    mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)

    if mask is None:
        return None

    # THRESHOLDING:
    # If a pixel value is below 'threshold' force it to 0.
    # This removes invisible noise caused by image compression (JPEG
artifacts).
    _, binary_mask = cv2.threshold(mask, threshold, 255,
cv2.THRESH_BINARY)

    # MORPHOLOGICAL CLEANING
    # Removes isolated single pixels (salt noise) that survived
thresholding
    kernel = np.ones((3,3), np.uint8)
    binary_mask = cv2.morphologyEx(binary_mask, cv2.MORPH_OPEN,
kernel, iterations=1)

    # AREA FILTERING
    # Analyzes connected white regions. If a region is smaller than
'min_area',
    # it is considered noise/dust and removed, not a real fabric
defect.
    num_labels, labels, stats, centroids =
cv2.connectedComponentsWithStats(binary_mask, connectivity=8)

    cleaned_mask = np.zeros_like(binary_mask)

    for i in range(1, num_labels): # 0 is background, start from 1
        area = stats[i, cv2.CC_STAT_AREA]
        if area >= min_area:
            cleaned_mask[labels == i] = 255

    # Normalize to 0.0 and 1.0 (float32) for the Neural Network
    return (cleaned_mask > 0).astype(np.float32)

class Config:
    DATA_PATH = ""

    # Image settings
    ORIG_WIDTH = 4096
    ORIG_HEIGHT = 256
```

```
    PATCH_SIZE = 256

    # Training settings
    BATCH_SIZE = 16
    EPOCHS = 200
    LEARNING_RATE = 1e-4

    # Data balance settings
    PATCHES_PER_IMAGE = 16  # Number of patches to extract from each
image
    DEFECT_PATCH_RATIO = 0.75  # 75% of patches will be defect-
centered
    MIN_DEFECT_PIXELS = 10  # At least 500 pixels should be defect

    # Augmentation
    AUGMENT_PROB = 0.8

    MASK_THRESHOLD = 0

# Automatically find dataset path
for root, dirs, files in os.walk("/kaggle/input"):
    if "train" in dirs and "val" in dirs:
        Config.DATA_PATH = root
        print(f"Dataset found: {Config.DATA_PATH}")
        break

if not Config.DATA_PATH:
    print("WARNING: Dataset not found. Please check /kaggle/input")
```

```
Dataset found: /kaggle/input/aitex-dataset/aitex_dataset_processed
```

```
import os
import cv2
import numpy as np
import glob

def terminate_bad_files():
    """
    Scans all mask files in the dataset to identify corrupted or
incorrectly labeled files.
    Returns a list of filenames to be excluded from training.
    """
    if not Config.DATA_PATH:
        print("ERROR: Config.DATA_PATH is not set.")
        return []

    dirs_to_check = [
        os.path.join(Config.DATA_PATH, "train", "masks"),
        os.path.join(Config.DATA_PATH, "val", "masks")
    ]
```

```
    bad_files_list = []
    total_scanned = 0

    for mask_dir in dirs_to_check:
        if not os.path.exists(mask_dir):
            print(f"Directory not found: {mask_dir}")
            continue

        mask_files = glob.glob(os.path.join(mask_dir, "*.png"))
        print(f"Found {len(mask_files)} files in {mask_dir}. Starting
analysis...")

        for mask_path in mask_files:
            total_scanned += 1
            filename = os.path.basename(mask_path)

            # Read the mask
            mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)
            if mask is None: continue

            # Noise Thresholding
            # Ignore pixels with value < 50 (sensor noise)
            _, binary_mask = cv2.threshold(mask, 50, 255,
cv2.THRESH_BINARY)
            defect_pixels = np.sum(binary_mask > 0)

            # Corruption Criteria
            # If a mask has > 2000 defect pixels, it is likely a
corrupted file
            # Real defects in this dataset are usually thin scratches
or small holes.

            if defect_pixels > 2000:
                print(f"CORRUPT FILE DETECTED: {filename} (Pixel
Count: {defect_pixels})")

                # Convert mask filename to original image filename for
the blacklist
                # '0103_002_mask.png' -> '0103_002.png'
                clean_name = filename.replace("_mask.png", ".png")

                if clean_name not in bad_files_list:
                    bad_files_list.append(clean_name)

            elif 0 < defect_pixels < 50:
                 # Small noise, will be handled by the cleaning
function later
                 pass
```

```
    print(f"Scanned {total_scanned} files.")
    print(f"FOUND {len(bad_files_list)} CORRUPT FILES.")

    print("BLACKLIST = [")
    for f in sorted(bad_files_list):
        print(f"    '{f}',")
    print("]")

    return bad_files_list

full_blacklist = terminate_bad_files()
```

```
🤓 TERMİNATÖR MODU: Klasördeki TÜM maskeler taranıyor (Dataset'ten
bağımsız)...

📂 /kaggle/input/aitex-dataset/aitex_dataset_processed/train/masks
içinde 82 dosya bulundu. Analiz başlıyor...
   ❌ BOZUK: 0073_010_03_mask.png (Hatalı Piksel: 4796)
   ❌ BOZUK: 0072_010_03_mask.png (Hatalı Piksel: 2534)
   ❌ BOZUK: 0075_010_03_mask.png (Hatalı Piksel: 16072)
   ❌ BOZUK: 0074_010_03_mask.png (Hatalı Piksel: 3029)
   ❌ BOZUK: 0010_006_02_mask.png (Hatalı Piksel: 2170)
   ❌ BOZUK: 0103_010_03_mask.png (Hatalı Piksel: 143220)
   ❌ BOZUK: 0011_006_02_mask.png (Hatalı Piksel: 2149)
   ❌ BOZUK: 0102_010_03_mask.png (Hatalı Piksel: 30545)
   ❌ BOZUK: 0016_006_02_mask.png (Hatalı Piksel: 2285)
   ❌ BOZUK: 0081_006_04_mask.png (Hatalı Piksel: 4078)
   ❌ BOZUK: 0105_010_03_mask.png (Hatalı Piksel: 5488)
   ❌ BOZUK: 0095_010_03_mask.png (Hatalı Piksel: 9481)

📂 /kaggle/input/aitex-dataset/aitex_dataset_processed/val/masks içinde
20 dosya bulundu. Analiz başlıyor...
   ❌ BOZUK: 0080_036_01_mask.png (Hatalı Piksel: 233415)
   ❌ BOZUK: 0005_002_01_mask.png (Hatalı Piksel: 2552)
   ❌ BOZUK: 0004_002_01_mask.png (Hatalı Piksel: 2035)

==========================================================================
==========
TARAMA BİTTİ. Toplam 102 dosya incelendi.
TOPLAM 15 ADET BOZUK DOSYA BULUNDU.
==========================================================================
==========

Kopyalamanız gereken liste:

BLACKLIST = [
    '0004_002_01.png',
    '0005_002_01.png',
    '0010_006_02.png',
```

```
    '0011_006_02.png',
    '0016_006_02.png',
    '0072_010_03.png',
    '0073_010_03.png',
    '0074_010_03.png',
    '0075_010_03.png',
    '0080_036_01.png',
    '0081_006_04.png',
    '0095_010_03.png',
    '0102_010_03.png',
    '0103_010_03.png',
    '0105_010_03.png',
]

train_image_dir = os.path.join(Config.DATA_PATH, "train", "images")
train_mask_dir = os.path.join(Config.DATA_PATH, "train", "masks")
val_image_dir = os.path.join(Config.DATA_PATH, "val", "images")
val_mask_dir = os.path.join(Config.DATA_PATH, "val", "masks")

# Filter valid train pairs
valid_train_mask_files = [f for f in os.listdir(train_mask_dir) if
f.endswith('.png')]
train_image_mask_pairs = []
for mask_file in valid_train_mask_files:
    image_file = mask_file.replace('_mask.png', '.png')
    image_path = os.path.join(train_image_dir, image_file)
    mask_path = os.path.join(train_mask_dir, mask_file)
    if os.path.exists(image_path):
        train_image_mask_pairs.append((image_path, mask_path))

# Filter valid val pairs
valid_val_mask_files = [f for f in os.listdir(val_mask_dir) if
f.endswith('.png')]
val_image_mask_pairs = []
for mask_file in valid_val_mask_files:
    image_file = mask_file.replace('_mask.png', '.png')
    image_path = os.path.join(val_image_dir, image_file)
    mask_path = os.path.join(val_mask_dir, mask_file)
    if os.path.exists(image_path):
        val_image_mask_pairs.append((image_path, mask_path))

# Split into separate lists
train_images = [pair[0] for pair in train_image_mask_pairs]
train_masks = [pair[1] for pair in train_image_mask_pairs]
val_images = [pair[0] for pair in val_image_mask_pairs]
val_masks = [pair[1] for pair in val_image_mask_pairs]

print(f"Training pairs: {len(train_images)}")
print(f"Validation pairs: {len(val_images)}")
```

```
Training pairs: 82
Validation pairs: 20
```

```
def extract_defect_centered_patches(image, mask, config=Config):
    h, w = image.shape[:2]
    patch_size = config.PATCH_SIZE
    num_patches = config.PATCHES_PER_IMAGE
    defect_ratio = config.DEFECT_PATCH_RATIO

    patches_img = []
    patches_mask = []
    is_defect = []

    # Identify where the defects are located
    # We use mask > 0 to catch even faint defects.
    defect_coords = np.argwhere(mask > 0)
    has_defects = len(defect_coords) > 0

    if has_defects:
        # Forced Defect Sampling
        num_defect_patches = int(num_patches * defect_ratio)

        for _ in range(num_defect_patches):
            # Pick a random defect pixel to be the center of our new
patch
            center_idx = np.random.randint(len(defect_coords))
            center_y, center_x = defect_coords[center_idx]

            # Calculate patch boundaries, ensuring we don't go outside
the image
            y_start = max(0, min(center_y - patch_size // 2, h -
patch_size))
            x_start = max(0, min(center_x - patch_size // 2, w -
patch_size))

            # Crop the image and the mask
            patch_img = image[y_start:y_start + patch_size,
                              x_start:x_start + patch_size]
            patch_mask = mask[y_start:y_start + patch_size,
                              x_start:x_start + patch_size]

            # Only keep this patch if it actually contains enough
defect pixels.
            # Sometimes a crop on the edge of a defect might miss the
defect itself.
            if np.sum(patch_mask > 0) >= config.MIN_DEFECT_PIXELS:
                patches_img.append(patch_img)
                patches_mask.append(patch_mask)
                is_defect.append(True)
```

```
    # Random Background Sampling (for context)
    # We fill the rest of the batch (e.g., 25%) with random crops.
    # This teaches the model what "clean" fabric looks like
    num_random = int(num_patches * (1 - defect_ratio)) if has_defects
else num_patches

    for _ in range(num_random):
        y_start = np.random.randint(0, max(1, h - patch_size))
        x_start = np.random.randint(0, max(1, w - patch_size))

        patch_img = image[y_start:y_start + patch_size,
                          x_start:x_start + patch_size]
        patch_mask = mask[y_start:y_start + patch_size,
                          x_start:x_start + patch_size]

        patches_img.append(patch_img)
        patches_mask.append(patch_mask)
        # Check if we accidentally cropped a defect in random mode
        is_defect.append(np.sum(patch_mask > 0) >=
config.MIN_DEFECT_PIXELS)

    return patches_img, patches_mask, is_defect

def get_augmentation_pipeline(mode='train'):
    if mode == 'train':
        return A.Compose([
            A.HorizontalFlip(p=0.5),
            A.VerticalFlip(p=0.5),
            A.Rotate(limit=15, border_mode=cv2.BORDER_CONSTANT,
value=0, p=0.3),
            A.ShiftScaleRotate(
                shift_limit=0.0625,
                scale_limit=0.1,
                rotate_limit=10,
                border_mode=cv2.BORDER_CONSTANT, # (value=0) to fill
empty space with black
                value=0,
                p=0.3
            ),

            A.RandomBrightnessContrast(
                brightness_limit=0.2,
                contrast_limit=0.2,
                p=0.5
            ),
            A.GaussNoise(var_limit=(10.0, 30.0), p=0.3),
            A.GaussianBlur(blur_limit=(3, 5), p=0.2),

            A.ElasticTransform(
                alpha=1,
```

```
                sigma=20,
                alpha_affine=20,
                border_mode=cv2.BORDER_CONSTANT,
                value=0,
                p=0.3
            ),

            A.OneOf([
                A.GridDistortion(border_mode=cv2.BORDER_CONSTANT,
value=0, p=1.0),
                A.OpticalDistortion(distort_limit=0.05,
shift_limit=0.05,
                                    border_mode=cv2.BORDER_CONSTANT,
value=0, p=1.0),
            ], p=0.2),
        ], additional_targets={'mask': 'mask'}) #apply the same
changes to mask
    else:
        return None

@tf.function
def focal_loss(y_true, y_pred, alpha=0.25, gamma=2.0):
    # FL = -α(1-p)^γ log(p)        y=1 (defect)
    # FL = -(1-α)p^γ log(1-p)      y=0 (background)

    y_pred = tf.clip_by_value(y_pred, 1e-7, 1.0 - 1e-7)

    # positive classes (defects)
    focal_pos = -alpha * tf.pow(1.0 - y_pred, gamma) * y_true *
tf.math.log(y_pred)

    # negative classes (background)
    focal_neg = -(1.0 - alpha) * tf.pow(y_pred, gamma) * (1.0 -
y_true) * tf.math.log(1.0 - y_pred)

    focal = focal_pos + focal_neg

    return tf.reduce_mean(focal)

@tf.function
def dice_loss(y_true, y_pred, smooth=1e-6):
    """
    The Dice coefficient measures the intersection/union ratio.
    It works well with unbalanced datasets because it is proportional.

    Dice = 2 * |A ∩ B| / (|A| + |B|)
    """
    y_true_f = tf.reshape(y_true, [-1])
    y_pred_f = tf.reshape(y_pred, [-1])
```

```
    intersection = tf.reduce_sum(y_true_f * y_pred_f)
    union = tf.reduce_sum(y_true_f) + tf.reduce_sum(y_pred_f)

    dice = (2.0 * intersection + smooth) / (union + smooth)

    return 1.0 - dice

@tf.function
def combined_loss(y_true, y_pred):
    """
    Combine loss: Focal + Dice

    Focal Loss: It resolves class inequality.
    Dice Loss: Measures segmentation quality.

    These two methods provide both balanced learning and good
segmentation.
    """
    focal = focal_loss(y_true, y_pred, alpha=0.25, gamma=2.0)
    dice = dice_loss(y_true, y_pred)

    return 0.5 * focal + 0.5 * dice

def conv_block(x, filters, kernel_size=3, activation='relu',
use_bn=True, dropout=0.0):
    """
    Standard convolutional block: Conv -> BN -> ReLU -> Conv -> BN ->
ReLU
    """
    x = Conv2D(filters, kernel_size, padding='same')(x)
    if use_bn:
        x = BatchNormalization()(x)
    x = Activation(activation)(x)
    if dropout > 0:
        x = Dropout(dropout)(x)

    x = Conv2D(filters, kernel_size, padding='same')(x)
    if use_bn:
        x = BatchNormalization()(x)
    x = Activation(activation)(x)

    return x

def attention_gate(x, g, filters):
    """
    Attention Gate - Learns what to focus on in the skip connection of
the decoder.

    Args:
        x: Skip connection from encoder (high resolution)
```

```
        g: Gating signal from decoder (low resolution, semantic)
        filters: Number of attention filters
    """
    # Transform skip connection
    theta_x = Conv2D(filters, kernel_size=1, padding='same')(x)  # No strides=2

    # Transform gating signal
    phi_g = Conv2D(filters, kernel_size=1, padding='same')(g)

    # Combine and activate
    concat = Add()([theta_x, phi_g])
    relu = Activation('relu')(concat)

    # Generate attention coefficients (between 0-1)
    psi = Conv2D(1, kernel_size=1, padding='same')(relu)
    psi = Activation('sigmoid')(psi)

    # Apply attention: important regions close to 1, unimportant close to 0
    output = Multiply()([x, psi])

    return output

def build_attention_unet(input_shape=(256, 256, 1), filters_base=32):
    """
    Constructs the Attention U-Net model.

    Architecture Overview:
    1. Encoder (Left side): Extracts features, reducing spatial size but increasing depth.
    2. Bottleneck (Bottom): Captures the most abstract/high-level representation.
    3. Decoder (Right side): Reconstructs the image size using UpSampling.
    4. Attention Gates: Filters the skip connections to suppress noise before merging.
    """
    inputs = Input(input_shape)

    # ENCODER (Downsampling Path)
    # Level 1: 256x256 -> 128x128
    conv1 = conv_block(inputs, filters_base, dropout=0.1)
    pool1 = MaxPooling2D((2, 2))(conv1)

    # Level 2: 128x128 -> 64x64
    conv2 = conv_block(pool1, filters_base * 2, dropout=0.1)
    pool2 = MaxPooling2D((2, 2))(conv2)

    # Level 3: 64x64 -> 32x32
```

```
    conv3 = conv_block(pool2, filters_base * 4, dropout=0.2)
    pool3 = MaxPooling2D((2, 2))(conv3)

    # Level 4: 32x32 -> 16x16
    conv4 = conv_block(pool3, filters_base * 8, dropout=0.2)
    pool4 = MaxPooling2D((2, 2))(conv4)

    # BOTTLENECK (The Bridge)
    # Level 5: 16x16. deepest layer, highest abstraction
    conv5 = conv_block(pool4, filters_base * 16, dropout=0.3)

    # DECODER (Upsampling Path with Attention)

    # Level 4 Reconstruction: 16x16 -> 32x32
    # We use Bilinear Interpolation for smoother upsampling (prevents
checkerboard artifacts)
    up6 = UpSampling2D(size=(2, 2), interpolation='bilinear')(conv5)
    # ATTENTION GATE: Filters the skip connection (conv4) using the
decoder signal (up6)
    att6 = attention_gate(x=conv4, g=up6, filters=filters_base * 8)
    merge6 = Concatenate()([up6, att6])
    conv6 = conv_block(merge6, filters_base * 8, dropout=0.2)

    # Level 3 Reconstruction: 32x32 -> 64x64
    up7 = UpSampling2D(size=(2, 2), interpolation='bilinear')(conv6)
    att7 = attention_gate(x=conv3, g=up7, filters=filters_base * 4)
    merge7 = Concatenate()([up7, att7])
    conv7 = conv_block(merge7, filters_base * 4, dropout=0.2)

    # Level 2 Reconstruction: 64x64 -> 128x128
    up8 = UpSampling2D(size=(2, 2), interpolation='bilinear')(conv7)
    att8 = attention_gate(x=conv2, g=up8, filters=filters_base * 2)
    merge8 = Concatenate()([up8, att8])
    conv8 = conv_block(merge8, filters_base * 2, dropout=0.1)

    # Level 1 Reconstruction: 128x128 -> 256x256
    up9 = UpSampling2D(size=(2, 2), interpolation='bilinear')(conv8)
    att9 = attention_gate(x=conv1, g=up9, filters=filters_base)
    merge9 = Concatenate()([up9, att9])
    conv9 = conv_block(merge9, filters_base, dropout=0.1)

    # OUTPUT LAYER
    # 1 Filter: Because we want a binary mask (Defect vs Background)
    # Sigmoid: Squashes output between 0.0 and 1.0 (Probability)
    outputs = Conv2D(1, kernel_size=1, activation='sigmoid',
name='output')(conv9)

    model = Model(inputs=[inputs], outputs=[outputs],
name='Attention_UNet')
```

```
    return model

import os
import cv2
import numpy as np

class AITexDataset:
    def __init__(self, image_paths, mask_paths, config=Config,
mode='train', ignore_list=None):
        self.image_paths = image_paths
        self.mask_paths = mask_paths
        self.config = config
        self.mode = mode

        # BLACKLIST MANAGEMENT
        # Normalize the ignore list to avoid matching errors
        if ignore_list:
            self.ignore_list = [str(x).strip() for x in ignore_list]
        else:
            self.ignore_list = []

        # Initialize Augmentation Pipeline (only if it exists and we
are in training mode)
        self.augmentation = get_augmentation_pipeline(mode) if
'get_augmentation_pipeline' in globals() else None

        self.patches = []
        self.labels = []

        self._extract_all_patches()

    def _extract_all_patches(self):
        print(f"EXTRACTING PATCHES ({self.mode.upper()}) - FILTERING
ACTIVE")
        print(f"Blacklist Size: {len(self.ignore_list)} files")

        ignored_count = 0

        for idx, (img_path, mask_path) in
enumerate(zip(self.image_paths, self.mask_paths)):
            filename = os.path.basename(img_path).strip()

            # AGGRESSIVE FILTERING LOGIC
            # Check if the filename contains any blacklisted root name
            should_ignore = False

            for bad_name in self.ignore_list:
                # Remove extension to match loosely (e.g., "file.png"
matches "file_copy.png")
```

```
                bad_root = bad_name.replace(".png", "")

                if bad_root in filename:
                    should_ignore = True
                    # Debug print for critical files
                    if "0103_010_03" in filename:
                        print(f"BLOCKED: '{filename}' (Matched
blacklist item: '{bad_name}')")
                    break

            if should_ignore:
                ignored_count += 1
                continue

            image = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

            # Use custom cleaning function (Threshold=50 eliminates
noise)
            mask = clean_mask_from_path(mask_path, threshold=50,
min_area=20)

            if image is None or mask is None: continue

            # Normalize Image (0-255 -> 0.0-1.0)
            image = image.astype(np.float32) / 255.0

            # Extract patches using the Defect-Centered Strategy
            patches_img, patches_mask, is_defect =
extract_defect_centered_patches(image, mask, self.config)

            for p_img, p_mask, defect_flag in zip(patches_img,
patches_mask, is_defect):
                self.patches.append({
                    'image': p_img,
                    'mask': p_mask,
                    'is_defect': defect_flag,
                    'source_file': filename
                })
                self.labels.append(1 if defect_flag else 0)

            if (idx + 1) % 50 == 0: print(f"Processed: {idx + 1}
images...")

        print(f"\nBlocked {ignored_count} corrupt files/copies.")

    def __len__(self): return len(self.patches)

    def get_item(self, idx):
        """
        Returns a single sample (Image + Mask) for the model.
```

```
        """
        patch = self.patches[idx]
        image = patch['image'].copy()
        mask = patch['mask'].copy()

        # Apply Augmentation (Flip, Rotate, Noise) if in Training Mode
        if self.augmentation and self.mode == 'train':
            augmented = self.augmentation(image=image, mask=mask)
            image = augmented['image']
            mask = augmented['mask']

        # Ensure mask is strictly 0.0 or 1.0
        mask = (mask > 0).astype(np.float32)

        # Add channel dimension (H, W) -> (H, W, 1) for TensorFlow
        image = np.expand_dims(image, axis=-1)
        mask = np.expand_dims(mask, axis=-1)

        return image.astype(np.float32), mask.astype(np.float32)


# INITIALIZE DATASETS

# Ensure we have the blacklist from the previous step
# If Cell 5 was not run, we define the critical list manually here as
a fallback
if 'full_blacklist' not in globals():
    BLACKLIST = [
        '0004_002_01.png', '0005_002_01.png', '0010_006_02.png',
'0011_006_02.png',
        '0016_006_02.png', '0072_010_03.png', '0073_010_03.png',
'0074_010_03.png',
        '0075_010_03.png', '0080_036_01.png', '0081_006_04.png',
'0095_010_03.png',
        '0102_010_03.png', '0103_010_03.png', '0105_010_03.png'
    ]
else:
    BLACKLIST = full_blacklist

print("Initializing Training Dataset...")
train_dataset_obj = AITexDataset(train_images, train_masks, Config,
mode='train', ignore_list=BLACKLIST)

print("Initializing Validation Dataset...")
val_dataset_obj = AITexDataset(val_images, val_masks, Config,
mode='val', ignore_list=BLACKLIST)

print("\nVerifying Data Integrity...")
leak_found = False
for item in train_dataset_obj.patches:
```

```
    if "0103_010_03" in item['source_file']:
        print(f"CRITICAL ERROR: Corrupt file leaked into dataset:
{item['source_file']}")
        leak_found = True
        break

if not leak_found:
    print("SUCCESS: Dataset is clean and ready for training.")


======================================================================
PATCH ÇIKARMA (TRAIN) - AGRESİF FİLTRELEME MODU
Yasaklı Listesi: ['0004_002_01.png', '0005_002_01.png',
'0010_006_02.png', '0011_006_02.png', '0016_006_02.png',
'0072_010_03.png', '0073_010_03.png', '0074_010_03.png',
'0075_010_03.png', '0080_036_01.png', '0081_006_04.png',
'0095_010_03.png', '0102_010_03.png', '0103_010_03.png',
'0105_010_03.png']
======================================================================
ENGELLENDİ: '0103_010_03.png' (Sebebi: '0103_010_03.png' listesinde
var)

TAMAMLANDI. Toplam 12 dosya/kopya engellendi.

======================================================================
PATCH ÇIKARMA (VAL) - AGRESİF FİLTRELEME MODU
Yasaklı Listesi: ['0004_002_01.png', '0005_002_01.png',
'0010_006_02.png', '0011_006_02.png', '0016_006_02.png',
'0072_010_03.png', '0073_010_03.png', '0074_010_03.png',
'0075_010_03.png', '0080_036_01.png', '0081_006_04.png',
'0095_010_03.png', '0102_010_03.png', '0103_010_03.png',
'0105_010_03.png']
======================================================================

TAMAMLANDI. Toplam 3 dosya/kopya engellendi.

Kontrol ediliyor...
 MÜKEMMEL: Hiçbir kaçak bulunamadı.

def verify_patches(dataset_obj, num_samples=10):
    """
    Verify that defect patches actually contain defect pixels.

    Args:
        dataset_obj: AITexDataset object
        num_samples: Number of random patches to check

    Returns:
        bool: True if all checked patches are valid
    """
```

```
    defect_indices = [i for i, label in enumerate(dataset_obj.labels)
if label == 1]

    if not defect_indices:
        print("ERROR: No defect patches found!")
        return False

    print(f"Total defect patches: {len(defect_indices)}")
    print(f"\nChecking {num_samples} random defect patches:\n")

    import random
    sample_indices = random.sample(defect_indices, min(num_samples,
len(defect_indices)))

    all_good = True
    zero_count = 0

    for idx in sample_indices:
        img, mask = dataset_obj.get_item(idx)

        # Use threshold > 0 (consistent with dataset preprocessing)
        defect_pixels = np.sum(mask > 0)

        print(f"Patch {idx}: {defect_pixels:.0f} defect pixels",
end="")

        if defect_pixels == 0:
            print(f"ERROR: Patch marked as defect but has 0 pixels!")
            all_good = False
            zero_count += 1
        elif defect_pixels < Config.MIN_DEFECT_PIXELS:
            print(f"Less than {Config.MIN_DEFECT_PIXELS} pixels")
        else:
            print(f"OK")

    if zero_count > 0:
        print(f"CRITICAL PROBLEM: {zero_count}/{num_samples} patches
have 0 defect pixels!")
        print(f"This means patch extraction is NOT working
correctly!")
    elif all_good:
        print(f"All checked patches OK!")
    else:
        print(f"Some patches below minimum threshold")
    return zero_count == 0
```

```
# UNIT TEST
verify_patches(train_dataset_obj)

def create_balanced_tf_dataset(dataset, batch_size, mode='train'):
    """
    Create a balanced TensorFlow dataset.

    Training: Oversample defect patches for 50-50 balance.
    Validation: Use as is (real distribution).
    """
    if mode == 'train':
        print(f"\n{'='*70}")
        print("CREATING BALANCED TRAINING DATASET")
        print(f"{'='*70}")

        # Separate defect and normal indices
        defect_indices = [i for i, label in enumerate(dataset.labels)
if label == 1]
        normal_indices = [i for i, label in enumerate(dataset.labels)
if label == 0]

        print(f"Original distribution:")
        print(f"  Defect: {len(defect_indices)}")
        print(f"  Normal: {len(normal_indices)}")
        print(f"  Ratio: 1:
{len(normal_indices)/max(len(defect_indices), 1):.1f}")

        # Oversample minority class (defects)
        num_defects = len(defect_indices)
        num_normals = len(normal_indices)

        if num_defects < num_normals:
            multiplier = num_normals // num_defects
            balanced_indices = defect_indices * multiplier +
normal_indices

            print(f"\nAfter balancing:")
            print(f"  Defect: {len(defect_indices) * multiplier}")
            print(f"  Normal: {len(normal_indices)}")
            print(f"  Ratio: ~1:1")
        else:
            balanced_indices = list(range(len(dataset)))

        np.random.shuffle(balanced_indices)
        indices = balanced_indices
        print(f"  Total training samples: {len(indices)}")
        print(f"{'='*70}\n")
    else:
        indices = list(range(len(dataset)))
```

```
    # Generator
    def generator():
        for idx in indices:
            yield dataset.get_item(idx)

    # TensorFlow dataset
    tf_dataset = tf.data.Dataset.from_generator(
        generator,
        output_signature=(
            tf.TensorSpec(shape=(256, 256, 1), dtype=tf.float32),
            tf.TensorSpec(shape=(256, 256, 1), dtype=tf.float32),
        )
    )

    if mode == 'train':
        tf_dataset = tf_dataset.shuffle(1000)

    tf_dataset = tf_dataset.batch(batch_size)
    tf_dataset = tf_dataset.prefetch(tf.data.AUTOTUNE)

    return tf_dataset

def calculate_metrics(y_true, y_pred, threshold=0.3):
    """
    Calculate segmentation metrics.

    Returns:
        dict: dice, iou, recall, precision, f1, tp, fp, fn, tn
    """
    y_pred_bin = (y_pred > threshold).astype(np.float32)
    y_true_f = y_true.flatten()
    y_pred_f = y_pred_bin.flatten()
    tp = np.sum(y_true_f * y_pred_f)
    fp = np.sum((1 - y_true_f) * y_pred_f)
    fn = np.sum(y_true_f * (1 - y_pred_f))
    tn = np.sum((1 - y_true_f) * (1 - y_pred_f))
    dice = (2 * tp) / (2 * tp + fp + fn + 1e-7)
    iou = tp / (tp + fp + fn + 1e-7)
    recall = tp / (tp + fn + 1e-7)
    precision = tp / (tp + fp + 1e-7)
    f1 = 2 * (precision * recall) / (precision + recall + 1e-7)
    return {
        'dice': dice,
        'iou': iou,
        'recall': recall,
        'precision': precision,
        'f1': f1,
        'tp': int(tp),
        'fp': int(fp),
        'fn': int(fn),
```

```
        'tn': int(tn)
    }

def multi_threshold_evaluation(model, val_dataset, save_path=None):
    """
    Evaluate at multiple thresholds and find the best one.
    """
    print(f"\n{'='*70}")
    print("MULTI-THRESHOLD EVALUATION")
    print(f"{'='*70}")
    y_true_all = []
    y_pred_all = []
    print("Generating predictions...")
    for images, masks in val_dataset:
        preds = model.predict(images, verbose=0)
        y_true_all.append(masks.numpy())
        y_pred_all.append(preds)
    y_true = np.concatenate(y_true_all, axis=0)
    y_pred = np.concatenate(y_pred_all, axis=0)
    thresholds = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7]
    results = []
    print(f"\n{'Threshold':<12}{'Dice':<10}{'IoU':<10}{'Recall':<10}
{'Precision':<10}{'F1':<10}")
    print("-" * 70)
    for thresh in thresholds:
        metrics = calculate_metrics(y_true, y_pred, threshold=thresh)
        results.append((thresh, metrics))
        print(f"{thresh:<12.2f}{metrics['dice']:<10.4f}
{metrics['iou']:<10.4f}"
              f"{metrics['recall']:<10.4f}
{metrics['precision']:<10.4f}{metrics['f1']:<10.4f}")
    best_thresh, best_metrics = max(results, key=lambda x: x[1]['f1'])
    print(f"\n{'='*70}")
    print(f"BEST THRESHOLD: {best_thresh:.2f}")
    print(f"  Dice: {best_metrics['dice']:.4f}")
    print(f"  IoU: {best_metrics['iou']:.4f}")
    print(f"  Recall: {best_metrics['recall']:.4f}")
    print(f"  Precision: {best_metrics['precision']:.4f}")
    print(f"  F1: {best_metrics['f1']:.4f}")
    print(f"{'='*70}\n")
    # Plot
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
    threshs = [r[0] for r in results]
    dices = [r[1]['dice'] for r in results]
    recalls = [r[1]['recall'] for r in results]
    precisions = [r[1]['precision'] for r in results]
    ax1.plot(threshs, dices, 'o-', label='Dice', linewidth=2,
markersize=8)
    ax1.plot(threshs, recalls, 's-', label='Recall', linewidth=2,
```

```
markersize=8)
    ax1.plot(threshs, precisions, '^-', label='Precision',
linewidth=2, markersize=8)
    ax1.axvline(best_thresh, color='r', linestyle='--', linewidth=2,
label=f'Best={best_thresh:.2f}')
    ax1.set_xlabel('Threshold', fontsize=12)
    ax1.set_ylabel('Score', fontsize=12)
    ax1.set_title('Metrics vs Threshold', fontsize=14,
fontweight='bold')
    ax1.legend(fontsize=11)
    ax1.grid(True, alpha=0.3)
    f1s = [r[1]['f1'] for r in results]
    ax2.plot(threshs, f1s, 'o-', color='green', linewidth=2,
markersize=8)
    ax2.axvline(best_thresh, color='r', linestyle='--', linewidth=2,
label=f'Best={best_thresh:.2f}')
    ax2.set_xlabel('Threshold', fontsize=12)
    ax2.set_ylabel('F1 Score', fontsize=12)
    ax2.set_title('F1 Score vs Threshold', fontsize=14,
fontweight='bold')
    ax2.legend(fontsize=11)
    ax2.grid(True, alpha=0.3)
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=150, bbox_inches='tight')
    plt.show()
    return best_thresh, best_metrics

def visualize_predictions(model, val_dataset, num_samples=6,
threshold=0.3):
    """
    Visualize model predictions.
    """
    for images, masks in val_dataset.take(1):
        preds = model.predict(images, verbose=0)
        n = min(num_samples, images.shape[0])
        fig, axes = plt.subplots(n, 3, figsize=(12, n * 3))
        if n == 1:
            axes = axes.reshape(1, -1)
        for i in range(n):
            axes[i, 0].imshow(images[i, :, :, 0], cmap='gray')
            axes[i, 0].set_title('Input Image', fontsize=12,
fontweight='bold')
            axes[i, 0].axis('off')
            axes[i, 1].imshow(masks[i, :, :, 0], cmap='gray')
            axes[i, 1].set_title('Ground Truth', fontsize=12,
fontweight='bold')
            axes[i, 1].axis('off')
            pred_binary = (preds[i, :, :, 0] >
threshold).astype(float)
```

```
            axes[i, 2].imshow(pred_binary, cmap='gray')
            metrics = calculate_metrics(
                masks[i:i+1].numpy(),
                preds[i:i+1],
                threshold=threshold
            )
            axes[i, 2].set_title(
                f"Prediction (Dice={metrics['dice']:.3f})",
                fontsize=12,
                fontweight='bold'
            )
            axes[i, 2].axis('off')
        plt.tight_layout()
        plt.show()
        break

def plot_training_history(history):
    """
    Visualize training history.
    """
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
    ax1.plot(history.history['loss'], label='Train Loss', linewidth=2)
    ax1.plot(history.history['val_loss'], label='Val Loss',
linewidth=2)
    ax1.set_xlabel('Epoch', fontsize=12)
    ax1.set_ylabel('Loss', fontsize=12)
    ax1.set_title('Training and Validation Loss', fontsize=14,
fontweight='bold')
    ax1.legend(fontsize=11)
    ax1.grid(True, alpha=0.3)
    ax2.plot(history.history['accuracy'], label='Train Accuracy',
linewidth=2)
    ax2.plot(history.history['val_accuracy'], label='Val Accuracy',
linewidth=2)
    ax2.set_xlabel('Epoch', fontsize=12)
    ax2.set_ylabel('Accuracy', fontsize=12)
    ax2.set_title('Training and Validation Accuracy', fontsize=14,
fontweight='bold')
    ax2.legend(fontsize=11)
    ax2.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

def build_hrnet(input_shape=(256, 256, 1), base_filters=32):
    """
    Simplified HRNet encoder-decoder for segmentation.
    Note: This is not a full HRNet, but a basic multi-resolution
block.
    """
    inputs = Input(input_shape)
```

```
    # Stage 1
    x1 = conv_block(inputs, base_filters)
    p1 = MaxPooling2D((2, 2))(x1)

    # Stage 2
    x2 = conv_block(p1, base_filters * 2)
    p2 = MaxPooling2D((2, 2))(x2)

    # Stage 3
    x3 = conv_block(p2, base_filters * 4)
    p3 = MaxPooling2D((2, 2))(x3)

    # Stage 4
    x4 = conv_block(p3, base_filters * 8)

    # Multi-resolution fusion (simple upsampling and concatenation)
    u3 = UpSampling2D((2, 2), interpolation='bilinear')(x4)
    f3 = Concatenate()([u3, x3])

    u2 = UpSampling2D((2, 2), interpolation='bilinear')(f3)
    f2 = Concatenate()([u2, x2])

    u1 = UpSampling2D((2, 2), interpolation='bilinear')(f2)
    f1 = Concatenate()([u1, x1])

    outputs = Conv2D(1, (1, 1), activation='sigmoid')(f1)

    model = Model(inputs, outputs, name='HRNet_Simple')
    return model

# Define callbacks for training
callbacks_list = [
    callbacks.ModelCheckpoint(
        'best_model.h5',
        monitor='val_loss',
        save_best_only=True,
        mode='min',
        verbose=1
    ),
    callbacks.EarlyStopping(
        monitor='val_loss',
        patience=30,
        restore_best_weights=True,
        verbose=1
    ),
    callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=10,
```

```
        min_lr=1e-7,
        verbose=1
    ),
    callbacks.CSVLogger('training_log.csv'),
]

# Prepare image and mask file lists using only valid .png mask files
train_image_dir = os.path.join(Config.DATA_PATH, "train", "images")
train_mask_dir = os.path.join(Config.DATA_PATH, "train", "masks")
val_image_dir = os.path.join(Config.DATA_PATH, "val", "images")
val_mask_dir = os.path.join(Config.DATA_PATH, "val", "masks")

# Filter valid train pairs
valid_train_mask_files = [f for f in os.listdir(train_mask_dir) if
f.endswith('.png')]
train_image_mask_pairs = []
for mask_file in valid_train_mask_files:
    image_file = mask_file.replace('_mask.png', '.png')
    image_path = os.path.join(train_image_dir, image_file)
    mask_path = os.path.join(train_mask_dir, mask_file)
    if os.path.exists(image_path):
        train_image_mask_pairs.append((image_path, mask_path))

# Filter valid val pairs
valid_val_mask_files = [f for f in os.listdir(val_mask_dir) if
f.endswith('.png')]
val_image_mask_pairs = []
for mask_file in valid_val_mask_files:
    image_file = mask_file.replace('_mask.png', '.png')
    image_path = os.path.join(val_image_dir, image_file)
    mask_path = os.path.join(val_mask_dir, mask_file)
    if os.path.exists(image_path):
        val_image_mask_pairs.append((image_path, mask_path))

# Split into separate lists
train_images = [pair[0] for pair in train_image_mask_pairs]
train_masks = [pair[1] for pair in train_image_mask_pairs]
val_images = [pair[0] for pair in val_image_mask_pairs]
val_masks = [pair[1] for pair in val_image_mask_pairs]

# Create dataset objects
train_dataset_obj = AITexDataset(train_images, train_masks, Config,
mode='train', ignore_list=BLACKLIST)
val_dataset_obj = AITexDataset(val_images, val_masks, Config,
mode='val', ignore_list=BLACKLIST)

# Create TensorFlow datasets
train_dataset = create_balanced_tf_dataset(train_dataset_obj,
Config.BATCH_SIZE, mode='train')
```

```
val_dataset = create_balanced_tf_dataset(val_dataset_obj,
Config.BATCH_SIZE, mode='val')


======================================================================
PATCH ÇIKARMA (TRAIN) - AGRESİF FİLTRELEME MODU
Yasaklı Listesi: ['0004_002_01.png', '0005_002_01.png',
'0010_006_02.png', '0011_006_02.png', '0016_006_02.png',
'0072_010_03.png', '0073_010_03.png', '0074_010_03.png',
'0075_010_03.png', '0080_036_01.png', '0081_006_04.png',
'0095_010_03.png', '0102_010_03.png', '0103_010_03.png',
'0105_010_03.png']
======================================================================
ENGELLENDİ: '0103_010_03.png' (Sebebi: '0103_010_03.png' listesinde
var)

TAMAMLANDI. Toplam 12 dosya/kopya engellendi.

======================================================================
PATCH ÇIKARMA (VAL) - AGRESİF FİLTRELEME MODU
Yasaklı Listesi: ['0004_002_01.png', '0005_002_01.png',
'0010_006_02.png', '0011_006_02.png', '0016_006_02.png',
'0072_010_03.png', '0073_010_03.png', '0074_010_03.png',
'0075_010_03.png', '0080_036_01.png', '0081_006_04.png',
'0095_010_03.png', '0102_010_03.png', '0103_010_03.png',
'0105_010_03.png']
======================================================================

TAMAMLANDI. Toplam 3 dosya/kopya engellendi.

======================================================================
CREATING BALANCED TRAINING DATASET
======================================================================
Original distribution:
  Defect: 625
  Normal: 495
  Ratio: 1:0.8
  Total training samples: 1120
======================================================================


I0000 00:00:1769883527.598215      55 gpu_device.cc:2019] Created
device /job:localhost/replica:0/task:0/device:GPU:0 with 13757 MB
memory:  -> device: 0, name: Tesla T4, pci bus id: 0000:00:04.0,
compute capability: 7.5
I0000 00:00:1769883527.604414      55 gpu_device.cc:2019] Created
device /job:localhost/replica:0/task:0/device:GPU:1 with 13757 MB
memory:  -> device: 1, name: Tesla T4, pci bus id: 0000:00:05.0,
compute capability: 7.5
```

```python
import matplotlib.pyplot as plt
import numpy as np

def simple_visualize(dataset, num_samples=5):
    """
    GÜNCELLENMİŞ VERSİYON:
    Dataset artık 3 değer döndürdüğü için (resim, maske, dosya_ismi)
    bu fonksiyon da ona göre güncellendi.
    """
    print(f"\n{'='*60}")
    print(f"DOĞAL GÖRÜNTÜLEME (DOSYA İSİMLİ) - {num_samples} ÖRNEK")
    print(f"{'='*60}")

    # Rastgele indexler seç
    if len(dataset) < num_samples:
        indices = range(len(dataset))
    else:
        indices = np.random.choice(len(dataset), num_samples,
replace=False)

    for i, idx in enumerate(indices):
        # --- DÜZELTME BURADA ---
        # Artık 3 değer geliyor, üçünü de karşılıyoruz.
        # Eğer eski dataset sınıfı hafızadaysa ve 2 değer dönerse diye
önlem alalım:
        data = dataset.get_item(idx)

        if len(data) == 3:
            image, mask, filename = data
        else:
            image, mask = data
            filename = "Dosya ismi yok"
        # -----------------------

        img_show = image[:, :, 0]
        mask_show = mask[:, :, 0]

        defect_pixels = np.sum(mask_show > 0)
        has_defect = defect_pixels > 0

        fig, axes = plt.subplots(1, 3, figsize=(15, 5))

        # 1. ORİJİNAL GÖRÜNTÜ
        axes[0].imshow(img_show, cmap='gray', vmin=0, vmax=1)
        # Başlığa dosya ismini de ekledik, kontrol için harika olur
        axes[0].set_title(f"Örnek #{idx}\n{filename}\nHam Görüntü",
fontsize=10, color='blue')
        axes[0].axis('off')

        # 2. MASKE
```

```
        axes[1].imshow(mask_show, cmap='gray', vmin=0, vmax=1)
        axes[1].set_title(f"Maske\n({int(defect_pixels)} px)",
fontsize=11)
        axes[1].axis('off')

        # 3. ÇAKIŞTIRMA
        axes[2].imshow(img_show, cmap='gray', vmin=0, vmax=1)
        if has_defect:
            overlay = np.zeros((img_show.shape[0], img_show.shape[1],
4))
            overlay[mask_show > 0] = [1, 0, 0, 0.4]
            axes[2].imshow(overlay)
            status = "HATA VAR"
        else:
            status = "TEMİZ"

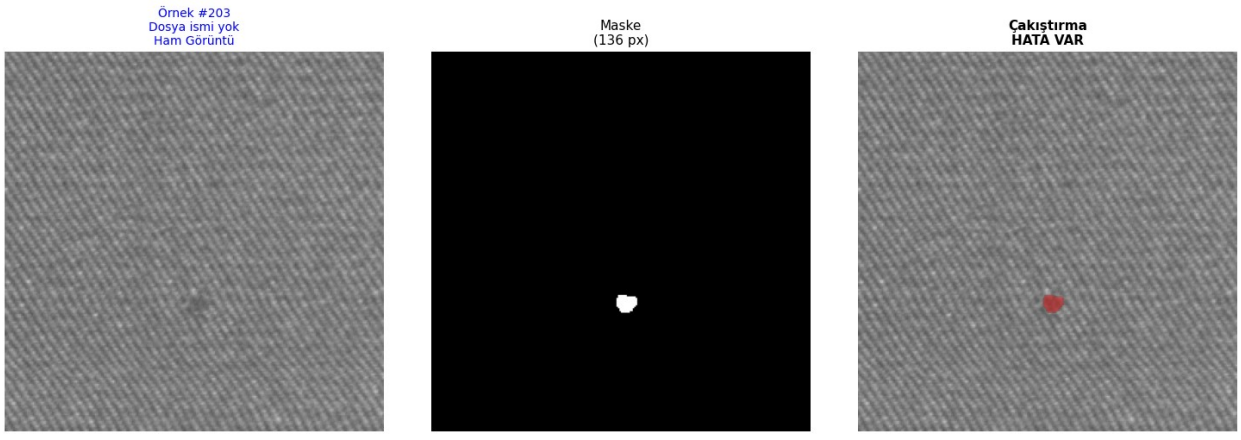
        axes[2].set_title(f"Çakıştırma\n{status}", fontsize=11,
fontweight='bold')
        axes[2].axis('off')

        plt.tight_layout()
        plt.show()

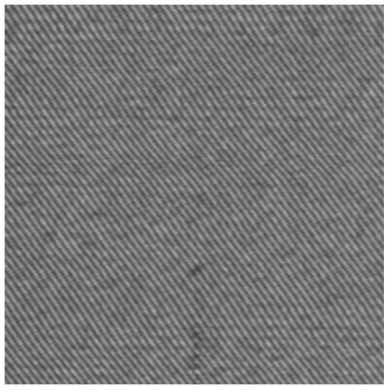
# Şimdi tekrar çalıştırabilirsin
print("Validation Setinden Örnekler:")
simple_visualize(val_dataset_obj, num_samples=5)
```

```
Validation Setinden Örnekler:

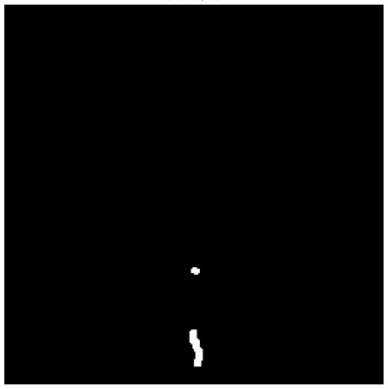
============================================================
DOĞAL GÖRÜNTÜLEME (DOSYA İSİMLİ) - 5 ÖRNEK
============================================================
```



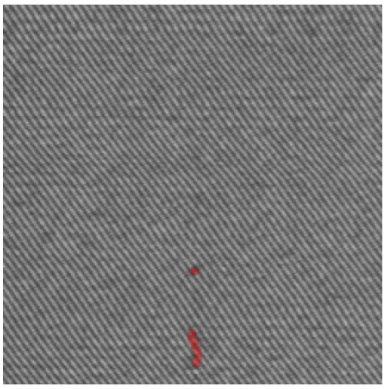
Örnek #100
Dosya ismi yok
Ham Görüntü



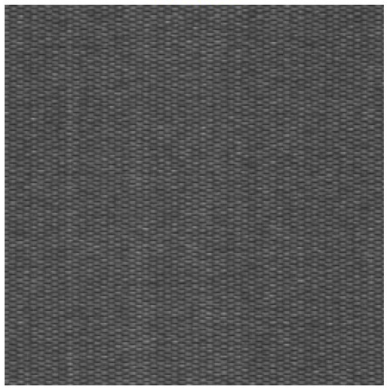
Maske
(158 px)



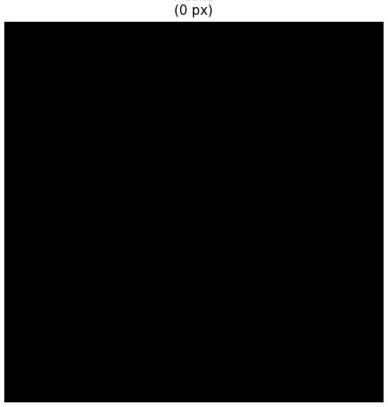
Çakıştırma
HATA VAR



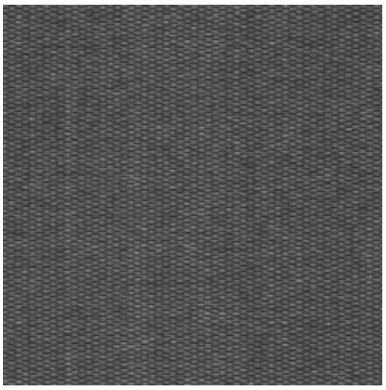
Örnek #76
Dosya ismi yok
Ham Görüntü



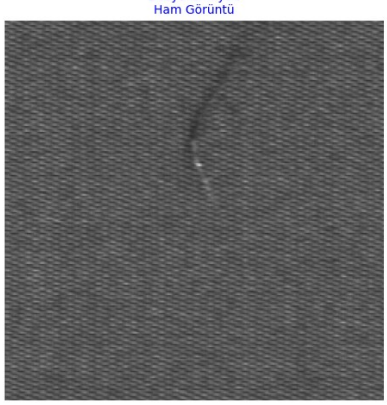
Maske
(0 px)



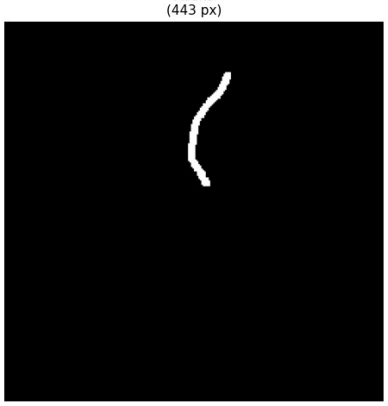
Çakıştırma
TEMİZ



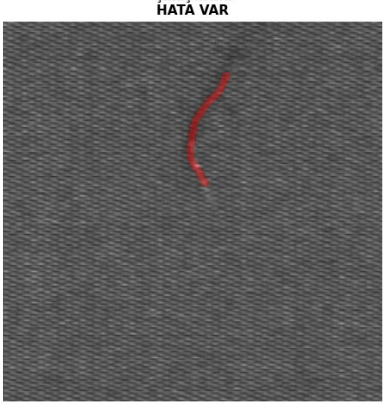
Örnek #86
Dosya ismi yok
Ham Görüntü

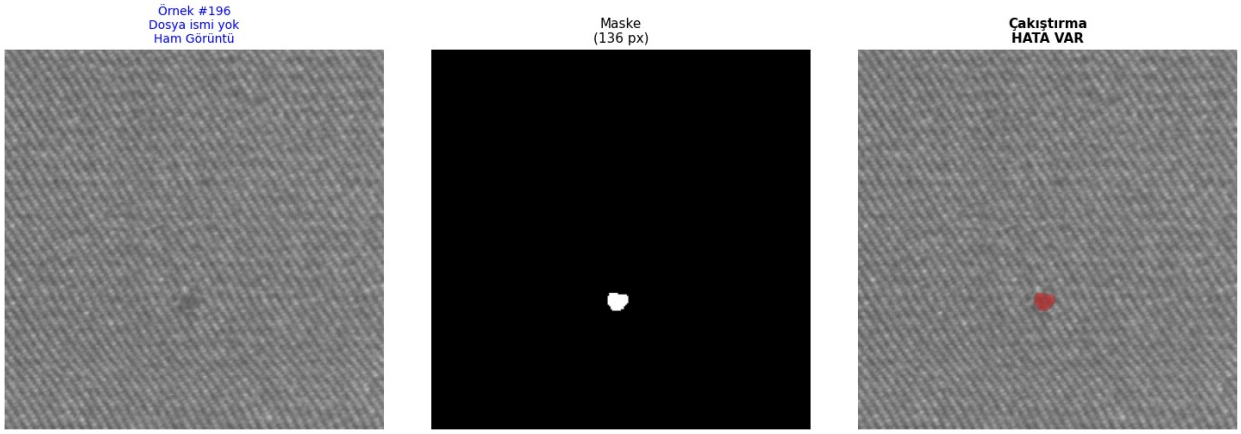


Maske
(443 px)



Çakıştırma
HATA VAR






```
import matplotlib.pyplot as plt
import numpy as np

def visualize_with_filename(dataset_obj, num_samples=5):
    """
    Bu fonksiyon artık hem 2'li (Eğitim Modu) hem 3'lü (Debug Modu)
    dönüşleri destekler. Hata vermez.
    """
    defect_indices = [i for i, label in enumerate(dataset_obj.labels)
if label == 1]
    if not defect_indices:
        print("Dataset'te hiç hata (defect) kalmamış!")
        return

    # En çok hatalı olanları bul
    defect_counts = []
    for idx in defect_indices:
        # --- GÜVENLİ VERİ ALMA ---
        data = dataset_obj.get_item(idx)
        if len(data) == 3:
            _, mask, _ = data
        else:
            _, mask = data

        count = np.sum(mask > 0)
        defect_counts.append((idx, count))

    defect_counts.sort(key=lambda x: x[1], reverse=True)

    print(f"\nEN ÇOK HATALI GÖRÜNEN {num_samples} ÖRNEK (TEMİZLENMİŞ
VERİ SETİNDEN):\n")

    for i in range(min(num_samples, len(defect_counts))):
        idx, count = defect_counts[i]

        # --- TEKRAR GÜVENLİ VERİ ALMA ---
```

```
        data = dataset_obj.get_item(idx)
        if len(data) == 3:
            img, mask, filename = data
        else:
            img, mask = data
            filename = "İsim Yok (Eğitim Modu)" # 2 değer döndüğü için
isim elimizde yok

        # Grafik
        fig, axes = plt.subplots(1, 3, figsize=(15, 5))

        # Resim
        axes[0].imshow(img[:,:,0], cmap='gray', vmin=0, vmax=1)
        axes[0].set_title(f"Patch #{idx}\n{filename}", fontsize=11,
color='green')
        axes[0].axis('off')

        # Maske
        axes[1].imshow(mask[:,:,0], cmap='gray', vmin=0, vmax=1)
        axes[1].set_title(f"Maske ({count:.0f} px)\nMax Değer:
{mask.max():.0f}", fontsize=10)
        axes[1].axis('off')

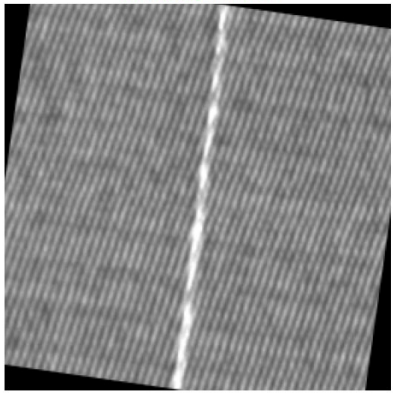
        # Çakıştırma
        axes[2].imshow(img[:,:,0], cmap='gray', vmin=0, vmax=1)
        overlay = np.zeros((256, 256, 4))
        overlay[mask[:,:,0] > 0] = [1, 0, 0, 0.5]
        axes[2].imshow(overlay)
        axes[2].set_title("Çakıştırma", fontsize=10)
        axes[2].axis('off')

        plt.tight_layout()
        plt.show()

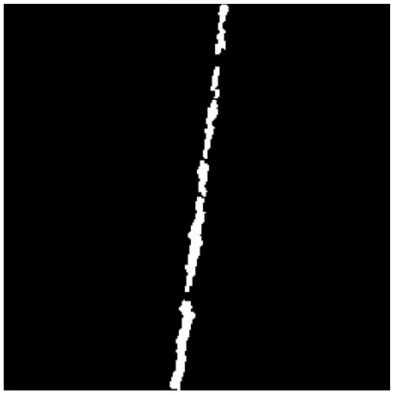
# Kodu Çalıştır
visualize_with_filename(train_dataset_obj, num_samples=5)


EN ÇOK HATALI GÖRÜNEN 5 ÖRNEK (TEMİZLENMİŞ VERİ SETİNDEN):
```

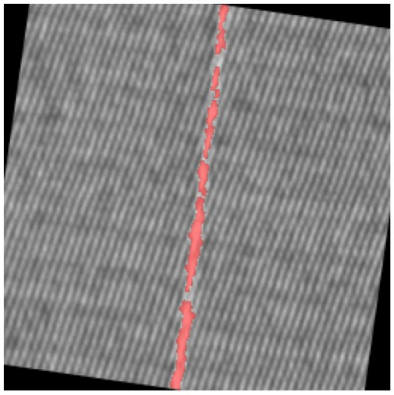
Patch #905
İsim Yok (Eğitim Modu)



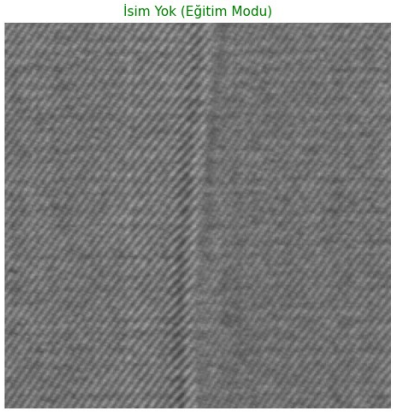
Maske (1535 px)
Max Değer: 1



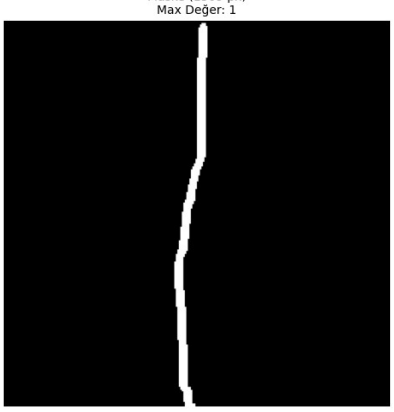
Çakıştırma



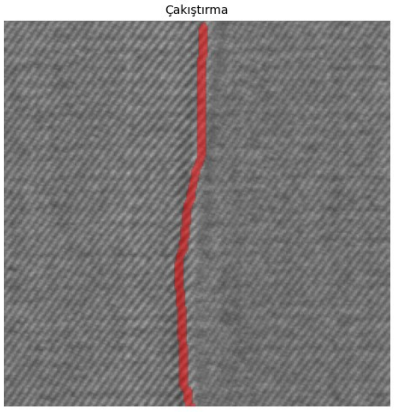
Patch #555
İsim Yok (Eğitim Modu)



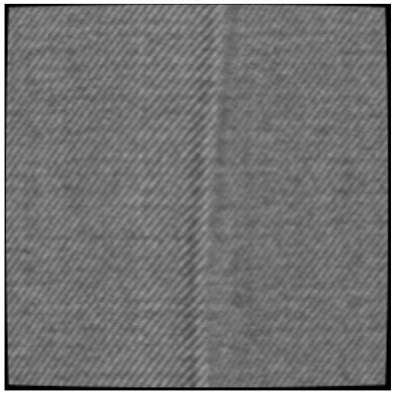
Maske (1509 px)
Max Değer: 1



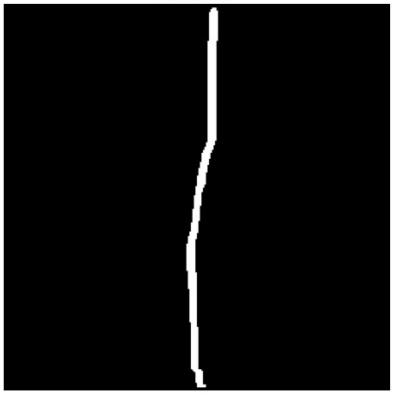
Çakıştırma



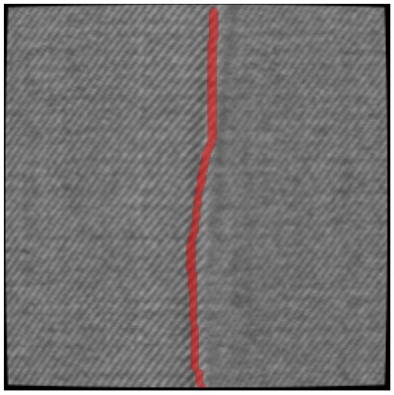
Patch #552
İsim Yok (Eğitim Modu)

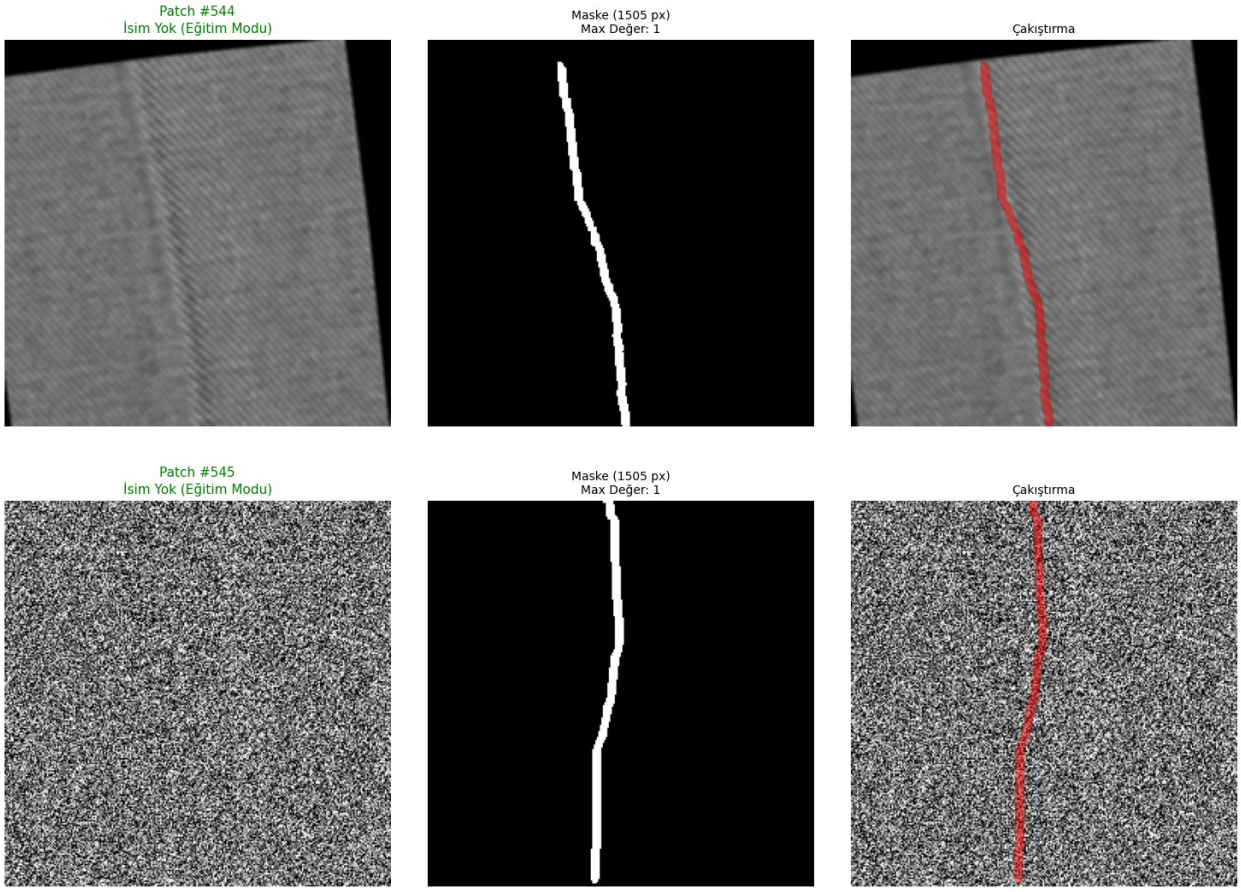


Maske (1506 px)
Max Değer: 1



Çakıştırma






```
# Custom metrics for Keras
def dice_coef(y_true, y_pred, smooth=1e-6):
    y_true_f = tf.reshape(y_true, [-1])
    y_pred_f = tf.reshape(y_pred, [-1])
    intersection = tf.reduce_sum(y_true_f * y_pred_f)
    return (2. * intersection + smooth) / (tf.reduce_sum(y_true_f) +
tf.reduce_sum(y_pred_f) + smooth)

def dice_loss(y_true, y_pred):
    return 1 - dice_coef(y_true, y_pred)

def bce_dice_loss(y_true, y_pred):
    # Hem binary cross entropy (genel dağılım)
    # Hem de dice loss (kesişim başarısı) beraber kullanılır
    bce = tf.keras.losses.binary_crossentropy(y_true, y_pred)
    dice = dice_loss(y_true, y_pred)
    return bce + dice

def iou_coef(y_true, y_pred, smooth=1e-6):
    y_true_f = tf.reshape(y_true, [-1])
    y_pred_f = tf.reshape(y_pred, [-1])
    intersection = tf.reduce_sum(y_true_f * y_pred_f)
    union = tf.reduce_sum(y_true_f) + tf.reduce_sum(y_pred_f) -
```

```
intersection
    return (intersection + smooth) / (union + smooth)

def recall_m(y_true, y_pred):
    y_true_f = tf.reshape(y_true, [-1])
    y_pred_f = tf.reshape(y_pred, [-1])
    true_positives = tf.reduce_sum(y_true_f * y_pred_f)
    possible_positives = tf.reduce_sum(y_true_f)
    return true_positives / (possible_positives + 1e-6)

def precision_m(y_true, y_pred):
    y_true_f = tf.reshape(y_true, [-1])
    y_pred_f = tf.reshape(y_pred, [-1])
    true_positives = tf.reduce_sum(y_true_f * y_pred_f)
    predicted_positives = tf.reduce_sum(y_pred_f)
    return true_positives / (predicted_positives + 1e-6)

def f1_m(y_true, y_pred):
    precision = precision_m(y_true, y_pred)
    recall = recall_m(y_true, y_pred)
    return 2*((precision*recall)/(precision+recall+1e-6))

# Veri Kontrolü
sample_img, sample_mask = train_dataset_obj.get_item(0)
print(f"Maske Min Değeri: {sample_mask.min()}")
print(f"Maske Max Değeri: {sample_mask.max()}")

if sample_mask.max() > 1.0:
    print("HATA: Maske hala 255 (veya büyük) değer içeriyor!")
else:
    print("BAŞARILI: Maske 0-1 aralığında.")
```

```
Maske Min Değeri: 0.0
Maske Max Değeri: 1.0
BAŞARILI: Maske 0-1 aralığında.
```

```
# Example: Training and comparing Attention U-Net and HRNet

# Build datasets
# (Assume train_dataset and val_dataset are already created as in your
pipeline)

# Build models
attention_unet_model = build_attention_unet(input_shape=(256, 256, 1),
filters_base=32)
hrnet_model = build_hrnet(input_shape=(Config.PATCH_SIZE,
Config.PATCH_SIZE, 1), base_filters=32)

# Compile models
attention_unet_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
```

```
    loss=bce_dice_loss,
    metrics=['accuracy', dice_coef]
)

hrnet_model.compile(

optimizer=tf.keras.optimizers.Adam(learning_rate=Config.LEARNING_RATE)
,
    loss=bce_dice_loss,
    metrics=['accuracy', dice_coef]
)


# Train Attention U-Net
print("\nTraining Attention U-Net model...")
history_attention_unet = attention_unet_model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=Config.EPOCHS,
    callbacks=callbacks_list,
    verbose=1
)

# Train HRNet
print("\nTraining HRNet model...")
history_hrnet = hrnet_model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=Config.EPOCHS,
    callbacks=callbacks_list,
    verbose=1
)

# Plot training histories
print("\nPlotting Attention U-Net training history...")
plot_training_history(history_attention_unet)
print("\nPlotting HRNet training history...")
plot_training_history(history_hrnet)

# Evaluate and compare
print("\nEvaluating Attention U-Net model...")
best_thresh_unet, best_metrics_unet =
multi_threshold_evaluation(attention_unet_model, val_dataset)
print("\nEvaluating HRNet model...")
best_thresh_hrnet, best_metrics_hrnet =
multi_threshold_evaluation(hrnet_model, val_dataset)

print("\nAttention U-Net Best Metrics:", best_metrics_unet)
print("HRNet Best Metrics:", best_metrics_hrnet)
```

```
Training Attention U-Net model...
Epoch 1/200

WARNING: All log messages before absl::InitializeLog() is called are
written to STDERR
I0000 00:00:1769883554.561872     133 service.cc:152] XLA service
0x7e6e64013ac0 initialized for platform CUDA (this does not guarantee
that XLA will be used). Devices:
I0000 00:00:1769883554.561915     133 service.cc:160]   StreamExecutor
device (0): Tesla T4, Compute Capability 7.5
I0000 00:00:1769883554.561922     133 service.cc:160]   StreamExecutor
device (1): Tesla T4, Compute Capability 7.5
I0000 00:00:1769883557.196855     133 cuda_dnn.cc:529] Loaded cuDNN
version 91002
2026-01-31 18:19:33.495273: E
external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay
kernel timed out: measured time has sub-optimal accuracy. There may be
a missing warmup execution, please investigate in Nsight Systems.
2026-01-31 18:19:33.792417: E
external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay
kernel timed out: measured time has sub-optimal accuracy. There may be
a missing warmup execution, please investigate in Nsight Systems.
2026-01-31 18:19:35.841011: E
external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay
kernel timed out: measured time has sub-optimal accuracy. There may be
a missing warmup execution, please investigate in Nsight Systems.
2026-01-31 18:19:36.216848: E
external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay
kernel timed out: measured time has sub-optimal accuracy. There may be
a missing warmup execution, please investigate in Nsight Systems.
2026-01-31 18:19:36.235504: E
external/local_xla/xla/service/slow_operation_alarm.cc:73] Trying
algorithm eng4{k11=1} for conv %cudnn-conv-bw-input.20 =
(f32[16,192,128,128]{3,2,1,0}, u8[0]{0}) custom-
call(f32[16,64,128,128]{3,2,1,0} %bitcast.51860, f32[64,192,3,3]
{3,2,1,0} %bitcast.43412), window={size=3x3 pad=1_1x1_1},
dim_labels=bf01_oi01->bf01,
custom_call_target="__cudnn$convBackwardInput",
metadata={op_type="Conv2DBackpropInput"
op_name="gradient_tape/Attention_UNet_1/conv2d_23_1/convolution/Conv2D
BackpropInput"
source_file="/usr/local/lib/python3.12/dist-packages/tensorflow/python
/framework/ops.py" source_line=1200},
backend_config={"operation_queue_id":"0","wait_on_operation_queues":
[],"cudnn_conv_backend_config":
{"conv_result_scale":1,"activation_mode":"kNone","side_input_scale":0,
"leakyrelu_alpha":0},"force_earliest_schedule":false} is taking a
while...
2026-01-31 18:19:36.242159: E
```

```
external/local_xla/xla/service/slow_operation_alarm.cc:140] The
operation took 1.006772468s
Trying algorithm eng4{k11=1} for conv %cudnn-conv-bw-input.20 =
(f32[16,192,128,128]{3,2,1,0}, u8[0]{0}) custom-
call(f32[16,64,128,128]{3,2,1,0} %bitcast.51860, f32[64,192,3,3]
{3,2,1,0} %bitcast.43412), window={size=3x3 pad=1_1x1_1},
dim_labels=bf01_oi01->bf01,
custom_call_target="__cudnn$convBackwardInput",
metadata={op_type="Conv2DBackpropInput"
op_name="gradient_tape/Attention_UNet_1/conv2d_23_1/convolution/Conv2D
BackpropInput"
source_file="/usr/local/lib/python3.12/dist-packages/tensorflow/python
/framework/ops.py" source_line=1200},
backend_config={"operation_queue_id":"0","wait_on_operation_queues":
[],"cudnn_conv_backend_config":
{"conv_result_scale":1,"activation_mode":"kNone","side_input_scale":0,
"leakyrelu_alpha":0},"force_earliest_schedule":false} is taking a
while...
2026-01-31 18:19:37.018790: E
external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay
kernel timed out: measured time has sub-optimal accuracy. There may be
a missing warmup execution, please investigate in Nsight Systems.
2026-01-31 18:19:37.202058: E
external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay
kernel timed out: measured time has sub-optimal accuracy. There may be
a missing warmup execution, please investigate in Nsight Systems.
2026-01-31 18:19:38.208480: E
external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay
kernel timed out: measured time has sub-optimal accuracy. There may be
a missing warmup execution, please investigate in Nsight Systems.
2026-01-31 18:19:38.473318: E
external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay
kernel timed out: measured time has sub-optimal accuracy. There may be
a missing warmup execution, please investigate in Nsight Systems.
2026-01-31 18:19:43.037332: E
external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay
kernel timed out: measured time has sub-optimal accuracy. There may be
a missing warmup execution, please investigate in Nsight Systems.
2026-01-31 18:19:43.200653: E
external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay
kernel timed out: measured time has sub-optimal accuracy. There may be
a missing warmup execution, please investigate in Nsight Systems.
2026-01-31 18:19:55.029778: E
external/local_xla/xla/service/slow_operation_alarm.cc:73] Trying
algorithm eng0{} for conv %cudnn-conv-bw-filter.59 = (f32[32,96,3,3]
{3,2,1,0}, u8[0]{0}) custom-call(f32[16,96,256,256]{3,2,1,0}
%bitcast.52599, f32[16,32,256,256]{3,2,1,0} %bitcast.52663),
window={size=3x3 pad=1_1x1_1}, dim_labels=bf01_oi01->bf01,
custom_call_target="__cudnn$convBackwardFilter",
```

```
metadata={op_type="Conv2DBackpropFilter"
op_name="gradient_tape/Attention_UNet_1/conv2d_28_1/convolution/Conv2D
BackpropFilter"
source_file="/usr/local/lib/python3.12/dist-packages/tensorflow/python
/framework/ops.py" source_line=1200},
backend_config={"operation_queue_id":"0","wait_on_operation_queues":
[],"cudnn_conv_backend_config":
{"conv_result_scale":1,"activation_mode":"kNone","side_input_scale":0,
"leakyrelu_alpha":0},"force_earliest_schedule":false} is taking a
while...
2026-01-31 18:19:55.429161: E
external/local_xla/xla/service/slow_operation_alarm.cc:140] The
operation took 1.399574563s
Trying algorithm eng0{} for conv %cudnn-conv-bw-filter.59 =
(f32[32,96,3,3]{3,2,1,0}, u8[0]{0}) custom-call(f32[16,96,256,256]
{3,2,1,0} %bitcast.52599, f32[16,32,256,256]{3,2,1,0} %bitcast.52663),
window={size=3x3 pad=1_1x1_1}, dim_labels=bf01_oi01->bf01,
custom_call_target="__cudnn$convBackwardFilter",
metadata={op_type="Conv2DBackpropFilter"
op_name="gradient_tape/Attention_UNet_1/conv2d_28_1/convolution/Conv2D
BackpropFilter"
source_file="/usr/local/lib/python3.12/dist-packages/tensorflow/python
/framework/ops.py" source_line=1200},
backend_config={"operation_queue_id":"0","wait_on_operation_queues":
[],"cudnn_conv_backend_config":
{"conv_result_scale":1,"activation_mode":"kNone","side_input_scale":0,
"leakyrelu_alpha":0},"force_earliest_schedule":false} is taking a
while...
I0000 00:00:1769883608.622790     133 device_compiler.h:188] Compiled
cluster using XLA!  This line is logged at most once for the lifetime
of the process.

     70/Unknown 104s 453ms/step - accuracy: 0.4932 - dice_coef: 0.0048
- loss: 1.7957
Epoch 1: val_loss improved from inf to 1.53951, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 110s 540ms/step - accuracy: 0.4972 -
dice_coef: 0.0048 - loss: 1.7929 - val_accuracy: 0.9930 -
val_dice_coef: 0.0060 - val_loss: 1.5395 - learning_rate: 1.0000e-04
Epoch 2/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 473ms/step - accuracy: 0.9935 -
dice_coef: 0.0053 - loss: 1.3888
```

```
Epoch 2: val_loss improved from 1.53951 to 1.41482, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 523ms/step - accuracy: 0.9935 -
dice_coef: 0.0053 - loss: 1.3885 - val_accuracy: 0.9969 -
val_dice_coef: 0.0061 - val_loss: 1.4148 - learning_rate: 1.0000e-04
Epoch 3/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 488ms/step - accuracy: 0.9955 -
dice_coef: 0.0050 - loss: 1.3358
Epoch 3: val_loss improved from 1.41482 to 1.32264, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 535ms/step - accuracy: 0.9955 -
dice_coef: 0.0050 - loss: 1.3358 - val_accuracy: 0.9969 -
val_dice_coef: 0.0060 - val_loss: 1.3226 - learning_rate: 1.0000e-04
Epoch 4/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 503ms/step - accuracy: 0.9962 -
dice_coef: 0.0050 - loss: 1.3055
Epoch 4: val_loss did not improve from 1.32264
70/70 ━━━━━━━━━━━━━━━━━━━━ 44s 543ms/step - accuracy: 0.9962 -
dice_coef: 0.0050 - loss: 1.3054 - val_accuracy: 0.9969 -
val_dice_coef: 0.0060 - val_loss: 1.3269 - learning_rate: 1.0000e-04
Epoch 5/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 507ms/step - accuracy: 0.9949 -
dice_coef: 0.0049 - loss: 1.2855
Epoch 5: val_loss improved from 1.32264 to 1.29226, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 555ms/step - accuracy: 0.9949 -
dice_coef: 0.0049 - loss: 1.2854 - val_accuracy: 0.9969 -
val_dice_coef: 0.0061 - val_loss: 1.2923 - learning_rate: 1.0000e-04
Epoch 6/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 534ms/step - accuracy: 0.9966 -
```

```
dice_coef: 0.0052 - loss: 1.2582
Epoch 6: val_loss improved from 1.29226 to 1.26650, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 46s 582ms/step - accuracy: 0.9966 -
dice_coef: 0.0052 - loss: 1.2581 - val_accuracy: 0.9969 -
val_dice_coef: 0.0061 - val_loss: 1.2665 - learning_rate: 1.0000e-04
Epoch 7/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 514ms/step - accuracy: 0.9972 -
dice_coef: 0.0046 - loss: 1.2362
Epoch 7: val_loss improved from 1.26650 to 1.24791, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 562ms/step - accuracy: 0.9972 -
dice_coef: 0.0047 - loss: 1.2362 - val_accuracy: 0.9969 -
val_dice_coef: 0.0061 - val_loss: 1.2479 - learning_rate: 1.0000e-04
Epoch 8/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 527ms/step - accuracy: 0.9972 -
dice_coef: 0.0049 - loss: 1.2201
Epoch 8: val_loss improved from 1.24791 to 1.23285, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 46s 575ms/step - accuracy: 0.9972 -
dice_coef: 0.0049 - loss: 1.2200 - val_accuracy: 0.9969 -
val_dice_coef: 0.0061 - val_loss: 1.2328 - learning_rate: 1.0000e-04
Epoch 9/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9968 -
dice_coef: 0.0057 - loss: 1.2082
Epoch 9: val_loss improved from 1.23285 to 1.22063, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
```

```
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 569ms/step - accuracy: 0.9968 -
dice_coef: 0.0056 - loss: 1.2082 - val_accuracy: 0.9969 -
val_dice_coef: 0.0059 - val_loss: 1.2206 - learning_rate: 1.0000e-04
Epoch 10/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9970 -
dice_coef: 0.0055 - loss: 1.1938
Epoch 10: val_loss improved from 1.22063 to 1.20138, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 46s 570ms/step - accuracy: 0.9970 -
dice_coef: 0.0055 - loss: 1.1937 - val_accuracy: 0.9969 -
val_dice_coef: 0.0057 - val_loss: 1.2014 - learning_rate: 1.0000e-04
Epoch 11/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9972 -
dice_coef: 0.0052 - loss: 1.1820
Epoch 11: val_loss improved from 1.20138 to 1.19073, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 46s 570ms/step - accuracy: 0.9972 -
dice_coef: 0.0052 - loss: 1.1820 - val_accuracy: 0.9969 -
val_dice_coef: 0.0060 - val_loss: 1.1907 - learning_rate: 1.0000e-04
Epoch 12/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9973 -
dice_coef: 0.0052 - loss: 1.1698
Epoch 12: val_loss improved from 1.19073 to 1.17980, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 569ms/step - accuracy: 0.9973 -
dice_coef: 0.0052 - loss: 1.1698 - val_accuracy: 0.9969 -
```

```
val_dice_coef: 0.0059 - val_loss: 1.1798 - learning_rate: 1.0000e-04
Epoch 13/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9973 -
dice_coef: 0.0054 - loss: 1.1591
Epoch 13: val_loss improved from 1.17980 to 1.17187, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 46s 572ms/step - accuracy: 0.9973 -
dice_coef: 0.0054 - loss: 1.1590 - val_accuracy: 0.9969 -
val_dice_coef: 0.0058 - val_loss: 1.1719 - learning_rate: 1.0000e-04
Epoch 14/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9974 -
dice_coef: 0.0050 - loss: 1.1494
Epoch 14: val_loss improved from 1.17187 to 1.16581, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 569ms/step - accuracy: 0.9974 -
dice_coef: 0.0050 - loss: 1.1494 - val_accuracy: 0.9960 -
val_dice_coef: 0.0058 - val_loss: 1.1658 - learning_rate: 1.0000e-04
Epoch 15/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9972 -
dice_coef: 0.0055 - loss: 1.1404
Epoch 15: val_loss did not improve from 1.16581
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9972 -
dice_coef: 0.0055 - loss: 1.1404 - val_accuracy: 0.9946 -
val_dice_coef: 0.0060 - val_loss: 1.1702 - learning_rate: 1.0000e-04
Epoch 16/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 520ms/step - accuracy: 0.9975 -
dice_coef: 0.0050 - loss: 1.1345
Epoch 16: val_loss improved from 1.16581 to 1.13858, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.
```

```
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 568ms/step - accuracy: 0.9975 -
dice_coef: 0.0050 - loss: 1.1345 - val_accuracy: 0.9969 -
val_dice_coef: 0.0060 - val_loss: 1.1386 - learning_rate: 1.0000e-04
Epoch 17/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9974 -
dice_coef: 0.0051 - loss: 1.1255
Epoch 17: val_loss improved from 1.13858 to 1.13223, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 46s 570ms/step - accuracy: 0.9974 -
dice_coef: 0.0051 - loss: 1.1254 - val_accuracy: 0.9969 -
val_dice_coef: 0.0060 - val_loss: 1.1322 - learning_rate: 1.0000e-04
Epoch 18/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9974 -
dice_coef: 0.0053 - loss: 1.1179
Epoch 18: val_loss improved from 1.13223 to 1.11997, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 569ms/step - accuracy: 0.9974 -
dice_coef: 0.0053 - loss: 1.1178 - val_accuracy: 0.9969 -
val_dice_coef: 0.0062 - val_loss: 1.1200 - learning_rate: 1.0000e-04
Epoch 19/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9975 -
dice_coef: 0.0052 - loss: 1.1109
Epoch 19: val_loss improved from 1.11997 to 1.10735, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 569ms/step - accuracy: 0.9975 -
dice_coef: 0.0052 - loss: 1.1109 - val_accuracy: 0.9969 -
val_dice_coef: 0.0069 - val_loss: 1.1073 - learning_rate: 1.0000e-04
Epoch 20/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9973 -
dice_coef: 0.0058 - loss: 1.1058
```

```
Epoch 20: val_loss improved from 1.10735 to 1.08098, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 571ms/step - accuracy: 0.9973 -
dice_coef: 0.0058 - loss: 1.1058 - val_accuracy: 0.9969 -
val_dice_coef: 0.0078 - val_loss: 1.0810 - learning_rate: 1.0000e-04
Epoch 21/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9975 -
dice_coef: 0.0060 - loss: 1.1003
Epoch 21: val_loss improved from 1.08098 to 1.07498, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 568ms/step - accuracy: 0.9975 -
dice_coef: 0.0060 - loss: 1.1003 - val_accuracy: 0.9961 -
val_dice_coef: 0.0105 - val_loss: 1.0750 - learning_rate: 1.0000e-04
Epoch 22/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9974 -
dice_coef: 0.0068 - loss: 1.0936
Epoch 22: val_loss did not improve from 1.07498
70/70 ━━━━━━━━━━━━━━━━━━━━ 44s 563ms/step - accuracy: 0.9974 -
dice_coef: 0.0068 - loss: 1.0935 - val_accuracy: 0.9952 -
val_dice_coef: 0.0113 - val_loss: 1.0828 - learning_rate: 1.0000e-04
Epoch 23/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9973 -
dice_coef: 0.0078 - loss: 1.0873
Epoch 23: val_loss improved from 1.07498 to 1.07362, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 568ms/step - accuracy: 0.9973 -
dice_coef: 0.0078 - loss: 1.0873 - val_accuracy: 0.9950 -
val_dice_coef: 0.0128 - val_loss: 1.0736 - learning_rate: 1.0000e-04
Epoch 24/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9968 -
```

```
dice_coef: 0.0100 - loss: 1.0793
Epoch 24: val_loss improved from 1.07362 to 1.06583, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 46s 570ms/step - accuracy: 0.9968 -
dice_coef: 0.0100 - loss: 1.0793 - val_accuracy: 0.9956 -
val_dice_coef: 0.0143 - val_loss: 1.0658 - learning_rate: 1.0000e-04
Epoch 25/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9965 -
dice_coef: 0.0147 - loss: 1.0699
Epoch 25: val_loss improved from 1.06583 to 1.05676, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 569ms/step - accuracy: 0.9965 -
dice_coef: 0.0147 - loss: 1.0699 - val_accuracy: 0.9967 -
val_dice_coef: 0.0162 - val_loss: 1.0568 - learning_rate: 1.0000e-04
Epoch 26/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9969 -
dice_coef: 0.0109 - loss: 1.0680
Epoch 26: val_loss improved from 1.05676 to 1.05470, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 46s 570ms/step - accuracy: 0.9969 -
dice_coef: 0.0109 - loss: 1.0680 - val_accuracy: 0.9960 -
val_dice_coef: 0.0176 - val_loss: 1.0547 - learning_rate: 1.0000e-04
Epoch 27/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9967 -
dice_coef: 0.0137 - loss: 1.0614
Epoch 27: val_loss improved from 1.05470 to 1.04667, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
```

```
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 569ms/step - accuracy: 0.9967 -
dice_coef: 0.0137 - loss: 1.0614 - val_accuracy: 0.9958 -
val_dice_coef: 0.0192 - val_loss: 1.0467 - learning_rate: 1.0000e-04
Epoch 28/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9970 -
dice_coef: 0.0150 - loss: 1.0553
Epoch 28: val_loss improved from 1.04667 to 1.04239, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 570ms/step - accuracy: 0.9970 -
dice_coef: 0.0150 - loss: 1.0553 - val_accuracy: 0.9967 -
val_dice_coef: 0.0201 - val_loss: 1.0424 - learning_rate: 1.0000e-04
Epoch 29/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9968 -
dice_coef: 0.0139 - loss: 1.0544
Epoch 29: val_loss did not improve from 1.04239
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9968 -
dice_coef: 0.0139 - loss: 1.0544 - val_accuracy: 0.9959 -
val_dice_coef: 0.0188 - val_loss: 1.0430 - learning_rate: 1.0000e-04
Epoch 30/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9967 -
dice_coef: 0.0144 - loss: 1.0495
Epoch 30: val_loss improved from 1.04239 to 1.03387, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 46s 570ms/step - accuracy: 0.9967 -
dice_coef: 0.0145 - loss: 1.0494 - val_accuracy: 0.9959 -
val_dice_coef: 0.0215 - val_loss: 1.0339 - learning_rate: 1.0000e-04
Epoch 31/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9970 -
dice_coef: 0.0153 - loss: 1.0442
Epoch 31: val_loss improved from 1.03387 to 1.03118, saving model to
best_model.h5
```

```
WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 569ms/step - accuracy: 0.9970 -
dice_coef: 0.0153 - loss: 1.0442 - val_accuracy: 0.9960 -
val_dice_coef: 0.0225 - val_loss: 1.0312 - learning_rate: 1.0000e-04
Epoch 32/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9964 -
dice_coef: 0.0174 - loss: 1.0406
Epoch 32: val_loss improved from 1.03118 to 1.03077, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 569ms/step - accuracy: 0.9964 -
dice_coef: 0.0174 - loss: 1.0405 - val_accuracy: 0.9962 -
val_dice_coef: 0.0214 - val_loss: 1.0308 - learning_rate: 1.0000e-04
Epoch 33/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9969 -
dice_coef: 0.0227 - loss: 1.0307
Epoch 33: val_loss improved from 1.03077 to 1.02206, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 568ms/step - accuracy: 0.9969 -
dice_coef: 0.0227 - loss: 1.0307 - val_accuracy: 0.9968 -
val_dice_coef: 0.0234 - val_loss: 1.0221 - learning_rate: 1.0000e-04
Epoch 34/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9969 -
dice_coef: 0.0278 - loss: 1.0226
Epoch 34: val_loss did not improve from 1.02206
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 562ms/step - accuracy: 0.9969 -
dice_coef: 0.0277 - loss: 1.0227 - val_accuracy: 0.9964 -
val_dice_coef: 0.0233 - val_loss: 1.0241 - learning_rate: 1.0000e-04
Epoch 35/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 520ms/step - accuracy: 0.9968 -
dice_coef: 0.0232 - loss: 1.0253
Epoch 35: val_loss did not improve from 1.02206
```

```
70/70 ━━━━━━━━━━━━━━━━━━━━ 44s 561ms/step - accuracy: 0.9968 -
dice_coef: 0.0232 - loss: 1.0253 - val_accuracy: 0.9961 -
val_dice_coef: 0.0227 - val_loss: 1.0232 - learning_rate: 1.0000e-04
Epoch 36/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9967 -
dice_coef: 0.0211 - loss: 1.0247
Epoch 36: val_loss did not improve from 1.02206
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 565ms/step - accuracy: 0.9967 -
dice_coef: 0.0212 - loss: 1.0246 - val_accuracy: 0.9964 -
val_dice_coef: 0.0234 - val_loss: 1.0255 - learning_rate: 1.0000e-04
Epoch 37/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9965 -
dice_coef: 0.0301 - loss: 1.0142
Epoch 37: val_loss improved from 1.02206 to 1.02158, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 46s 569ms/step - accuracy: 0.9965 -
dice_coef: 0.0300 - loss: 1.0143 - val_accuracy: 0.9957 -
val_dice_coef: 0.0234 - val_loss: 1.0216 - learning_rate: 1.0000e-04
Epoch 38/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9961 -
dice_coef: 0.0307 - loss: 1.0118
Epoch 38: val_loss improved from 1.02158 to 1.01322, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 570ms/step - accuracy: 0.9961 -
dice_coef: 0.0307 - loss: 1.0119 - val_accuracy: 0.9965 -
val_dice_coef: 0.0283 - val_loss: 1.0132 - learning_rate: 1.0000e-04
Epoch 39/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 525ms/step - accuracy: 0.9963 -
dice_coef: 0.0342 - loss: 1.0059
Epoch 39: val_loss improved from 1.01322 to 1.01009, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
```

```
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 572ms/step - accuracy: 0.9963 -
dice_coef: 0.0341 - loss: 1.0060 - val_accuracy: 0.9965 -
val_dice_coef: 0.0292 - val_loss: 1.0101 - learning_rate: 1.0000e-04
Epoch 40/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9957 -
dice_coef: 0.0355 - loss: 1.0039
Epoch 40: val_loss improved from 1.01009 to 1.00382, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 568ms/step - accuracy: 0.9957 -
dice_coef: 0.0355 - loss: 1.0039 - val_accuracy: 0.9962 -
val_dice_coef: 0.0331 - val_loss: 1.0038 - learning_rate: 1.0000e-04
Epoch 41/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9962 -
dice_coef: 0.0325 - loss: 1.0037
Epoch 41: val_loss did not improve from 1.00382
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 562ms/step - accuracy: 0.9962 -
dice_coef: 0.0325 - loss: 1.0036 - val_accuracy: 0.9951 -
val_dice_coef: 0.0356 - val_loss: 1.0053 - learning_rate: 1.0000e-04
Epoch 42/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9948 -
dice_coef: 0.0494 - loss: 0.9870
Epoch 42: val_loss improved from 1.00382 to 1.00172, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 570ms/step - accuracy: 0.9948 -
dice_coef: 0.0494 - loss: 0.9870 - val_accuracy: 0.9949 -
val_dice_coef: 0.0352 - val_loss: 1.0017 - learning_rate: 1.0000e-04
Epoch 43/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9951 -
dice_coef: 0.0409 - loss: 0.9934
Epoch 43: val_loss improved from 1.00172 to 0.99264, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
```

```
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 569ms/step - accuracy: 0.9951 -
dice_coef: 0.0410 - loss: 0.9933 - val_accuracy: 0.9966 -
val_dice_coef: 0.0421 - val_loss: 0.9926 - learning_rate: 1.0000e-04
Epoch 44/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9950 -
dice_coef: 0.0663 - loss: 0.9666
Epoch 44: val_loss improved from 0.99264 to 0.96906, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 569ms/step - accuracy: 0.9950 -
dice_coef: 0.0662 - loss: 0.9666 - val_accuracy: 0.9956 -
val_dice_coef: 0.0639 - val_loss: 0.9691 - learning_rate: 1.0000e-04
Epoch 45/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 526ms/step - accuracy: 0.9941 -
dice_coef: 0.0770 - loss: 0.9547
Epoch 45: val_loss improved from 0.96906 to 0.95401, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 46s 574ms/step - accuracy: 0.9941 -
dice_coef: 0.0770 - loss: 0.9547 - val_accuracy: 0.9936 -
val_dice_coef: 0.0792 - val_loss: 0.9540 - learning_rate: 1.0000e-04
Epoch 46/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9955 -
dice_coef: 0.0798 - loss: 0.9484
Epoch 46: val_loss did not improve from 0.95401
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 561ms/step - accuracy: 0.9955 -
dice_coef: 0.0799 - loss: 0.9484 - val_accuracy: 0.9752 -
val_dice_coef: 0.0758 - val_loss: 0.9975 - learning_rate: 1.0000e-04
Epoch 47/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9949 -
dice_coef: 0.1099 - loss: 0.9181
Epoch 47: val_loss improved from 0.95401 to 0.91451, saving model to
best_model.h5
```

```
WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 570ms/step - accuracy: 0.9949 -
dice_coef: 0.1097 - loss: 0.9182 - val_accuracy: 0.9934 -
val_dice_coef: 0.1181 - val_loss: 0.9145 - learning_rate: 1.0000e-04
Epoch 48/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9950 -
dice_coef: 0.1116 - loss: 0.9149
Epoch 48: val_loss did not improve from 0.91451
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9951 -
dice_coef: 0.1118 - loss: 0.9147 - val_accuracy: 0.9963 -
val_dice_coef: 0.1046 - val_loss: 0.9191 - learning_rate: 1.0000e-04
Epoch 49/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9955 -
dice_coef: 0.1343 - loss: 0.8903
Epoch 49: val_loss improved from 0.91451 to 0.89084, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 570ms/step - accuracy: 0.9955 -
dice_coef: 0.1342 - loss: 0.8904 - val_accuracy: 0.9952 -
val_dice_coef: 0.1344 - val_loss: 0.8908 - learning_rate: 1.0000e-04
Epoch 50/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9956 -
dice_coef: 0.1467 - loss: 0.8771
Epoch 50: val_loss improved from 0.89084 to 0.86547, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 570ms/step - accuracy: 0.9956 -
dice_coef: 0.1467 - loss: 0.8772 - val_accuracy: 0.9957 -
val_dice_coef: 0.1555 - val_loss: 0.8655 - learning_rate: 1.0000e-04
Epoch 51/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9959 -
dice_coef: 0.1554 - loss: 0.8667
Epoch 51: val_loss did not improve from 0.86547
```

```
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9959 -
dice_coef: 0.1555 - loss: 0.8666 - val_accuracy: 0.9970 -
val_dice_coef: 0.1424 - val_loss: 0.8754 - learning_rate: 1.0000e-04
Epoch 52/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9968 -
dice_coef: 0.1555 - loss: 0.8630
Epoch 52: val_loss did not improve from 0.86547
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9968 -
dice_coef: 0.1558 - loss: 0.8627 - val_accuracy: 0.9965 -
val_dice_coef: 0.1452 - val_loss: 0.8752 - learning_rate: 1.0000e-04
Epoch 53/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9958 -
dice_coef: 0.1964 - loss: 0.8243
Epoch 53: val_loss improved from 0.86547 to 0.84225, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 571ms/step - accuracy: 0.9958 -
dice_coef: 0.1964 - loss: 0.8243 - val_accuracy: 0.9963 -
val_dice_coef: 0.1756 - val_loss: 0.8423 - learning_rate: 1.0000e-04
Epoch 54/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9963 -
dice_coef: 0.2271 - loss: 0.7916
Epoch 54: val_loss improved from 0.84225 to 0.82357, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 570ms/step - accuracy: 0.9963 -
dice_coef: 0.2269 - loss: 0.7918 - val_accuracy: 0.9962 -
val_dice_coef: 0.1956 - val_loss: 0.8236 - learning_rate: 1.0000e-04
Epoch 55/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9967 -
dice_coef: 0.2041 - loss: 0.8133
Epoch 55: val_loss did not improve from 0.82357
70/70 ━━━━━━━━━━━━━━━━━━━━ 44s 562ms/step - accuracy: 0.9967 -
dice_coef: 0.2044 - loss: 0.8130 - val_accuracy: 0.9966 -
val_dice_coef: 0.1508 - val_loss: 0.8677 - learning_rate: 1.0000e-04
Epoch 56/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9966 -
dice_coef: 0.2419 - loss: 0.7751
```

```
Epoch 56: val_loss improved from 0.82357 to 0.80908, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 570ms/step - accuracy: 0.9966 -
dice_coef: 0.2417 - loss: 0.7753 - val_accuracy: 0.9965 -
val_dice_coef: 0.2083 - val_loss: 0.8091 - learning_rate: 1.0000e-04
Epoch 57/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9967 -
dice_coef: 0.2416 - loss: 0.7751
Epoch 57: val_loss did not improve from 0.80908
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 562ms/step - accuracy: 0.9967 -
dice_coef: 0.2419 - loss: 0.7748 - val_accuracy: 0.9964 -
val_dice_coef: 0.1965 - val_loss: 0.8212 - learning_rate: 1.0000e-04
Epoch 58/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9969 -
dice_coef: 0.2763 - loss: 0.7399
Epoch 58: val_loss did not improve from 0.80908
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 562ms/step - accuracy: 0.9969 -
dice_coef: 0.2764 - loss: 0.7398 - val_accuracy: 0.9968 -
val_dice_coef: 0.1888 - val_loss: 0.8276 - learning_rate: 1.0000e-04
Epoch 59/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9969 -
dice_coef: 0.2777 - loss: 0.7383
Epoch 59: val_loss did not improve from 0.80908
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9969 -
dice_coef: 0.2780 - loss: 0.7381 - val_accuracy: 0.9973 -
val_dice_coef: 0.1862 - val_loss: 0.8282 - learning_rate: 1.0000e-04
Epoch 60/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9971 -
dice_coef: 0.2848 - loss: 0.7302
Epoch 60: val_loss did not improve from 0.80908
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9971 -
dice_coef: 0.2850 - loss: 0.7300 - val_accuracy: 0.9972 -
val_dice_coef: 0.2053 - val_loss: 0.8098 - learning_rate: 1.0000e-04
Epoch 61/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9971 -
dice_coef: 0.3236 - loss: 0.6920
Epoch 61: val_loss improved from 0.80908 to 0.76367, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
```

```
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 46s 570ms/step - accuracy: 0.9971 -
dice_coef: 0.3235 - loss: 0.6922 - val_accuracy: 0.9972 -
val_dice_coef: 0.2503 - val_loss: 0.7637 - learning_rate: 1.0000e-04
Epoch 62/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9974 -
dice_coef: 0.3342 - loss: 0.6804
Epoch 62: val_loss improved from 0.76367 to 0.74785, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 46s 571ms/step - accuracy: 0.9974 -
dice_coef: 0.3345 - loss: 0.6801 - val_accuracy: 0.9973 -
val_dice_coef: 0.2660 - val_loss: 0.7479 - learning_rate: 1.0000e-04
Epoch 63/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9973 -
dice_coef: 0.3216 - loss: 0.6934
Epoch 63: val_loss improved from 0.74785 to 0.73602, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 570ms/step - accuracy: 0.9973 -
dice_coef: 0.3221 - loss: 0.6930 - val_accuracy: 0.9970 -
val_dice_coef: 0.2790 - val_loss: 0.7360 - learning_rate: 1.0000e-04
Epoch 64/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9971 -
dice_coef: 0.3819 - loss: 0.6337
Epoch 64: val_loss improved from 0.73602 to 0.72591, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 570ms/step - accuracy: 0.9971 -
dice_coef: 0.3817 - loss: 0.6340 - val_accuracy: 0.9973 -
val_dice_coef: 0.2878 - val_loss: 0.7259 - learning_rate: 1.0000e-04
```

```
Epoch 65/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9972 -
dice_coef: 0.3674 - loss: 0.6485
Epoch 65: val_loss improved from 0.72591 to 0.72145, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 570ms/step - accuracy: 0.9972 -
dice_coef: 0.3673 - loss: 0.6486 - val_accuracy: 0.9973 -
val_dice_coef: 0.2925 - val_loss: 0.7215 - learning_rate: 1.0000e-04
Epoch 66/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9976 -
dice_coef: 0.4215 - loss: 0.5918
Epoch 66: val_loss did not improve from 0.72145
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9976 -
dice_coef: 0.4212 - loss: 0.5921 - val_accuracy: 0.9974 -
val_dice_coef: 0.2796 - val_loss: 0.7344 - learning_rate: 1.0000e-04
Epoch 67/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9974 -
dice_coef: 0.3661 - loss: 0.6489
Epoch 67: val_loss did not improve from 0.72145
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9974 -
dice_coef: 0.3661 - loss: 0.6489 - val_accuracy: 0.9975 -
val_dice_coef: 0.2883 - val_loss: 0.7249 - learning_rate: 1.0000e-04
Epoch 68/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9973 -
dice_coef: 0.4116 - loss: 0.6037
Epoch 68: val_loss improved from 0.72145 to 0.69718, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 570ms/step - accuracy: 0.9973 -
dice_coef: 0.4114 - loss: 0.6039 - val_accuracy: 0.9974 -
val_dice_coef: 0.3167 - val_loss: 0.6972 - learning_rate: 1.0000e-04
Epoch 69/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9975 -
dice_coef: 0.4250 - loss: 0.5890
Epoch 69: val_loss did not improve from 0.69718
70/70 ━━━━━━━━━━━━━━━━━━━━ 44s 563ms/step - accuracy: 0.9975 -
dice_coef: 0.4248 - loss: 0.5893 - val_accuracy: 0.9971 -
```

```
val_dice_coef: 0.3137 - val_loss: 0.7016 - learning_rate: 1.0000e-04
Epoch 70/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9976 -
dice_coef: 0.4354 - loss: 0.5784
Epoch 70: val_loss did not improve from 0.69718
70/70 ━━━━━━━━━━━━━━━━━━━━ 44s 562ms/step - accuracy: 0.9976 -
dice_coef: 0.4352 - loss: 0.5785 - val_accuracy: 0.9976 -
val_dice_coef: 0.3030 - val_loss: 0.7110 - learning_rate: 1.0000e-04
Epoch 71/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9976 -
dice_coef: 0.3953 - loss: 0.6190
Epoch 71: val_loss did not improve from 0.69718
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9976 -
dice_coef: 0.3955 - loss: 0.6188 - val_accuracy: 0.9974 -
val_dice_coef: 0.3174 - val_loss: 0.6975 - learning_rate: 1.0000e-04
Epoch 72/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9975 -
dice_coef: 0.4051 - loss: 0.6099
Epoch 72: val_loss improved from 0.69718 to 0.66548, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 570ms/step - accuracy: 0.9975 -
dice_coef: 0.4054 - loss: 0.6096 - val_accuracy: 0.9976 -
val_dice_coef: 0.3479 - val_loss: 0.6655 - learning_rate: 1.0000e-04
Epoch 73/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 525ms/step - accuracy: 0.9978 -
dice_coef: 0.4506 - loss: 0.5625
Epoch 73: val_loss improved from 0.66548 to 0.66038, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 573ms/step - accuracy: 0.9978 -
dice_coef: 0.4507 - loss: 0.5624 - val_accuracy: 0.9975 -
val_dice_coef: 0.3542 - val_loss: 0.6604 - learning_rate: 1.0000e-04
Epoch 74/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9978 -
dice_coef: 0.4378 - loss: 0.5760
Epoch 74: val_loss did not improve from 0.66038
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 562ms/step - accuracy: 0.9978 -
```

```
dice_coef: 0.4381 - loss: 0.5757 - val_accuracy: 0.9974 -
val_dice_coef: 0.3409 - val_loss: 0.6745 - learning_rate: 1.0000e-04
Epoch 75/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9978 -
dice_coef: 0.4311 - loss: 0.5830
Epoch 75: val_loss improved from 0.66038 to 0.64651, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 571ms/step - accuracy: 0.9978 -
dice_coef: 0.4311 - loss: 0.5831 - val_accuracy: 0.9976 -
val_dice_coef: 0.3672 - val_loss: 0.6465 - learning_rate: 1.0000e-04
Epoch 76/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9978 -
dice_coef: 0.4377 - loss: 0.5766
Epoch 76: val_loss did not improve from 0.64651
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9978 -
dice_coef: 0.4380 - loss: 0.5763 - val_accuracy: 0.9972 -
val_dice_coef: 0.3584 - val_loss: 0.6578 - learning_rate: 1.0000e-04
Epoch 77/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9977 -
dice_coef: 0.4588 - loss: 0.5556
Epoch 77: val_loss did not improve from 0.64651
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9977 -
dice_coef: 0.4590 - loss: 0.5554 - val_accuracy: 0.9974 -
val_dice_coef: 0.3488 - val_loss: 0.6665 - learning_rate: 1.0000e-04
Epoch 78/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9978 -
dice_coef: 0.4367 - loss: 0.5778
Epoch 78: val_loss improved from 0.64651 to 0.63926, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 46s 571ms/step - accuracy: 0.9978 -
dice_coef: 0.4371 - loss: 0.5775 - val_accuracy: 0.9975 -
val_dice_coef: 0.3752 - val_loss: 0.6393 - learning_rate: 1.0000e-04
Epoch 79/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9977 -
dice_coef: 0.4871 - loss: 0.5275
```

```
Epoch 79: val_loss improved from 0.63926 to 0.63855, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 570ms/step - accuracy: 0.9977 -
dice_coef: 0.4869 - loss: 0.5278 - val_accuracy: 0.9975 -
val_dice_coef: 0.3765 - val_loss: 0.6386 - learning_rate: 1.0000e-04
Epoch 80/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 525ms/step - accuracy: 0.9979 -
dice_coef: 0.4108 - loss: 0.6035
Epoch 80: val_loss improved from 0.63855 to 0.62864, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 573ms/step - accuracy: 0.9979 -
dice_coef: 0.4111 - loss: 0.6031 - val_accuracy: 0.9976 -
val_dice_coef: 0.3854 - val_loss: 0.6286 - learning_rate: 1.0000e-04
Epoch 81/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9978 -
dice_coef: 0.4578 - loss: 0.5570
Epoch 81: val_loss did not improve from 0.62864
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9978 -
dice_coef: 0.4577 - loss: 0.5570 - val_accuracy: 0.9974 -
val_dice_coef: 0.3754 - val_loss: 0.6410 - learning_rate: 1.0000e-04
Epoch 82/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9978 -
dice_coef: 0.4528 - loss: 0.5620
Epoch 82: val_loss did not improve from 0.62864
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9978 -
dice_coef: 0.4530 - loss: 0.5618 - val_accuracy: 0.9972 -
val_dice_coef: 0.3705 - val_loss: 0.6473 - learning_rate: 1.0000e-04
Epoch 83/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9978 -
dice_coef: 0.4673 - loss: 0.5478
Epoch 83: val_loss did not improve from 0.62864
70/70 ━━━━━━━━━━━━━━━━━━━━ 44s 562ms/step - accuracy: 0.9978 -
dice_coef: 0.4670 - loss: 0.5481 - val_accuracy: 0.9976 -
val_dice_coef: 0.3691 - val_loss: 0.6461 - learning_rate: 1.0000e-04
Epoch 84/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9976 -
```

```
dice_coef: 0.4731 - loss: 0.5426
Epoch 84: val_loss improved from 0.62864 to 0.60451, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 571ms/step - accuracy: 0.9976 -
dice_coef: 0.4732 - loss: 0.5426 - val_accuracy: 0.9978 -
val_dice_coef: 0.4096 - val_loss: 0.6045 - learning_rate: 1.0000e-04
Epoch 85/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9978 -
dice_coef: 0.4420 - loss: 0.5731
Epoch 85: val_loss did not improve from 0.60451
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9978 -
dice_coef: 0.4422 - loss: 0.5729 - val_accuracy: 0.9976 -
val_dice_coef: 0.3921 - val_loss: 0.6231 - learning_rate: 1.0000e-04
Epoch 86/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9980 -
dice_coef: 0.4595 - loss: 0.5545
Epoch 86: val_loss did not improve from 0.60451
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9980 -
dice_coef: 0.4599 - loss: 0.5541 - val_accuracy: 0.9974 -
val_dice_coef: 0.3954 - val_loss: 0.6214 - learning_rate: 1.0000e-04
Epoch 87/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9980 -
dice_coef: 0.5161 - loss: 0.4973
Epoch 87: val_loss did not improve from 0.60451
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 565ms/step - accuracy: 0.9980 -
dice_coef: 0.5157 - loss: 0.4977 - val_accuracy: 0.9976 -
val_dice_coef: 0.4028 - val_loss: 0.6127 - learning_rate: 1.0000e-04
Epoch 88/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9980 -
dice_coef: 0.5237 - loss: 0.4903
Epoch 88: val_loss did not improve from 0.60451
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9980 -
dice_coef: 0.5234 - loss: 0.4906 - val_accuracy: 0.9976 -
val_dice_coef: 0.3952 - val_loss: 0.6204 - learning_rate: 1.0000e-04
Epoch 89/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9978 -
dice_coef: 0.4967 - loss: 0.5185
Epoch 89: val_loss did not improve from 0.60451
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9978 -
dice_coef: 0.4967 - loss: 0.5185 - val_accuracy: 0.9976 -
val_dice_coef: 0.3705 - val_loss: 0.6453 - learning_rate: 1.0000e-04
Epoch 90/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9980 -
```

```
dice_coef: 0.4767 - loss: 0.5377
Epoch 90: val_loss did not improve from 0.60451
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9980 -
dice_coef: 0.4768 - loss: 0.5376 - val_accuracy: 0.9975 -
val_dice_coef: 0.3930 - val_loss: 0.6232 - learning_rate: 1.0000e-04
Epoch 91/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9980 -
dice_coef: 0.4613 - loss: 0.5532
Epoch 91: val_loss did not improve from 0.60451
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9980 -
dice_coef: 0.4619 - loss: 0.5525 - val_accuracy: 0.9972 -
val_dice_coef: 0.3651 - val_loss: 0.6528 - learning_rate: 1.0000e-04
Epoch 92/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9975 -
dice_coef: 0.4796 - loss: 0.5371
Epoch 92: val_loss did not improve from 0.60451
70/70 ━━━━━━━━━━━━━━━━━━━━ 44s 562ms/step - accuracy: 0.9975 -
dice_coef: 0.4799 - loss: 0.5368 - val_accuracy: 0.9975 -
val_dice_coef: 0.4072 - val_loss: 0.6101 - learning_rate: 1.0000e-04
Epoch 93/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9980 -
dice_coef: 0.5097 - loss: 0.5048
Epoch 93: val_loss did not improve from 0.60451
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 565ms/step - accuracy: 0.9980 -
dice_coef: 0.5096 - loss: 0.5048 - val_accuracy: 0.9978 -
val_dice_coef: 0.3998 - val_loss: 0.6147 - learning_rate: 1.0000e-04
Epoch 94/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9981 -
dice_coef: 0.4867 - loss: 0.5271
Epoch 94: val_loss did not improve from 0.60451

Epoch 94: ReduceLROnPlateau reducing learning rate to
4.999999873689376e-05.
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9981 -
dice_coef: 0.4871 - loss: 0.5267 - val_accuracy: 0.9976 -
val_dice_coef: 0.3829 - val_loss: 0.6331 - learning_rate: 1.0000e-04
Epoch 95/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9980 -
dice_coef: 0.5140 - loss: 0.5006
Epoch 95: val_loss did not improve from 0.60451
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9980 -
dice_coef: 0.5138 - loss: 0.5008 - val_accuracy: 0.9977 -
val_dice_coef: 0.4038 - val_loss: 0.6119 - learning_rate: 5.0000e-05
Epoch 96/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9981 -
dice_coef: 0.5172 - loss: 0.4971
Epoch 96: val_loss improved from 0.60451 to 0.59737, saving model to
best_model.h5
```

```
WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 572ms/step - accuracy: 0.9981 -
dice_coef: 0.5172 - loss: 0.4971 - val_accuracy: 0.9976 -
val_dice_coef: 0.4183 - val_loss: 0.5974 - learning_rate: 5.0000e-05
Epoch 97/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9981 -
dice_coef: 0.5281 - loss: 0.4862
Epoch 97: val_loss did not improve from 0.59737
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9981 -
dice_coef: 0.5281 - loss: 0.4862 - val_accuracy: 0.9976 -
val_dice_coef: 0.4181 - val_loss: 0.5982 - learning_rate: 5.0000e-05
Epoch 98/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9980 -
dice_coef: 0.5068 - loss: 0.5078
Epoch 98: val_loss improved from 0.59737 to 0.58351, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 570ms/step - accuracy: 0.9980 -
dice_coef: 0.5069 - loss: 0.5077 - val_accuracy: 0.9977 -
val_dice_coef: 0.4321 - val_loss: 0.5835 - learning_rate: 5.0000e-05
Epoch 99/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 526ms/step - accuracy: 0.9981 -
dice_coef: 0.5117 - loss: 0.5021
Epoch 99: val_loss did not improve from 0.58351
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 568ms/step - accuracy: 0.9981 -
dice_coef: 0.5116 - loss: 0.5021 - val_accuracy: 0.9977 -
val_dice_coef: 0.4258 - val_loss: 0.5901 - learning_rate: 5.0000e-05
Epoch 100/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9980 -
dice_coef: 0.4973 - loss: 0.5180
Epoch 100: val_loss did not improve from 0.58351
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 562ms/step - accuracy: 0.9980 -
dice_coef: 0.4977 - loss: 0.5177 - val_accuracy: 0.9977 -
val_dice_coef: 0.4203 - val_loss: 0.5952 - learning_rate: 5.0000e-05
Epoch 101/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 526ms/step - accuracy: 0.9982 -
dice_coef: 0.4947 - loss: 0.5194
Epoch 101: val_loss did not improve from 0.58351
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 567ms/step - accuracy: 0.9982 -
```

```
dice_coef: 0.4948 - loss: 0.5193 - val_accuracy: 0.9977 -
val_dice_coef: 0.4194 - val_loss: 0.5967 - learning_rate: 5.0000e-05
Epoch 102/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9982 -
dice_coef: 0.5392 - loss: 0.4736
Epoch 102: val_loss did not improve from 0.58351
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 562ms/step - accuracy: 0.9982 -
dice_coef: 0.5389 - loss: 0.4739 - val_accuracy: 0.9975 -
val_dice_coef: 0.4114 - val_loss: 0.6058 - learning_rate: 5.0000e-05
Epoch 103/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 526ms/step - accuracy: 0.9982 -
dice_coef: 0.5548 - loss: 0.4586
Epoch 103: val_loss did not improve from 0.58351
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 567ms/step - accuracy: 0.9982 -
dice_coef: 0.5546 - loss: 0.4589 - val_accuracy: 0.9976 -
val_dice_coef: 0.4299 - val_loss: 0.5868 - learning_rate: 5.0000e-05
Epoch 104/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9981 -
dice_coef: 0.5411 - loss: 0.4735
Epoch 104: val_loss did not improve from 0.58351
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9981 -
dice_coef: 0.5410 - loss: 0.4736 - val_accuracy: 0.9977 -
val_dice_coef: 0.4276 - val_loss: 0.5884 - learning_rate: 5.0000e-05
Epoch 105/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9980 -
dice_coef: 0.5470 - loss: 0.4678
Epoch 105: val_loss improved from 0.58351 to 0.57975, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 571ms/step - accuracy: 0.9980 -
dice_coef: 0.5470 - loss: 0.4679 - val_accuracy: 0.9976 -
val_dice_coef: 0.4372 - val_loss: 0.5797 - learning_rate: 5.0000e-05
Epoch 106/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9983 -
dice_coef: 0.5097 - loss: 0.5043
Epoch 106: val_loss did not improve from 0.57975
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 565ms/step - accuracy: 0.9983 -
dice_coef: 0.5096 - loss: 0.5044 - val_accuracy: 0.9976 -
val_dice_coef: 0.4350 - val_loss: 0.5827 - learning_rate: 5.0000e-05
Epoch 107/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9982 -
dice_coef: 0.5161 - loss: 0.4979
Epoch 107: val_loss did not improve from 0.57975
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9982 -
```

```
dice_coef: 0.5163 - loss: 0.4976 - val_accuracy: 0.9977 -
val_dice_coef: 0.4313 - val_loss: 0.5852 - learning_rate: 5.0000e-05
Epoch 108/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9983 -
dice_coef: 0.5362 - loss: 0.4765
Epoch 108: val_loss improved from 0.57975 to 0.57625, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 572ms/step - accuracy: 0.9983 -
dice_coef: 0.5361 - loss: 0.4766 - val_accuracy: 0.9978 -
val_dice_coef: 0.4389 - val_loss: 0.5763 - learning_rate: 5.0000e-05
Epoch 109/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9983 -
dice_coef: 0.5580 - loss: 0.4553
Epoch 109: val_loss did not improve from 0.57625
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9983 -
dice_coef: 0.5578 - loss: 0.4554 - val_accuracy: 0.9978 -
val_dice_coef: 0.4328 - val_loss: 0.5829 - learning_rate: 5.0000e-05
Epoch 110/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9981 -
dice_coef: 0.5321 - loss: 0.4823
Epoch 110: val_loss did not improve from 0.57625
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9981 -
dice_coef: 0.5323 - loss: 0.4821 - val_accuracy: 0.9977 -
val_dice_coef: 0.4335 - val_loss: 0.5826 - learning_rate: 5.0000e-05
Epoch 111/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9982 -
dice_coef: 0.5467 - loss: 0.4674
Epoch 111: val_loss did not improve from 0.57625
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9982 -
dice_coef: 0.5466 - loss: 0.4676 - val_accuracy: 0.9977 -
val_dice_coef: 0.4305 - val_loss: 0.5858 - learning_rate: 5.0000e-05
Epoch 112/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9982 -
dice_coef: 0.5045 - loss: 0.5097
Epoch 112: val_loss did not improve from 0.57625
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9982 -
dice_coef: 0.5047 - loss: 0.5095 - val_accuracy: 0.9976 -
val_dice_coef: 0.4306 - val_loss: 0.5863 - learning_rate: 5.0000e-05
Epoch 113/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9983 -
dice_coef: 0.5409 - loss: 0.4724
Epoch 113: val_loss did not improve from 0.57625
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 565ms/step - accuracy: 0.9983 -
```

```
dice_coef: 0.5409 - loss: 0.4725 - val_accuracy: 0.9978 -
val_dice_coef: 0.4384 - val_loss: 0.5777 - learning_rate: 5.0000e-05
Epoch 114/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9982 -
dice_coef: 0.5545 - loss: 0.4596
Epoch 114: val_loss improved from 0.57625 to 0.56998, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 571ms/step - accuracy: 0.9982 -
dice_coef: 0.5546 - loss: 0.4595 - val_accuracy: 0.9978 -
val_dice_coef: 0.4459 - val_loss: 0.5700 - learning_rate: 5.0000e-05
Epoch 115/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 525ms/step - accuracy: 0.9984 -
dice_coef: 0.5443 - loss: 0.4688
Epoch 115: val_loss improved from 0.56998 to 0.56917, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 574ms/step - accuracy: 0.9984 -
dice_coef: 0.5446 - loss: 0.4685 - val_accuracy: 0.9977 -
val_dice_coef: 0.4473 - val_loss: 0.5692 - learning_rate: 5.0000e-05
Epoch 116/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9982 -
dice_coef: 0.5253 - loss: 0.4892
Epoch 116: val_loss did not improve from 0.56917
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 563ms/step - accuracy: 0.9982 -
dice_coef: 0.5257 - loss: 0.4888 - val_accuracy: 0.9978 -
val_dice_coef: 0.4396 - val_loss: 0.5768 - learning_rate: 5.0000e-05
Epoch 117/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 526ms/step - accuracy: 0.9983 -
dice_coef: 0.5290 - loss: 0.4847
Epoch 117: val_loss did not improve from 0.56917
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 566ms/step - accuracy: 0.9983 -
dice_coef: 0.5293 - loss: 0.4844 - val_accuracy: 0.9977 -
val_dice_coef: 0.4371 - val_loss: 0.5796 - learning_rate: 5.0000e-05
Epoch 118/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9982 -
dice_coef: 0.5575 - loss: 0.4570
Epoch 118: val_loss did not improve from 0.56917
```

```
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 565ms/step - accuracy: 0.9982 -
dice_coef: 0.5573 - loss: 0.4572 - val_accuracy: 0.9978 -
val_dice_coef: 0.4463 - val_loss: 0.5702 - learning_rate: 5.0000e-05
Epoch 119/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9981 -
dice_coef: 0.5575 - loss: 0.4574
Epoch 119: val_loss did not improve from 0.56917
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 565ms/step - accuracy: 0.9981 -
dice_coef: 0.5574 - loss: 0.4575 - val_accuracy: 0.9976 -
val_dice_coef: 0.4486 - val_loss: 0.5694 - learning_rate: 5.0000e-05
Epoch 120/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 525ms/step - accuracy: 0.9980 -
dice_coef: 0.5287 - loss: 0.4869
Epoch 120: val_loss improved from 0.56917 to 0.56874, saving model to
best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 573ms/step - accuracy: 0.9980 -
dice_coef: 0.5291 - loss: 0.4864 - val_accuracy: 0.9977 -
val_dice_coef: 0.4483 - val_loss: 0.5687 - learning_rate: 5.0000e-05
Epoch 121/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9982 -
dice_coef: 0.5061 - loss: 0.5079
Epoch 121: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9982 -
dice_coef: 0.5065 - loss: 0.5075 - val_accuracy: 0.9977 -
val_dice_coef: 0.4424 - val_loss: 0.5745 - learning_rate: 5.0000e-05
Epoch 122/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9984 -
dice_coef: 0.5578 - loss: 0.4547
Epoch 122: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9984 -
dice_coef: 0.5577 - loss: 0.4548 - val_accuracy: 0.9977 -
val_dice_coef: 0.4365 - val_loss: 0.5804 - learning_rate: 5.0000e-05
Epoch 123/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9983 -
dice_coef: 0.5348 - loss: 0.4783
Epoch 123: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9983 -
dice_coef: 0.5351 - loss: 0.4780 - val_accuracy: 0.9977 -
val_dice_coef: 0.4469 - val_loss: 0.5701 - learning_rate: 5.0000e-05
Epoch 124/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9984 -
dice_coef: 0.5166 - loss: 0.4963
Epoch 124: val_loss did not improve from 0.56874
```

```
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9984 -
dice_coef: 0.5172 - loss: 0.4958 - val_accuracy: 0.9976 -
val_dice_coef: 0.4266 - val_loss: 0.5917 - learning_rate: 5.0000e-05
Epoch 125/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9985 -
dice_coef: 0.5873 - loss: 0.4246
Epoch 125: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9985 -
dice_coef: 0.5872 - loss: 0.4247 - val_accuracy: 0.9976 -
val_dice_coef: 0.4233 - val_loss: 0.5948 - learning_rate: 5.0000e-05
Epoch 126/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9983 -
dice_coef: 0.5377 - loss: 0.4767
Epoch 126: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9983 -
dice_coef: 0.5379 - loss: 0.4766 - val_accuracy: 0.9977 -
val_dice_coef: 0.4356 - val_loss: 0.5816 - learning_rate: 5.0000e-05
Epoch 127/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9984 -
dice_coef: 0.5368 - loss: 0.4768
Epoch 127: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9984 -
dice_coef: 0.5370 - loss: 0.4766 - val_accuracy: 0.9977 -
val_dice_coef: 0.4326 - val_loss: 0.5849 - learning_rate: 5.0000e-05
Epoch 128/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9983 -
dice_coef: 0.5238 - loss: 0.4900
Epoch 128: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 565ms/step - accuracy: 0.9983 -
dice_coef: 0.5240 - loss: 0.4898 - val_accuracy: 0.9977 -
val_dice_coef: 0.3937 - val_loss: 0.6241 - learning_rate: 5.0000e-05
Epoch 129/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9982 -
dice_coef: 0.5659 - loss: 0.4487
Epoch 129: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 565ms/step - accuracy: 0.9982 -
dice_coef: 0.5656 - loss: 0.4490 - val_accuracy: 0.9977 -
val_dice_coef: 0.4092 - val_loss: 0.6082 - learning_rate: 5.0000e-05
Epoch 130/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 521ms/step - accuracy: 0.9982 -
dice_coef: 0.5626 - loss: 0.4513
Epoch 130: val_loss did not improve from 0.56874

Epoch 130: ReduceLROnPlateau reducing learning rate to
2.499999936844688e-05.
70/70 ━━━━━━━━━━━━━━━━━━━━ 44s 562ms/step - accuracy: 0.9982 -
dice_coef: 0.5627 - loss: 0.4512 - val_accuracy: 0.9977 -
val_dice_coef: 0.4394 - val_loss: 0.5780 - learning_rate: 5.0000e-05
Epoch 131/200
```

```
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 525ms/step - accuracy: 0.9983 -
dice_coef: 0.5613 - loss: 0.4527
Epoch 131: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 565ms/step - accuracy: 0.9983 -
dice_coef: 0.5614 - loss: 0.4526 - val_accuracy: 0.9977 -
val_dice_coef: 0.4281 - val_loss: 0.5896 - learning_rate: 2.5000e-05
Epoch 132/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9982 -
dice_coef: 0.5239 - loss: 0.4914
Epoch 132: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9982 -
dice_coef: 0.5244 - loss: 0.4909 - val_accuracy: 0.9977 -
val_dice_coef: 0.4344 - val_loss: 0.5831 - learning_rate: 2.5000e-05
Epoch 133/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9983 -
dice_coef: 0.5606 - loss: 0.4541
Epoch 133: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9983 -
dice_coef: 0.5608 - loss: 0.4538 - val_accuracy: 0.9977 -
val_dice_coef: 0.4289 - val_loss: 0.5887 - learning_rate: 2.5000e-05
Epoch 134/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9984 -
dice_coef: 0.5750 - loss: 0.4376
Epoch 134: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9984 -
dice_coef: 0.5751 - loss: 0.4376 - val_accuracy: 0.9977 -
val_dice_coef: 0.4374 - val_loss: 0.5808 - learning_rate: 2.5000e-05
Epoch 135/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9984 -
dice_coef: 0.5625 - loss: 0.4507
Epoch 135: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 565ms/step - accuracy: 0.9984 -
dice_coef: 0.5630 - loss: 0.4503 - val_accuracy: 0.9978 -
val_dice_coef: 0.4342 - val_loss: 0.5832 - learning_rate: 2.5000e-05
Epoch 136/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 522ms/step - accuracy: 0.9983 -
dice_coef: 0.5495 - loss: 0.4646
Epoch 136: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9983 -
dice_coef: 0.5498 - loss: 0.4643 - val_accuracy: 0.9977 -
val_dice_coef: 0.4376 - val_loss: 0.5801 - learning_rate: 2.5000e-05
Epoch 137/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 525ms/step - accuracy: 0.9984 -
dice_coef: 0.5974 - loss: 0.4158
Epoch 137: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 566ms/step - accuracy: 0.9984 -
dice_coef: 0.5971 - loss: 0.4161 - val_accuracy: 0.9978 -
val_dice_coef: 0.4439 - val_loss: 0.5734 - learning_rate: 2.5000e-05
Epoch 138/200
```

```
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9982 -
dice_coef: 0.5731 - loss: 0.4419
Epoch 138: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 565ms/step - accuracy: 0.9982 -
dice_coef: 0.5732 - loss: 0.4419 - val_accuracy: 0.9978 -
val_dice_coef: 0.4278 - val_loss: 0.5902 - learning_rate: 2.5000e-05
Epoch 139/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9984 -
dice_coef: 0.5828 - loss: 0.4308
Epoch 139: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9984 -
dice_coef: 0.5826 - loss: 0.4310 - val_accuracy: 0.9977 -
val_dice_coef: 0.4332 - val_loss: 0.5850 - learning_rate: 2.5000e-05
Epoch 140/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9982 -
dice_coef: 0.5473 - loss: 0.4674
Epoch 140: val_loss did not improve from 0.56874

Epoch 140: ReduceLROnPlateau reducing learning rate to
1.249999968422344e-05.
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9982 -
dice_coef: 0.5478 - loss: 0.4669 - val_accuracy: 0.9977 -
val_dice_coef: 0.4304 - val_loss: 0.5879 - learning_rate: 2.5000e-05
Epoch 141/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9984 -
dice_coef: 0.5533 - loss: 0.4602
Epoch 141: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 44s 565ms/step - accuracy: 0.9984 -
dice_coef: 0.5537 - loss: 0.4599 - val_accuracy: 0.9978 -
val_dice_coef: 0.4240 - val_loss: 0.5941 - learning_rate: 1.2500e-05
Epoch 142/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 524ms/step - accuracy: 0.9983 -
dice_coef: 0.5651 - loss: 0.4491
Epoch 142: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 565ms/step - accuracy: 0.9983 -
dice_coef: 0.5651 - loss: 0.4490 - val_accuracy: 0.9978 -
val_dice_coef: 0.4359 - val_loss: 0.5821 - learning_rate: 1.2500e-05
Epoch 143/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9984 -
dice_coef: 0.5528 - loss: 0.4607
Epoch 143: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 565ms/step - accuracy: 0.9984 -
dice_coef: 0.5533 - loss: 0.4602 - val_accuracy: 0.9977 -
val_dice_coef: 0.4228 - val_loss: 0.5958 - learning_rate: 1.2500e-05
Epoch 144/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9984 -
dice_coef: 0.5770 - loss: 0.4364
Epoch 144: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9984 -
```

```
dice_coef: 0.5771 - loss: 0.4363 - val_accuracy: 0.9978 -
val_dice_coef: 0.4253 - val_loss: 0.5927 - learning_rate: 1.2500e-05
Epoch 145/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9983 -
dice_coef: 0.5721 - loss: 0.4423
Epoch 145: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9983 -
dice_coef: 0.5721 - loss: 0.4423 - val_accuracy: 0.9977 -
val_dice_coef: 0.4311 - val_loss: 0.5872 - learning_rate: 1.2500e-05
Epoch 146/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9984 -
dice_coef: 0.5787 - loss: 0.4351
Epoch 146: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9983 -
dice_coef: 0.5788 - loss: 0.4350 - val_accuracy: 0.9978 -
val_dice_coef: 0.4370 - val_loss: 0.5811 - learning_rate: 1.2500e-05
Epoch 147/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9986 -
dice_coef: 0.5830 - loss: 0.4290
Epoch 147: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9985 -
dice_coef: 0.5828 - loss: 0.4292 - val_accuracy: 0.9977 -
val_dice_coef: 0.4326 - val_loss: 0.5859 - learning_rate: 1.2500e-05
Epoch 148/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9982 -
dice_coef: 0.5675 - loss: 0.4475
Epoch 148: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9982 -
dice_coef: 0.5675 - loss: 0.4474 - val_accuracy: 0.9978 -
val_dice_coef: 0.4406 - val_loss: 0.5773 - learning_rate: 1.2500e-05
Epoch 149/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.9985 -
dice_coef: 0.5882 - loss: 0.4245
Epoch 149: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 564ms/step - accuracy: 0.9985 -
dice_coef: 0.5881 - loss: 0.4246 - val_accuracy: 0.9977 -
val_dice_coef: 0.4393 - val_loss: 0.5789 - learning_rate: 1.2500e-05
Epoch 150/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 525ms/step - accuracy: 0.9982 -
dice_coef: 0.5821 - loss: 0.4324
Epoch 150: val_loss did not improve from 0.56874

Epoch 150: ReduceLROnPlateau reducing learning rate to
6.24999984211172e-06.
70/70 ━━━━━━━━━━━━━━━━━━━━ 45s 566ms/step - accuracy: 0.9982 -
dice_coef: 0.5820 - loss: 0.4325 - val_accuracy: 0.9977 -
val_dice_coef: 0.4386 - val_loss: 0.5801 - learning_rate: 1.2500e-05
Epoch 150: early stopping
Restoring model weights from the end of the best epoch: 120.
```

```
Training HRNet model...
Epoch 1/200
     70/Unknown 55s 508ms/step - accuracy: 0.9113 - dice_coef: 0.0059
- loss: 1.3347
Epoch 1: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 58s 562ms/step - accuracy: 0.9122 -
dice_coef: 0.0059 - loss: 1.3328 - val_accuracy: 0.9969 -
val_dice_coef: 0.0060 - val_loss: 1.3286 - learning_rate: 1.0000e-04
Epoch 2/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 496ms/step - accuracy: 0.9973 -
dice_coef: 0.0090 - loss: 1.0575
Epoch 2: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 526ms/step - accuracy: 0.9973 -
dice_coef: 0.0090 - loss: 1.0573 - val_accuracy: 0.9969 -
val_dice_coef: 0.0058 - val_loss: 1.1094 - learning_rate: 1.0000e-04
Epoch 3/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9968 -
dice_coef: 0.0290 - loss: 1.0098
Epoch 3: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 530ms/step - accuracy: 0.9968 -
dice_coef: 0.0289 - loss: 1.0098 - val_accuracy: 0.9969 -
val_dice_coef: 0.0061 - val_loss: 1.0798 - learning_rate: 1.0000e-04
Epoch 4/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9966 -
dice_coef: 0.0332 - loss: 0.9970
Epoch 4: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9966 -
dice_coef: 0.0332 - loss: 0.9970 - val_accuracy: 0.9969 -
val_dice_coef: 0.0061 - val_loss: 1.0229 - learning_rate: 1.0000e-04
Epoch 5/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9964 -
dice_coef: 0.0426 - loss: 0.9827
Epoch 5: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 530ms/step - accuracy: 0.9964 -
dice_coef: 0.0428 - loss: 0.9825 - val_accuracy: 0.9970 -
val_dice_coef: 0.0259 - val_loss: 0.9946 - learning_rate: 1.0000e-04
Epoch 6/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9964 -
dice_coef: 0.0681 - loss: 0.9535
Epoch 6: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 530ms/step - accuracy: 0.9964 -
dice_coef: 0.0682 - loss: 0.9534 - val_accuracy: 0.9970 -
val_dice_coef: 0.0351 - val_loss: 0.9826 - learning_rate: 1.0000e-04
Epoch 7/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9962 -
dice_coef: 0.0950 - loss: 0.9281
Epoch 7: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 530ms/step - accuracy: 0.9962 -
```

```
dice_coef: 0.0951 - loss: 0.9279 - val_accuracy: 0.9969 -
val_dice_coef: 0.0405 - val_loss: 0.9790 - learning_rate: 1.0000e-04
Epoch 8/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 497ms/step - accuracy: 0.9967 -
dice_coef: 0.1148 - loss: 0.9057
Epoch 8: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 528ms/step - accuracy: 0.9967 -
dice_coef: 0.1149 - loss: 0.9055 - val_accuracy: 0.9967 -
val_dice_coef: 0.0425 - val_loss: 0.9768 - learning_rate: 1.0000e-04
Epoch 9/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 498ms/step - accuracy: 0.9964 -
dice_coef: 0.1426 - loss: 0.8804
Epoch 9: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 529ms/step - accuracy: 0.9964 -
dice_coef: 0.1424 - loss: 0.8806 - val_accuracy: 0.9968 -
val_dice_coef: 0.0430 - val_loss: 0.9771 - learning_rate: 1.0000e-04
Epoch 10/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 498ms/step - accuracy: 0.9970 -
dice_coef: 0.1467 - loss: 0.8733
Epoch 10: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 529ms/step - accuracy: 0.9970 -
dice_coef: 0.1466 - loss: 0.8734 - val_accuracy: 0.9951 -
val_dice_coef: 0.0330 - val_loss: 1.0000 - learning_rate: 1.0000e-04
Epoch 11/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9964 -
dice_coef: 0.1124 - loss: 0.9106
Epoch 11: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 530ms/step - accuracy: 0.9964 -
dice_coef: 0.1129 - loss: 0.9101 - val_accuracy: 0.9969 -
val_dice_coef: 0.0448 - val_loss: 0.9767 - learning_rate: 1.0000e-04
Epoch 12/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9971 -
dice_coef: 0.1487 - loss: 0.8707
Epoch 12: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 530ms/step - accuracy: 0.9971 -
dice_coef: 0.1486 - loss: 0.8709 - val_accuracy: 0.9964 -
val_dice_coef: 0.0421 - val_loss: 0.9818 - learning_rate: 1.0000e-04
Epoch 13/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 498ms/step - accuracy: 0.9968 -
dice_coef: 0.1562 - loss: 0.8651
Epoch 13: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 529ms/step - accuracy: 0.9968 -
dice_coef: 0.1564 - loss: 0.8649 - val_accuracy: 0.9969 -
val_dice_coef: 0.0440 - val_loss: 0.9788 - learning_rate: 1.0000e-04
Epoch 14/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 498ms/step - accuracy: 0.9971 -
dice_coef: 0.2274 - loss: 0.7932
Epoch 14: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 529ms/step - accuracy: 0.9971 -
```

```
dice_coef: 0.2267 - loss: 0.7938 - val_accuracy: 0.9968 -
val_dice_coef: 0.0438 - val_loss: 0.9797 - learning_rate: 1.0000e-04
Epoch 15/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9969 -
dice_coef: 0.1544 - loss: 0.8678
Epoch 15: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 529ms/step - accuracy: 0.9969 -
dice_coef: 0.1547 - loss: 0.8676 - val_accuracy: 0.9969 -
val_dice_coef: 0.0443 - val_loss: 0.9786 - learning_rate: 1.0000e-04
Epoch 16/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 498ms/step - accuracy: 0.9969 -
dice_coef: 0.1415 - loss: 0.8791
Epoch 16: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 529ms/step - accuracy: 0.9969 -
dice_coef: 0.1419 - loss: 0.8787 - val_accuracy: 0.9969 -
val_dice_coef: 0.0445 - val_loss: 0.9796 - learning_rate: 1.0000e-04
Epoch 17/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9969 -
dice_coef: 0.1854 - loss: 0.8374
Epoch 17: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 530ms/step - accuracy: 0.9969 -
dice_coef: 0.1852 - loss: 0.8375 - val_accuracy: 0.9972 -
val_dice_coef: 0.0466 - val_loss: 0.9743 - learning_rate: 1.0000e-04
Epoch 18/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 497ms/step - accuracy: 0.9970 -
dice_coef: 0.1743 - loss: 0.8470
Epoch 18: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 528ms/step - accuracy: 0.9970 -
dice_coef: 0.1743 - loss: 0.8469 - val_accuracy: 0.9970 -
val_dice_coef: 0.0459 - val_loss: 0.9783 - learning_rate: 1.0000e-04
Epoch 19/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9974 -
dice_coef: 0.2514 - loss: 0.7666
Epoch 19: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 531ms/step - accuracy: 0.9974 -
dice_coef: 0.2506 - loss: 0.7675 - val_accuracy: 0.9973 -
val_dice_coef: 0.0403 - val_loss: 0.9825 - learning_rate: 1.0000e-04
Epoch 20/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9965 -
dice_coef: 0.1463 - loss: 0.8745
Epoch 20: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 530ms/step - accuracy: 0.9965 -
dice_coef: 0.1468 - loss: 0.8739 - val_accuracy: 0.9971 -
val_dice_coef: 0.0672 - val_loss: 0.9538 - learning_rate: 1.0000e-04
Epoch 21/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9963 -
dice_coef: 0.2002 - loss: 0.8208
Epoch 21: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 530ms/step - accuracy: 0.9963 -
```

```
dice_coef: 0.2005 - loss: 0.8205 - val_accuracy: 0.9967 -
val_dice_coef: 0.0809 - val_loss: 0.9415 - learning_rate: 1.0000e-04
Epoch 22/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9964 -
dice_coef: 0.2239 - loss: 0.7965
Epoch 22: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 531ms/step - accuracy: 0.9964 -
dice_coef: 0.2240 - loss: 0.7964 - val_accuracy: 0.9972 -
val_dice_coef: 0.1104 - val_loss: 0.9090 - learning_rate: 1.0000e-04
Epoch 23/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9970 -
dice_coef: 0.2948 - loss: 0.7225
Epoch 23: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 531ms/step - accuracy: 0.9970 -
dice_coef: 0.2948 - loss: 0.7225 - val_accuracy: 0.9971 -
val_dice_coef: 0.1344 - val_loss: 0.8852 - learning_rate: 1.0000e-04
Epoch 24/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9974 -
dice_coef: 0.2629 - loss: 0.7529
Epoch 24: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 530ms/step - accuracy: 0.9974 -
dice_coef: 0.2631 - loss: 0.7526 - val_accuracy: 0.9972 -
val_dice_coef: 0.1420 - val_loss: 0.8774 - learning_rate: 1.0000e-04
Epoch 25/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9971 -
dice_coef: 0.3414 - loss: 0.6758
Epoch 25: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 531ms/step - accuracy: 0.9971 -
dice_coef: 0.3406 - loss: 0.6766 - val_accuracy: 0.9969 -
val_dice_coef: 0.1659 - val_loss: 0.8523 - learning_rate: 1.0000e-04
Epoch 26/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9971 -
dice_coef: 0.3407 - loss: 0.6764
Epoch 26: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 531ms/step - accuracy: 0.9971 -
dice_coef: 0.3405 - loss: 0.6766 - val_accuracy: 0.9971 -
val_dice_coef: 0.1752 - val_loss: 0.8375 - learning_rate: 1.0000e-04
Epoch 27/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9975 -
dice_coef: 0.3384 - loss: 0.6774
Epoch 27: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 530ms/step - accuracy: 0.9975 -
dice_coef: 0.3382 - loss: 0.6776 - val_accuracy: 0.9967 -
val_dice_coef: 0.1780 - val_loss: 0.8410 - learning_rate: 1.0000e-04
Epoch 28/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9973 -
dice_coef: 0.2995 - loss: 0.7179
Epoch 28: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 531ms/step - accuracy: 0.9973 -
```

```
dice_coef: 0.2995 - loss: 0.7179 - val_accuracy: 0.9972 -
val_dice_coef: 0.1998 - val_loss: 0.8126 - learning_rate: 1.0000e-04
Epoch 29/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9972 -
dice_coef: 0.3216 - loss: 0.6960
Epoch 29: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 534ms/step - accuracy: 0.9972 -
dice_coef: 0.3215 - loss: 0.6961 - val_accuracy: 0.9973 -
val_dice_coef: 0.2331 - val_loss: 0.7789 - learning_rate: 1.0000e-04
Epoch 30/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9975 -
dice_coef: 0.3368 - loss: 0.6807
Epoch 30: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 530ms/step - accuracy: 0.9975 -
dice_coef: 0.3369 - loss: 0.6806 - val_accuracy: 0.9974 -
val_dice_coef: 0.2336 - val_loss: 0.7785 - learning_rate: 1.0000e-04
Epoch 31/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9977 -
dice_coef: 0.3304 - loss: 0.6843
Epoch 31: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9977 -
dice_coef: 0.3305 - loss: 0.6842 - val_accuracy: 0.9973 -
val_dice_coef: 0.2500 - val_loss: 0.7644 - learning_rate: 1.0000e-04
Epoch 32/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9972 -
dice_coef: 0.3429 - loss: 0.6744
Epoch 32: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 534ms/step - accuracy: 0.9972 -
dice_coef: 0.3430 - loss: 0.6743 - val_accuracy: 0.9973 -
val_dice_coef: 0.2927 - val_loss: 0.7201 - learning_rate: 1.0000e-04
Epoch 33/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9975 -
dice_coef: 0.3470 - loss: 0.6688
Epoch 33: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 529ms/step - accuracy: 0.9975 -
dice_coef: 0.3467 - loss: 0.6691 - val_accuracy: 0.9971 -
val_dice_coef: 0.2499 - val_loss: 0.7640 - learning_rate: 1.0000e-04
Epoch 34/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9975 -
dice_coef: 0.3797 - loss: 0.6382
Epoch 34: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9975 -
dice_coef: 0.3797 - loss: 0.6382 - val_accuracy: 0.9972 -
val_dice_coef: 0.2885 - val_loss: 0.7251 - learning_rate: 1.0000e-04
Epoch 35/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9972 -
dice_coef: 0.3448 - loss: 0.6718
Epoch 35: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 531ms/step - accuracy: 0.9972 -
```

```
dice_coef: 0.3451 - loss: 0.6715 - val_accuracy: 0.9973 -
val_dice_coef: 0.2505 - val_loss: 0.7630 - learning_rate: 1.0000e-04
Epoch 36/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9972 -
dice_coef: 0.3776 - loss: 0.6406
Epoch 36: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 531ms/step - accuracy: 0.9972 -
dice_coef: 0.3776 - loss: 0.6407 - val_accuracy: 0.9972 -
val_dice_coef: 0.2865 - val_loss: 0.7267 - learning_rate: 1.0000e-04
Epoch 37/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9974 -
dice_coef: 0.3719 - loss: 0.6442
Epoch 37: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9974 -
dice_coef: 0.3719 - loss: 0.6442 - val_accuracy: 0.9971 -
val_dice_coef: 0.2736 - val_loss: 0.7395 - learning_rate: 1.0000e-04
Epoch 38/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 498ms/step - accuracy: 0.9974 -
dice_coef: 0.3586 - loss: 0.6587
Epoch 38: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 529ms/step - accuracy: 0.9974 -
dice_coef: 0.3588 - loss: 0.6584 - val_accuracy: 0.9973 -
val_dice_coef: 0.2705 - val_loss: 0.7411 - learning_rate: 1.0000e-04
Epoch 39/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9975 -
dice_coef: 0.3957 - loss: 0.6207
Epoch 39: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9975 -
dice_coef: 0.3953 - loss: 0.6211 - val_accuracy: 0.9971 -
val_dice_coef: 0.2845 - val_loss: 0.7279 - learning_rate: 1.0000e-04
Epoch 40/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9974 -
dice_coef: 0.4078 - loss: 0.6089
Epoch 40: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 531ms/step - accuracy: 0.9974 -
dice_coef: 0.4077 - loss: 0.6090 - val_accuracy: 0.9975 -
val_dice_coef: 0.2770 - val_loss: 0.7352 - learning_rate: 1.0000e-04
Epoch 41/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9975 -
dice_coef: 0.3876 - loss: 0.6288
Epoch 41: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 531ms/step - accuracy: 0.9975 -
dice_coef: 0.3877 - loss: 0.6287 - val_accuracy: 0.9974 -
val_dice_coef: 0.2712 - val_loss: 0.7420 - learning_rate: 1.0000e-04
Epoch 42/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9974 -
dice_coef: 0.3664 - loss: 0.6507
Epoch 42: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9974 -
dice_coef: 0.3668 - loss: 0.6504 - val_accuracy: 0.9970 -
```

```
val_dice_coef: 0.3274 - val_loss: 0.6871 - learning_rate: 1.0000e-04
Epoch 43/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9976 -
dice_coef: 0.3880 - loss: 0.6281
Epoch 43: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 530ms/step - accuracy: 0.9976 -
dice_coef: 0.3879 - loss: 0.6282 - val_accuracy: 0.9975 -
val_dice_coef: 0.3136 - val_loss: 0.6993 - learning_rate: 1.0000e-04
Epoch 44/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9976 -
dice_coef: 0.3996 - loss: 0.6160
Epoch 44: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 534ms/step - accuracy: 0.9976 -
dice_coef: 0.3994 - loss: 0.6161 - val_accuracy: 0.9975 -
val_dice_coef: 0.3263 - val_loss: 0.6843 - learning_rate: 1.0000e-04
Epoch 45/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9977 -
dice_coef: 0.3831 - loss: 0.6330
Epoch 45: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 531ms/step - accuracy: 0.9977 -
dice_coef: 0.3836 - loss: 0.6326 - val_accuracy: 0.9974 -
val_dice_coef: 0.3294 - val_loss: 0.6838 - learning_rate: 1.0000e-04
Epoch 46/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9978 -
dice_coef: 0.3711 - loss: 0.6443
Epoch 46: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9978 -
dice_coef: 0.3713 - loss: 0.6441 - val_accuracy: 0.9975 -
val_dice_coef: 0.3066 - val_loss: 0.7061 - learning_rate: 1.0000e-04
Epoch 47/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9977 -
dice_coef: 0.4654 - loss: 0.5504
Epoch 47: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 530ms/step - accuracy: 0.9977 -
dice_coef: 0.4649 - loss: 0.5509 - val_accuracy: 0.9976 -
val_dice_coef: 0.2976 - val_loss: 0.7143 - learning_rate: 1.0000e-04
Epoch 48/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9976 -
dice_coef: 0.4200 - loss: 0.5960
Epoch 48: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9976 -
dice_coef: 0.4200 - loss: 0.5960 - val_accuracy: 0.9974 -
val_dice_coef: 0.3171 - val_loss: 0.6939 - learning_rate: 1.0000e-04
Epoch 49/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9976 -
dice_coef: 0.4482 - loss: 0.5683
Epoch 49: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 531ms/step - accuracy: 0.9976 -
dice_coef: 0.4479 - loss: 0.5686 - val_accuracy: 0.9976 -
```

```
val_dice_coef: 0.2951 - val_loss: 0.7175 - learning_rate: 1.0000e-04
Epoch 50/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9977 -
dice_coef: 0.4253 - loss: 0.5914
Epoch 50: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9977 -
dice_coef: 0.4253 - loss: 0.5914 - val_accuracy: 0.9975 -
val_dice_coef: 0.3256 - val_loss: 0.6864 - learning_rate: 1.0000e-04
Epoch 51/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9977 -
dice_coef: 0.4648 - loss: 0.5505
Epoch 51: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9977 -
dice_coef: 0.4644 - loss: 0.5509 - val_accuracy: 0.9976 -
val_dice_coef: 0.3544 - val_loss: 0.6568 - learning_rate: 1.0000e-04
Epoch 52/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9976 -
dice_coef: 0.4099 - loss: 0.6079
Epoch 52: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9976 -
dice_coef: 0.4101 - loss: 0.6075 - val_accuracy: 0.9976 -
val_dice_coef: 0.3637 - val_loss: 0.6474 - learning_rate: 1.0000e-04
Epoch 53/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9977 -
dice_coef: 0.4416 - loss: 0.5741
Epoch 53: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9977 -
dice_coef: 0.4413 - loss: 0.5744 - val_accuracy: 0.9975 -
val_dice_coef: 0.3706 - val_loss: 0.6412 - learning_rate: 1.0000e-04
Epoch 54/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9975 -
dice_coef: 0.4306 - loss: 0.5870
Epoch 54: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9975 -
dice_coef: 0.4307 - loss: 0.5869 - val_accuracy: 0.9977 -
val_dice_coef: 0.3209 - val_loss: 0.6915 - learning_rate: 1.0000e-04
Epoch 55/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9975 -
dice_coef: 0.4470 - loss: 0.5692
Epoch 55: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 531ms/step - accuracy: 0.9975 -
dice_coef: 0.4469 - loss: 0.5693 - val_accuracy: 0.9975 -
val_dice_coef: 0.3513 - val_loss: 0.6601 - learning_rate: 1.0000e-04
Epoch 56/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9974 -
dice_coef: 0.3968 - loss: 0.6225
Epoch 56: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9974 -
dice_coef: 0.3973 - loss: 0.6220 - val_accuracy: 0.9977 -
```

```
val_dice_coef: 0.3380 - val_loss: 0.6744 - learning_rate: 1.0000e-04
Epoch 57/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9976 -
dice_coef: 0.4567 - loss: 0.5596
Epoch 57: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9976 -
dice_coef: 0.4567 - loss: 0.5597 - val_accuracy: 0.9976 -
val_dice_coef: 0.3227 - val_loss: 0.6904 - learning_rate: 1.0000e-04
Epoch 58/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9978 -
dice_coef: 0.4858 - loss: 0.5300
Epoch 58: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 531ms/step - accuracy: 0.9978 -
dice_coef: 0.4856 - loss: 0.5302 - val_accuracy: 0.9977 -
val_dice_coef: 0.3715 - val_loss: 0.6399 - learning_rate: 1.0000e-04
Epoch 59/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9974 -
dice_coef: 0.4240 - loss: 0.5952
Epoch 59: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9974 -
dice_coef: 0.4242 - loss: 0.5950 - val_accuracy: 0.9977 -
val_dice_coef: 0.3568 - val_loss: 0.6558 - learning_rate: 1.0000e-04
Epoch 60/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9977 -
dice_coef: 0.4750 - loss: 0.5409
Epoch 60: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 531ms/step - accuracy: 0.9977 -
dice_coef: 0.4745 - loss: 0.5414 - val_accuracy: 0.9976 -
val_dice_coef: 0.3795 - val_loss: 0.6326 - learning_rate: 1.0000e-04
Epoch 61/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9976 -
dice_coef: 0.4190 - loss: 0.5996
Epoch 61: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9976 -
dice_coef: 0.4194 - loss: 0.5991 - val_accuracy: 0.9976 -
val_dice_coef: 0.3636 - val_loss: 0.6495 - learning_rate: 1.0000e-04
Epoch 62/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9978 -
dice_coef: 0.4537 - loss: 0.5624
Epoch 62: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 532ms/step - accuracy: 0.9978 -
dice_coef: 0.4540 - loss: 0.5621 - val_accuracy: 0.9976 -
val_dice_coef: 0.3964 - val_loss: 0.6157 - learning_rate: 1.0000e-04
Epoch 63/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9975 -
dice_coef: 0.4693 - loss: 0.5478
Epoch 63: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9975 -
dice_coef: 0.4692 - loss: 0.5478 - val_accuracy: 0.9976 -
```

```
val_dice_coef: 0.3839 - val_loss: 0.6278 - learning_rate: 1.0000e-04
Epoch 64/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9979 -
dice_coef: 0.4422 - loss: 0.5734
Epoch 64: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9979 -
dice_coef: 0.4421 - loss: 0.5735 - val_accuracy: 0.9977 -
val_dice_coef: 0.3574 - val_loss: 0.6564 - learning_rate: 1.0000e-04
Epoch 65/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9978 -
dice_coef: 0.4214 - loss: 0.5936
Epoch 65: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9978 -
dice_coef: 0.4219 - loss: 0.5932 - val_accuracy: 0.9977 -
val_dice_coef: 0.3811 - val_loss: 0.6303 - learning_rate: 1.0000e-04
Epoch 66/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9976 -
dice_coef: 0.4710 - loss: 0.5455
Epoch 66: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9976 -
dice_coef: 0.4709 - loss: 0.5456 - val_accuracy: 0.9977 -
val_dice_coef: 0.3651 - val_loss: 0.6477 - learning_rate: 1.0000e-04
Epoch 67/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 503ms/step - accuracy: 0.9978 -
dice_coef: 0.4464 - loss: 0.5700
Epoch 67: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 535ms/step - accuracy: 0.9978 -
dice_coef: 0.4463 - loss: 0.5701 - val_accuracy: 0.9976 -
val_dice_coef: 0.3973 - val_loss: 0.6133 - learning_rate: 1.0000e-04
Epoch 68/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9979 -
dice_coef: 0.4918 - loss: 0.5228
Epoch 68: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 531ms/step - accuracy: 0.9979 -
dice_coef: 0.4915 - loss: 0.5230 - val_accuracy: 0.9973 -
val_dice_coef: 0.3805 - val_loss: 0.6332 - learning_rate: 1.0000e-04
Epoch 69/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9978 -
dice_coef: 0.4192 - loss: 0.5970
Epoch 69: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 533ms/step - accuracy: 0.9978 -
dice_coef: 0.4194 - loss: 0.5968 - val_accuracy: 0.9978 -
val_dice_coef: 0.3987 - val_loss: 0.6142 - learning_rate: 1.0000e-04
Epoch 70/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9980 -
dice_coef: 0.3907 - loss: 0.6243
Epoch 70: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9980 -
dice_coef: 0.3916 - loss: 0.6235 - val_accuracy: 0.9976 -
```

```
val_dice_coef: 0.3953 - val_loss: 0.6164 - learning_rate: 1.0000e-04
Epoch 71/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9980 -
dice_coef: 0.4841 - loss: 0.5310
Epoch 71: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9980 -
dice_coef: 0.4841 - loss: 0.5310 - val_accuracy: 0.9978 -
val_dice_coef: 0.3779 - val_loss: 0.6366 - learning_rate: 1.0000e-04
Epoch 72/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9977 -
dice_coef: 0.4880 - loss: 0.5291
Epoch 72: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9977 -
dice_coef: 0.4878 - loss: 0.5293 - val_accuracy: 0.9978 -
val_dice_coef: 0.3621 - val_loss: 0.6506 - learning_rate: 1.0000e-04
Epoch 73/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9980 -
dice_coef: 0.4843 - loss: 0.5301
Epoch 73: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9980 -
dice_coef: 0.4842 - loss: 0.5302 - val_accuracy: 0.9978 -
val_dice_coef: 0.3385 - val_loss: 0.6746 - learning_rate: 1.0000e-04
Epoch 74/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9979 -
dice_coef: 0.4371 - loss: 0.5795
Epoch 74: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 531ms/step - accuracy: 0.9979 -
dice_coef: 0.4374 - loss: 0.5791 - val_accuracy: 0.9978 -
val_dice_coef: 0.3837 - val_loss: 0.6282 - learning_rate: 1.0000e-04
Epoch 75/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 505ms/step - accuracy: 0.9978 -
dice_coef: 0.5191 - loss: 0.4961
Epoch 75: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 537ms/step - accuracy: 0.9978 -
dice_coef: 0.5186 - loss: 0.4966 - val_accuracy: 0.9977 -
val_dice_coef: 0.3893 - val_loss: 0.6231 - learning_rate: 1.0000e-04
Epoch 76/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9978 -
dice_coef: 0.4989 - loss: 0.5178
Epoch 76: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9978 -
dice_coef: 0.4987 - loss: 0.5180 - val_accuracy: 0.9977 -
val_dice_coef: 0.3867 - val_loss: 0.6273 - learning_rate: 1.0000e-04
Epoch 77/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9980 -
dice_coef: 0.4879 - loss: 0.5274
Epoch 77: val_loss did not improve from 0.56874

Epoch 77: ReduceLROnPlateau reducing learning rate to
```

```
4.999999873689376e-05.
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9980 -
dice_coef: 0.4877 - loss: 0.5275 - val_accuracy: 0.9977 -
val_dice_coef: 0.3723 - val_loss: 0.6412 - learning_rate: 1.0000e-04
Epoch 78/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9981 -
dice_coef: 0.5265 - loss: 0.4883
Epoch 78: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9981 -
dice_coef: 0.5263 - loss: 0.4885 - val_accuracy: 0.9978 -
val_dice_coef: 0.3980 - val_loss: 0.6139 - learning_rate: 5.0000e-05
Epoch 79/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 503ms/step - accuracy: 0.9981 -
dice_coef: 0.5049 - loss: 0.5100
Epoch 79: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 535ms/step - accuracy: 0.9981 -
dice_coef: 0.5046 - loss: 0.5103 - val_accuracy: 0.9978 -
val_dice_coef: 0.3893 - val_loss: 0.6223 - learning_rate: 5.0000e-05
Epoch 80/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9979 -
dice_coef: 0.4887 - loss: 0.5275
Epoch 80: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 531ms/step - accuracy: 0.9979 -
dice_coef: 0.4885 - loss: 0.5276 - val_accuracy: 0.9979 -
val_dice_coef: 0.4043 - val_loss: 0.6081 - learning_rate: 5.0000e-05
Epoch 81/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9980 -
dice_coef: 0.4979 - loss: 0.5177
Epoch 81: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9980 -
dice_coef: 0.4981 - loss: 0.5175 - val_accuracy: 0.9978 -
val_dice_coef: 0.4101 - val_loss: 0.6025 - learning_rate: 5.0000e-05
Epoch 82/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9980 -
dice_coef: 0.4831 - loss: 0.5328
Epoch 82: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 533ms/step - accuracy: 0.9980 -
dice_coef: 0.4832 - loss: 0.5327 - val_accuracy: 0.9978 -
val_dice_coef: 0.4165 - val_loss: 0.5953 - learning_rate: 5.0000e-05
Epoch 83/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9980 -
dice_coef: 0.4652 - loss: 0.5502
Epoch 83: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 534ms/step - accuracy: 0.9980 -
dice_coef: 0.4653 - loss: 0.5501 - val_accuracy: 0.9978 -
val_dice_coef: 0.4081 - val_loss: 0.6051 - learning_rate: 5.0000e-05
Epoch 84/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9982 -
dice_coef: 0.4833 - loss: 0.5305
```

```
Epoch 84: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 534ms/step - accuracy: 0.9982 -
dice_coef: 0.4834 - loss: 0.5305 - val_accuracy: 0.9978 -
val_dice_coef: 0.3908 - val_loss: 0.6221 - learning_rate: 5.0000e-05
Epoch 85/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9979 -
dice_coef: 0.5005 - loss: 0.5154
Epoch 85: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9979 -
dice_coef: 0.5006 - loss: 0.5152 - val_accuracy: 0.9977 -
val_dice_coef: 0.3949 - val_loss: 0.6181 - learning_rate: 5.0000e-05
Epoch 86/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9977 -
dice_coef: 0.4892 - loss: 0.5282
Epoch 86: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9977 -
dice_coef: 0.4893 - loss: 0.5280 - val_accuracy: 0.9978 -
val_dice_coef: 0.4079 - val_loss: 0.6047 - learning_rate: 5.0000e-05
Epoch 87/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9981 -
dice_coef: 0.5325 - loss: 0.4813
Epoch 87: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9981 -
dice_coef: 0.5323 - loss: 0.4815 - val_accuracy: 0.9977 -
val_dice_coef: 0.3989 - val_loss: 0.6142 - learning_rate: 5.0000e-05
Epoch 88/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9980 -
dice_coef: 0.4810 - loss: 0.5350
Epoch 88: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9980 -
dice_coef: 0.4813 - loss: 0.5348 - val_accuracy: 0.9978 -
val_dice_coef: 0.4052 - val_loss: 0.6088 - learning_rate: 5.0000e-05
Epoch 89/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9981 -
dice_coef: 0.4841 - loss: 0.5304
Epoch 89: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9981 -
dice_coef: 0.4840 - loss: 0.5305 - val_accuracy: 0.9979 -
val_dice_coef: 0.4048 - val_loss: 0.6074 - learning_rate: 5.0000e-05
Epoch 90/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 504ms/step - accuracy: 0.9980 -
dice_coef: 0.5064 - loss: 0.5081
Epoch 90: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 536ms/step - accuracy: 0.9980 -
dice_coef: 0.5067 - loss: 0.5078 - val_accuracy: 0.9978 -
val_dice_coef: 0.3985 - val_loss: 0.6144 - learning_rate: 5.0000e-05
Epoch 91/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9980 -
dice_coef: 0.5205 - loss: 0.4945
```

```
Epoch 91: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9980 -
dice_coef: 0.5203 - loss: 0.4947 - val_accuracy: 0.9977 -
val_dice_coef: 0.4167 - val_loss: 0.5954 - learning_rate: 5.0000e-05
Epoch 92/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9979 -
dice_coef: 0.5094 - loss: 0.5063
Epoch 92: val_loss did not improve from 0.56874

Epoch 92: ReduceLROnPlateau reducing learning rate to
2.499999936844688e-05.
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 532ms/step - accuracy: 0.9979 -
dice_coef: 0.5093 - loss: 0.5063 - val_accuracy: 0.9979 -
val_dice_coef: 0.4144 - val_loss: 0.5996 - learning_rate: 5.0000e-05
Epoch 93/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 503ms/step - accuracy: 0.9981 -
dice_coef: 0.4913 - loss: 0.5231
Epoch 93: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 534ms/step - accuracy: 0.9981 -
dice_coef: 0.4915 - loss: 0.5230 - val_accuracy: 0.9978 -
val_dice_coef: 0.4061 - val_loss: 0.6077 - learning_rate: 2.5000e-05
Epoch 94/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9980 -
dice_coef: 0.4898 - loss: 0.5253
Epoch 94: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9980 -
dice_coef: 0.4897 - loss: 0.5254 - val_accuracy: 0.9979 -
val_dice_coef: 0.4145 - val_loss: 0.5991 - learning_rate: 2.5000e-05
Epoch 95/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 503ms/step - accuracy: 0.9978 -
dice_coef: 0.5202 - loss: 0.4954
Epoch 95: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 534ms/step - accuracy: 0.9978 -
dice_coef: 0.5202 - loss: 0.4953 - val_accuracy: 0.9979 -
val_dice_coef: 0.4124 - val_loss: 0.6002 - learning_rate: 2.5000e-05
Epoch 96/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9982 -
dice_coef: 0.5678 - loss: 0.4456
Epoch 96: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 531ms/step - accuracy: 0.9982 -
dice_coef: 0.5674 - loss: 0.4460 - val_accuracy: 0.9978 -
val_dice_coef: 0.3985 - val_loss: 0.6152 - learning_rate: 2.5000e-05
Epoch 97/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 509ms/step - accuracy: 0.9980 -
dice_coef: 0.5455 - loss: 0.4690
Epoch 97: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 540ms/step - accuracy: 0.9980 -
dice_coef: 0.5450 - loss: 0.4695 - val_accuracy: 0.9979 -
val_dice_coef: 0.4102 - val_loss: 0.6029 - learning_rate: 2.5000e-05
```

```
Epoch 98/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9983 -
dice_coef: 0.4816 - loss: 0.5319
Epoch 98: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 530ms/step - accuracy: 0.9983 -
dice_coef: 0.4819 - loss: 0.5317 - val_accuracy: 0.9979 -
val_dice_coef: 0.4133 - val_loss: 0.5991 - learning_rate: 2.5000e-05
Epoch 99/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 509ms/step - accuracy: 0.9981 -
dice_coef: 0.5148 - loss: 0.5002
Epoch 99: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 540ms/step - accuracy: 0.9981 -
dice_coef: 0.5151 - loss: 0.4999 - val_accuracy: 0.9979 -
val_dice_coef: 0.4095 - val_loss: 0.6035 - learning_rate: 2.5000e-05
Epoch 100/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9981 -
dice_coef: 0.5222 - loss: 0.4920
Epoch 100: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 531ms/step - accuracy: 0.9981 -
dice_coef: 0.5220 - loss: 0.4921 - val_accuracy: 0.9979 -
val_dice_coef: 0.4138 - val_loss: 0.5979 - learning_rate: 2.5000e-05
Epoch 101/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 503ms/step - accuracy: 0.9982 -
dice_coef: 0.5193 - loss: 0.4940
Epoch 101: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 535ms/step - accuracy: 0.9982 -
dice_coef: 0.5195 - loss: 0.4939 - val_accuracy: 0.9978 -
val_dice_coef: 0.4193 - val_loss: 0.5932 - learning_rate: 2.5000e-05
Epoch 102/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9981 -
dice_coef: 0.5080 - loss: 0.5061
Epoch 102: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9981 -
dice_coef: 0.5083 - loss: 0.5058 - val_accuracy: 0.9979 -
val_dice_coef: 0.4137 - val_loss: 0.5990 - learning_rate: 2.5000e-05
Epoch 103/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9983 -
dice_coef: 0.5703 - loss: 0.4421
Epoch 103: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9983 -
dice_coef: 0.5697 - loss: 0.4427 - val_accuracy: 0.9979 -
val_dice_coef: 0.4120 - val_loss: 0.6006 - learning_rate: 2.5000e-05
Epoch 104/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 503ms/step - accuracy: 0.9982 -
dice_coef: 0.5437 - loss: 0.4692
Epoch 104: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 534ms/step - accuracy: 0.9982 -
dice_coef: 0.5437 - loss: 0.4692 - val_accuracy: 0.9978 -
val_dice_coef: 0.4027 - val_loss: 0.6118 - learning_rate: 2.5000e-05
```

```
Epoch 105/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9982 -
dice_coef: 0.5152 - loss: 0.4977
Epoch 105: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9982 -
dice_coef: 0.5153 - loss: 0.4977 - val_accuracy: 0.9978 -
val_dice_coef: 0.4157 - val_loss: 0.5972 - learning_rate: 2.5000e-05
Epoch 106/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 504ms/step - accuracy: 0.9981 -
dice_coef: 0.5230 - loss: 0.4921
Epoch 106: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 536ms/step - accuracy: 0.9981 -
dice_coef: 0.5229 - loss: 0.4921 - val_accuracy: 0.9979 -
val_dice_coef: 0.4100 - val_loss: 0.6041 - learning_rate: 2.5000e-05
Epoch 107/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9981 -
dice_coef: 0.5518 - loss: 0.4618
Epoch 107: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9981 -
dice_coef: 0.5516 - loss: 0.4620 - val_accuracy: 0.9978 -
val_dice_coef: 0.3990 - val_loss: 0.6138 - learning_rate: 2.5000e-05
Epoch 108/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9982 -
dice_coef: 0.5323 - loss: 0.4818
Epoch 108: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9982 -
dice_coef: 0.5325 - loss: 0.4816 - val_accuracy: 0.9978 -
val_dice_coef: 0.4154 - val_loss: 0.5976 - learning_rate: 2.5000e-05
Epoch 109/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9983 -
dice_coef: 0.5277 - loss: 0.4849
Epoch 109: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9983 -
dice_coef: 0.5275 - loss: 0.4851 - val_accuracy: 0.9979 -
val_dice_coef: 0.4117 - val_loss: 0.6012 - learning_rate: 2.5000e-05
Epoch 110/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9981 -
dice_coef: 0.5391 - loss: 0.4748
Epoch 110: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9981 -
dice_coef: 0.5389 - loss: 0.4750 - val_accuracy: 0.9978 -
val_dice_coef: 0.4148 - val_loss: 0.5974 - learning_rate: 2.5000e-05
Epoch 111/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9979 -
dice_coef: 0.4974 - loss: 0.5186
Epoch 111: val_loss did not improve from 0.56874

Epoch 111: ReduceLROnPlateau reducing learning rate to
1.249999968422344e-05.
```

```
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 533ms/step - accuracy: 0.9979 -
dice_coef: 0.4976 - loss: 0.5184 - val_accuracy: 0.9978 -
val_dice_coef: 0.4097 - val_loss: 0.6030 - learning_rate: 2.5000e-05
Epoch 112/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9981 -
dice_coef: 0.5041 - loss: 0.5108
Epoch 112: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 534ms/step - accuracy: 0.9981 -
dice_coef: 0.5042 - loss: 0.5107 - val_accuracy: 0.9978 -
val_dice_coef: 0.4147 - val_loss: 0.5985 - learning_rate: 1.2500e-05
Epoch 113/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9982 -
dice_coef: 0.5259 - loss: 0.4873
Epoch 113: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 534ms/step - accuracy: 0.9982 -
dice_coef: 0.5262 - loss: 0.4870 - val_accuracy: 0.9978 -
val_dice_coef: 0.4096 - val_loss: 0.6043 - learning_rate: 1.2500e-05
Epoch 114/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9982 -
dice_coef: 0.5256 - loss: 0.4879
Epoch 114: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9982 -
dice_coef: 0.5254 - loss: 0.4882 - val_accuracy: 0.9978 -
val_dice_coef: 0.4112 - val_loss: 0.6018 - learning_rate: 1.2500e-05
Epoch 115/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9982 -
dice_coef: 0.4716 - loss: 0.5422
Epoch 115: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 532ms/step - accuracy: 0.9982 -
dice_coef: 0.4722 - loss: 0.5416 - val_accuracy: 0.9978 -
val_dice_coef: 0.4150 - val_loss: 0.5975 - learning_rate: 1.2500e-05
Epoch 116/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9982 -
dice_coef: 0.5688 - loss: 0.4438
Epoch 116: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 532ms/step - accuracy: 0.9982 -
dice_coef: 0.5682 - loss: 0.4444 - val_accuracy: 0.9978 -
val_dice_coef: 0.4008 - val_loss: 0.6123 - learning_rate: 1.2500e-05
Epoch 117/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 503ms/step - accuracy: 0.9981 -
dice_coef: 0.5481 - loss: 0.4656
Epoch 117: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 534ms/step - accuracy: 0.9981 -
dice_coef: 0.5481 - loss: 0.4656 - val_accuracy: 0.9979 -
val_dice_coef: 0.4051 - val_loss: 0.6076 - learning_rate: 1.2500e-05
Epoch 118/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9981 -
dice_coef: 0.5073 - loss: 0.5066
Epoch 118: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 531ms/step - accuracy: 0.9981 -
```

```
dice_coef: 0.5073 - loss: 0.5066 - val_accuracy: 0.9978 -
val_dice_coef: 0.4004 - val_loss: 0.6123 - learning_rate: 1.2500e-05
Epoch 119/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9982 -
dice_coef: 0.5308 - loss: 0.4818
Epoch 119: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 534ms/step - accuracy: 0.9982 -
dice_coef: 0.5309 - loss: 0.4818 - val_accuracy: 0.9978 -
val_dice_coef: 0.3964 - val_loss: 0.6170 - learning_rate: 1.2500e-05
Epoch 120/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9982 -
dice_coef: 0.4907 - loss: 0.5226
Epoch 120: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9982 -
dice_coef: 0.4914 - loss: 0.5219 - val_accuracy: 0.9978 -
val_dice_coef: 0.3940 - val_loss: 0.6198 - learning_rate: 1.2500e-05
Epoch 121/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9981 -
dice_coef: 0.5052 - loss: 0.5088
Epoch 121: val_loss did not improve from 0.56874

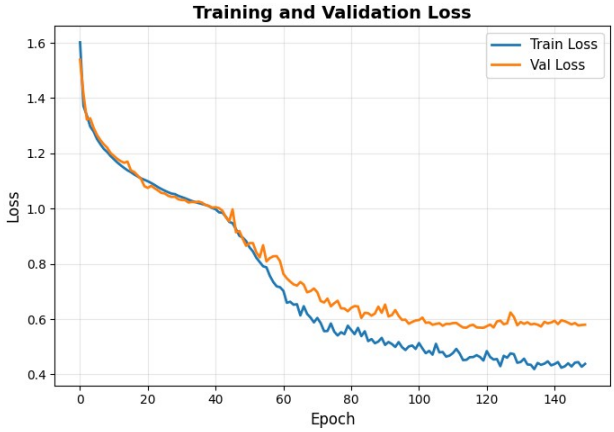
Epoch 121: ReduceLROnPlateau reducing learning rate to
6.24999984211172e-06.
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9981 -
dice_coef: 0.5054 - loss: 0.5086 - val_accuracy: 0.9978 -
val_dice_coef: 0.3946 - val_loss: 0.6191 - learning_rate: 1.2500e-05
Epoch 122/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9982 -
dice_coef: 0.5652 - loss: 0.4474
Epoch 122: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 532ms/step - accuracy: 0.9982 -
dice_coef: 0.5646 - loss: 0.4479 - val_accuracy: 0.9978 -
val_dice_coef: 0.4000 - val_loss: 0.6133 - learning_rate: 6.2500e-06
Epoch 123/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9982 -
dice_coef: 0.5308 - loss: 0.4823
Epoch 123: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 533ms/step - accuracy: 0.9982 -
dice_coef: 0.5310 - loss: 0.4822 - val_accuracy: 0.9978 -
val_dice_coef: 0.3934 - val_loss: 0.6202 - learning_rate: 6.2500e-06
Epoch 124/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9981 -
dice_coef: 0.5669 - loss: 0.4457
Epoch 124: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 534ms/step - accuracy: 0.9981 -
dice_coef: 0.5669 - loss: 0.4458 - val_accuracy: 0.9978 -
val_dice_coef: 0.3990 - val_loss: 0.6143 - learning_rate: 6.2500e-06
Epoch 125/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9983 -
```

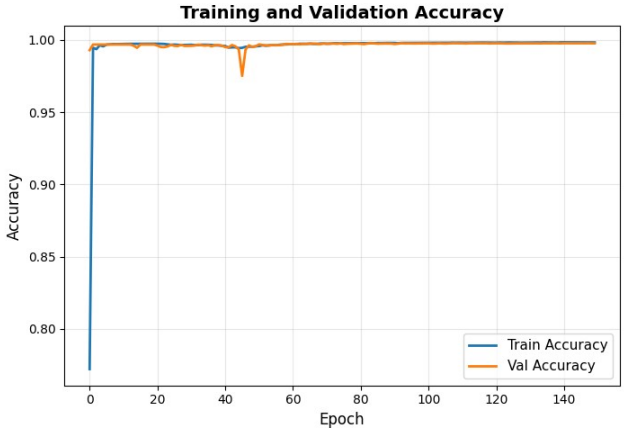
```
dice_coef: 0.5388 - loss: 0.4732
Epoch 125: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9983 -
dice_coef: 0.5387 - loss: 0.4733 - val_accuracy: 0.9978 -
val_dice_coef: 0.3976 - val_loss: 0.6161 - learning_rate: 6.2500e-06
Epoch 126/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9981 -
dice_coef: 0.5170 - loss: 0.4974
Epoch 126: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 532ms/step - accuracy: 0.9981 -
dice_coef: 0.5171 - loss: 0.4973 - val_accuracy: 0.9978 -
val_dice_coef: 0.3987 - val_loss: 0.6150 - learning_rate: 6.2500e-06
Epoch 127/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9982 -
dice_coef: 0.5288 - loss: 0.4839
Epoch 127: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9982 -
dice_coef: 0.5290 - loss: 0.4837 - val_accuracy: 0.9979 -
val_dice_coef: 0.3974 - val_loss: 0.6164 - learning_rate: 6.2500e-06
Epoch 128/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9982 -
dice_coef: 0.5648 - loss: 0.4475
Epoch 128: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.9982 -
dice_coef: 0.5644 - loss: 0.4478 - val_accuracy: 0.9978 -
val_dice_coef: 0.4075 - val_loss: 0.6055 - learning_rate: 6.2500e-06
Epoch 129/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9981 -
dice_coef: 0.5452 - loss: 0.4679
Epoch 129: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 42s 533ms/step - accuracy: 0.9981 -
dice_coef: 0.5452 - loss: 0.4679 - val_accuracy: 0.9978 -
val_dice_coef: 0.4012 - val_loss: 0.6121 - learning_rate: 6.2500e-06
Epoch 130/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9981 -
dice_coef: 0.5254 - loss: 0.4885
Epoch 130: val_loss did not improve from 0.56874
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 534ms/step - accuracy: 0.9981 -
dice_coef: 0.5258 - loss: 0.4882 - val_accuracy: 0.9979 -
val_dice_coef: 0.4065 - val_loss: 0.6064 - learning_rate: 6.2500e-06
Epoch 131/200
70/70 ━━━━━━━━━━━━━━━━━━━━ 0s 502ms/step - accuracy: 0.9982 -
dice_coef: 0.5312 - loss: 0.4821
Epoch 131: val_loss did not improve from 0.56874

Epoch 131: ReduceLROnPlateau reducing learning rate to
3.12499992105586e-06.
70/70 ━━━━━━━━━━━━━━━━━━━━ 43s 534ms/step - accuracy: 0.9982 -
dice_coef: 0.5315 - loss: 0.4817 - val_accuracy: 0.9978 -
```

```
val_dice_coef: 0.4098 - val_loss: 0.6036 - learning_rate: 6.2500e-06
Epoch 131: early stopping
Restoring model weights from the end of the best epoch: 101.

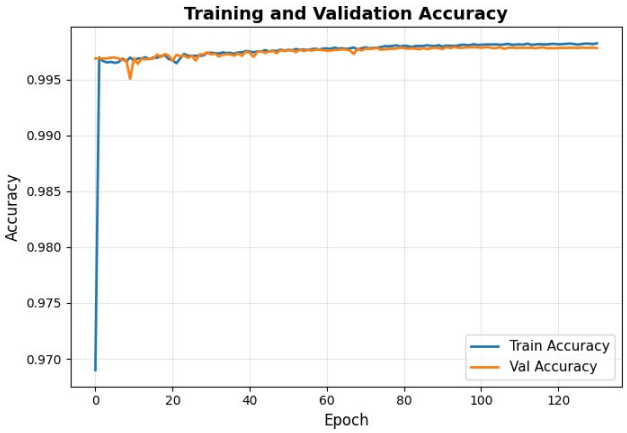
Plotting Attention U-Net training history...
```






```
Plotting HRNet training history...
```





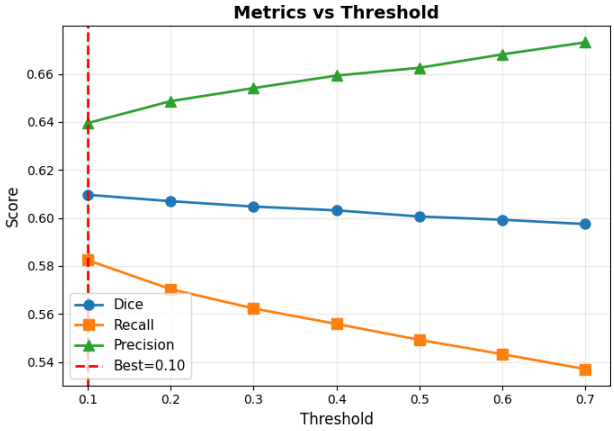

```
Evaluating Attention U-Net model...

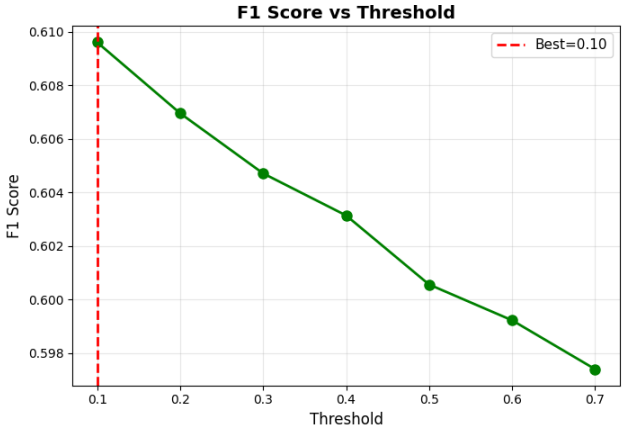
================================================================================
MULTI-THRESHOLD EVALUATION
================================================================================
Generating predictions...

Threshold   Dice      IoU       Recall    Precision F1
--------------------------------------------------------------------------------
0.10        0.6096    0.4384    0.5824    0.6395    0.6096
0.20        0.6070    0.4357    0.5703    0.6486    0.6070
```

```
0.30        0.6047    0.4334    0.5623    0.6541    0.6047
0.40        0.6031    0.4318    0.5558    0.6593    0.6031
0.50        0.6006    0.4291    0.5492    0.6625    0.6006
0.60        0.5992    0.4278    0.5432    0.6681    0.5992
0.70        0.5974    0.4259    0.5370    0.6731    0.5974

======================================================================
BEST THRESHOLD: 0.10
  Dice: 0.6096
  IoU: 0.4384
  Recall: 0.5824
  Precision: 0.6395
  F1: 0.6096
======================================================================
```





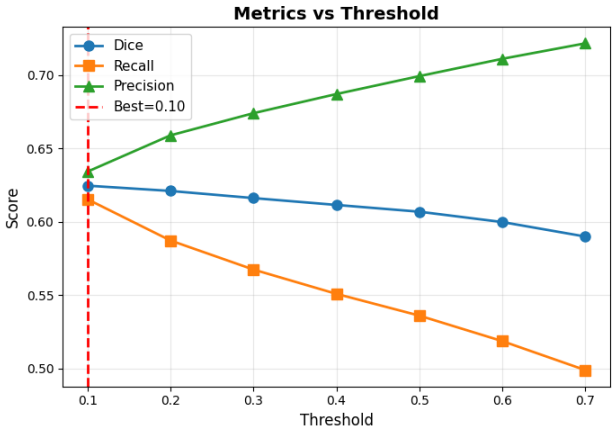

```
Evaluating HRNet model...

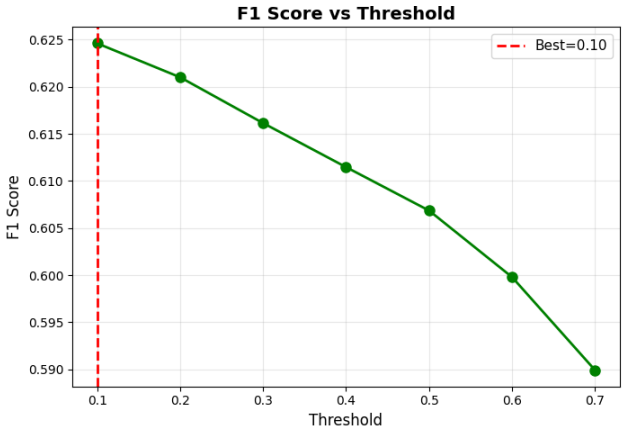
======================================================================
MULTI-THRESHOLD EVALUATION
======================================================================
Generating predictions...

Threshold   Dice      IoU       Recall    Precision F1
----------------------------------------------------------------------
0.10        0.6246    0.4541    0.6153    0.6342    0.6246
0.20        0.6210    0.4503    0.5872    0.6589    0.6210
0.30        0.6161    0.4452    0.5674    0.6740    0.6161
0.40        0.6115    0.4404    0.5509    0.6871    0.6115
0.50        0.6069    0.4356    0.5360    0.6993    0.6069
0.60        0.5998    0.4284    0.5186    0.7111    0.5998
0.70        0.5899    0.4184    0.4989    0.7216    0.5899

======================================================================
BEST THRESHOLD: 0.10
```

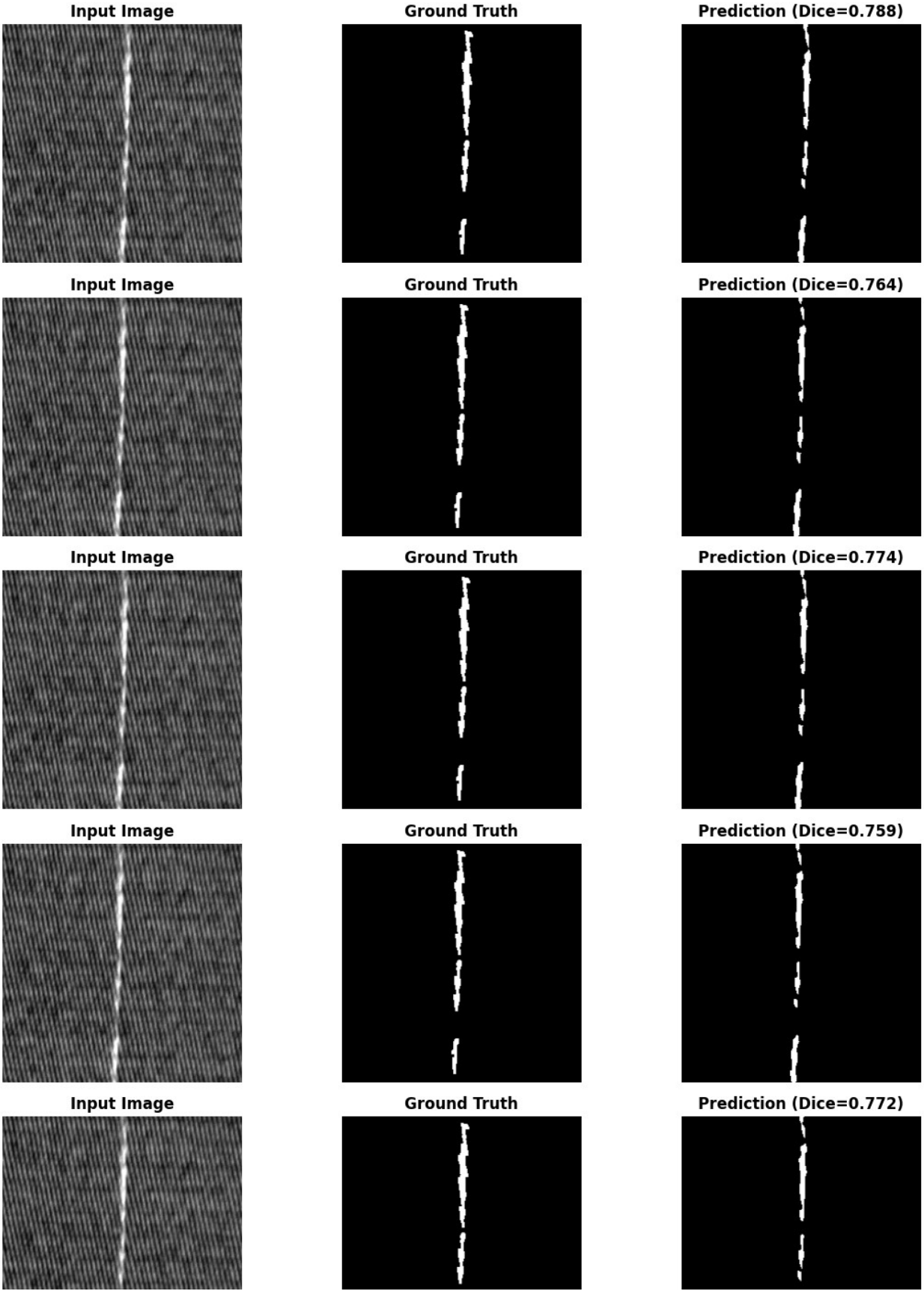
```
  Dice: 0.6246
  IoU: 0.4541
  Recall: 0.6153
  Precision: 0.6342
  F1: 0.6246
==========================================================================
```

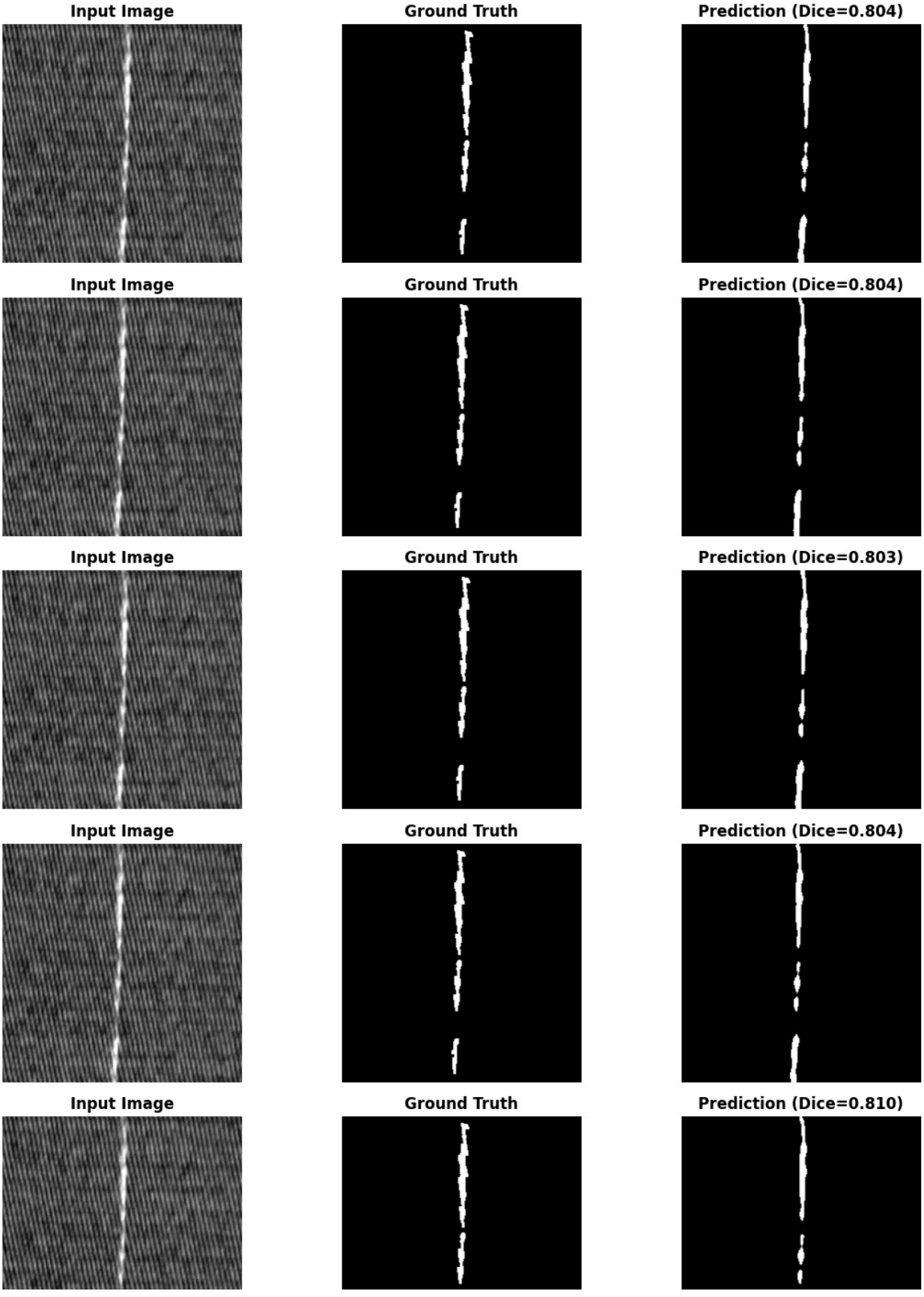





```
Attention U-Net Best Metrics: {'dice': np.float32(0.60960954), 'iou':
np.float32(0.43844482), 'recall': np.float32(0.58240646), 'precision':
np.float32(0.63947827), 'f1': np.float32(0.6096095), 'tp': 32309,
'fp': 18215, 'fn': 23166, 'tn': 17752102}
HRNet Best Metrics: {'dice': np.float32(0.6246066), 'iou':
np.float32(0.45412943), 'recall': np.float32(0.61534023), 'precision':
np.float32(0.6341563), 'f1': np.float32(0.62460655), 'tp': 34136,
'fp': 19693, 'fn': 21339, 'tn': 17750624}

visualize_predictions(attention_unet_model, val_dataset,
num_samples=6, threshold=best_thresh_unet)
visualize_predictions(hrnet_model, val_dataset, num_samples=6,
threshold=best_thresh_hrnet)
```


Input Image
Ground Truth
Prediction (Dice=0.788)
Input Image
Ground Truth
Prediction (Dice=0.764)
Input Image
Ground Truth
Prediction (Dice=0.774)
Input Image
Ground Truth
Prediction (Dice=0.759)
Input Image
Ground Truth
Prediction (Dice=0.772)


Input Image
Ground Truth
Prediction (Dice=0.804)
Input Image
Ground Truth
Prediction (Dice=0.804)
Input Image
Ground Truth
Prediction (Dice=0.803)
Input Image
Ground Truth
Prediction (Dice=0.804)
Input Image
Ground Truth
Prediction (Dice=0.810)

```python
import os
import cv2
import matplotlib.pyplot as plt
import numpy as np
import shutil

def save_prediction_results(model, dataset_obj,
output_folder="Model_Sonuclari", num_samples=20, threshold=0.5):
    """
    Model tahminlerini görselleştirir ve klasöre kaydeder.
    """
    # 1. Klasör Yapısını Oluştur
    plot_dir = os.path.join(output_folder, "karsilastirma_tablolari")
    mask_dir = os.path.join(output_folder, "saf_tahmin_maskeleri")

    os.makedirs(plot_dir, exist_ok=True)
    os.makedirs(mask_dir, exist_ok=True)

    print(f"📂 Kayıt İşlemi Başladı... Hedef Klasör: {output_folder}")

    count_saved = 0

    # 2. Döngü
    for i in range(min(num_samples, len(dataset_obj))):
        # Veriyi al
        img, true_mask = dataset_obj.get_item(i)

        # Model Tahmini
        input_img = np.expand_dims(img, axis=0)
        pred_prob = model.predict(input_img, verbose=0)[0] # Olasılık
haritası (0.0 - 1.0)

        # Binary Maske (Eşiklenmiş - 0 veya 1)
        pred_binary = (pred_prob > threshold).astype(np.float32)

        # --- A) KARŞILAŞTIRMA GRAFİĞİNİ KAYDET ---
        fig, axes = plt.subplots(1, 3, figsize=(15, 5))

        # Orijinal
        axes[0].imshow(img[:,:,0], cmap='gray')
        axes[0].set_title(f"Örnek #{i} - Orijinal")
        axes[0].axis('off')

        # Gerçek Maske
        axes[1].imshow(true_mask[:,:,0], cmap='gray', vmin=0, vmax=1)
        axes[1].set_title("Gerçek Maske (Ground Truth)")
        axes[1].axis('off')

        # Model Tahmini (Renkli Olasılık)
        # Olasılıkları görelim ki modelin ne kadar emin olduğunu
```

```
anlayalım
        axes[2].imshow(pred_prob[:,:,0], cmap='inferno', vmin=0,
vmax=1)
        axes[2].set_title(f"Model Tahmini\n(Max Güven:
{pred_prob.max():.2f})")
        axes[2].axis('off')

        # Grafiği Kaydet
        plot_path = os.path.join(plot_dir, f"result_{i:03d}_plot.png")
        plt.savefig(plot_path, bbox_inches='tight')
        plt.close(fig) # Hafızayı şişirmemek için kapat

        # --- B) SAF MASKEYİ KAYDET (PNG) ---
        # 0-1 arası değeri 0-255 yapıp kaydediyoruz
        mask_path = os.path.join(mask_dir, f"result_{i:03d}_mask.png")
        cv2.imwrite(mask_path, (pred_binary * 255).astype(np.uint8))

        count_saved += 1
        if (i+1) % 10 == 0:
            print(f"   -> {i+1} adet görsel işlendi...")

    print(f"\n İŞLEM TAMAM! Toplam {count_saved} görsel
'{output_folder}' içine kaydedildi.")

    # 3. Klasörü İndirmek İçin Zip Yap (Kaggle/Colab için çok yararlı)
    shutil.make_archive(output_folder, 'zip', output_folder)
    print(f" '{output_folder}.zip' dosyası oluşturuldu.
İndirebilirsiniz!")

# --- KULLANIM ---
# Hangi modeli ve dataseti kullanmak istiyorsan buraya yaz
# Öneri: Validation setini kullan ki modelin görmediği verideki
başarısını gör.

save_prediction_results(
    model=attention_unet_model,       # Eğittiğin model ismi
    dataset_obj=val_dataset_obj,      # Validation datasetin
    output_folder="AITex_Sonuclar",   # Klasör ismi
    num_samples=50,                   # Kaç resim kaydedilsin?
    threshold=0.5                     # Hangi değerin üstü hata
sayılsın?
)

 Kayıt İşlemi Başladı... Hedef Klasör: AITex_Sonuclar

2026-01-31 22:00:56.884261: E
external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay
kernel timed out: measured time has sub-optimal accuracy. There may be
a missing warmup execution, please investigate in Nsight Systems.
2026-01-31 22:00:57.107412: E
```

```
external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay
kernel timed out: measured time has sub-optimal accuracy. There may be
a missing warmup execution, please investigate in Nsight Systems.

   -> 10 adet görsel işlendi...
   -> 20 adet görsel işlendi...
   -> 30 adet görsel işlendi...
   -> 40 adet görsel işlendi...
   -> 50 adet görsel işlendi...

 İŞLEM TAMAM! Toplam 50 görsel 'AITex_Sonuclar' içine kaydedildi.
 'AITex_Sonuclar.zip' dosyası oluşturuldu. İndirebilirsiniz!
```

```python
import pandas as pd
import matplotlib.pyplot as plt
import os

def save_training_metrics(history, model_name="Model",
output_folder="Model_Metrikleri"):
    """
    Eğitim geçmişini (history) CSV olarak kaydeder ve grafiklerini
çizer.
    """
    # 1. Klasörü Oluştur
    os.makedirs(output_folder, exist_ok=True)
    print(f" Metrik Kaydı Başladı... Hedef: {output_folder}")

    # 2. Verileri DataFrame'e Çevir ve Kaydet (Excel/CSV)
    # history.history bir sözlüktür, bunu tabloya çeviriyoruz
    df = pd.DataFrame(history.history)

    # En iyi epoch'u bul (Dice skoruna göre)
    if 'val_dice_coef' in df.columns:
        best_epoch = df['val_dice_coef'].idxmax()
        print(f"    En İyi Epoch: {best_epoch + 1} (Val Dice:
{df.loc[best_epoch, 'val_dice_coef']:.4f})")

    # CSV Dosyası Olarak Kaydet
    csv_path = os.path.join(output_folder,
f"{model_name}_training_log.csv")
    df.to_csv(csv_path, index_label="Epoch")
    print(f"    Tablo kaydedildi: {csv_path}")

    # 3. Grafikleri Çiz ve Kaydet
    metrics_to_plot = [
        ('loss', 'val_loss', 'Loss (Kayıp) Grafiği', 'min'),
        ('dice_coef', 'val_dice_coef', 'Dice Skoru (Başarı)', 'max'),
        ('accuracy', 'val_accuracy', 'Accuracy (Doğruluk)', 'max')
    ]
```

```
    plt.figure(figsize=(18, 5))

    for i, (train_metric, val_metric, title, mode) in
enumerate(metrics_to_plot):
        if train_metric not in df.columns: continue

        plt.subplot(1, 3, i+1)
        plt.plot(df[train_metric], label=f'Eğitim {train_metric}',
linewidth=2)
        plt.plot(df[val_metric], label=f'Doğrulama {val_metric}',
linewidth=2, linestyle='--')

        # En iyi noktayı işaretle
        if mode == 'min':
            best_idx = df[val_metric].idxmin()
            best_val = df[val_metric].min()
            color = 'red'
        else:
            best_idx = df[val_metric].idxmax()
            best_val = df[val_metric].max()
            color = 'green'

        plt.scatter(best_idx, best_val, c=color, s=100, zorder=5,
label=f'En İyi: {best_val:.4f}')

        plt.title(f"{model_name} - {title}")
        plt.xlabel("Epoch")
        plt.ylabel("Değer")
        plt.legend()
        plt.grid(True, alpha=0.3)

    # Grafiği Kaydet
    plot_path = os.path.join(output_folder,
f"{model_name}_metrics_plot.png")
    plt.savefig(plot_path, bbox_inches='tight')
    plt.close()
    print(f"   □ Grafik kaydedildi: {plot_path}")
    print("-" * 50)

# --- KULLANIM ---

# 1. Attention U-Net için Kaydet (Eğitim değişkenin adı 'history_unet'
ise)
# Eğer değişken adın farklıysa (örneğin sadece 'history') onu yaz.
if 'history_unet' in globals():
    save_training_metrics(history_unet, model_name="Attention_UNet")
elif 'history' in globals(): # Genelde sadece history kullanılır
    save_training_metrics(history, model_name="Attention_UNet")
else:
```

```
    print("⚠ 'history' değişkeni bulunamadı. Eğitimi 'history =
model.fit(...)' şeklinde başlattığından emin ol.")

# 2. HRNet için Kaydet (Varsa)
if 'history_hrnet' in globals():
    save_training_metrics(history_hrnet, model_name="HRNet")

# 3. Tüm dosyaları indirmek için zip yap
import shutil
shutil.make_archive("Model_Metrikleri", 'zip', "Model_Metrikleri")
print("\n 'Model_Metrikleri.zip' hazır! İndirebilirsin.")
```

```
⚠ 'history' değişkeni bulunamadı. Eğitimi 'history = model.fit(...)'
şeklinde başlattığından emin ol.
 Metrik Kaydı Başladı... Hedef: Model_Metrikleri
   En İyi Epoch: 101 (Val Dice: 0.4193)
   Tablo kaydedildi: Model_Metrikleri/HRNet_training_log.csv
   Grafik kaydedildi: Model_Metrikleri/HRNet_metrics_plot.png
--------------------------------------------------

 'Model_Metrikleri.zip' hazır! İndirebilirsin.
```